

FIM590-003

Iowa House Price Prediction

Linear, Ridge, and Lasso Regression on a Log-Price Target

Group members: Joy Wu, Jialin Zhang, Luolin Zhang
Course: FIM590-003
Submission date: October 6, 2026
Data set: Iowa House Price Original Data

Abstract

This project develops a reproducible linear-modeling workflow for Iowa house prices. The data contain 2,908 usable observations. Following the project specification, the exploratory analysis documents the target distribution, univariate predictor relationships, neighborhood price differences, and correlation among physical size measures. We construct 47 predictors, allocate 1,800 observations to training, 600 to validation, and 508 to a held-out test set, and compare ordinary least squares (OLS), three Ridge models, and three Lasso models. All models are estimated using $\log(1 + \text{SalePrice})$ as the response. The validation set selects OLS with a validation mean squared error (MSE) of 0.014917, although Ridge with $\alpha = 0.60$ is nearly tied at 0.014929. After refitting OLS on the combined 2,400-observation development sample, the test-set results are MSE 0.014423, root mean squared error (RMSE) 0.120097, mean absolute error (MAE) 0.086159, and $R^2 = 0.908726$ on the log-price scale. The corresponding dollar-scale RMSE and MAE are approximately \$21,545 and \$14,997. Because the prescribed feature set contains exact linear dependencies, OLS predictions are valid but individual OLS coefficients are not uniquely identified. We therefore interpret coefficient patterns using the nearly tied Ridge model and treat all coefficients as predictive associations rather than causal effects.

Contents

1	Project Objective	1
2	Data Preparation	1
2.1	Source data and cleaning	1
2.2	Exploratory data analysis	1
2.2.1	Target distribution	2
2.2.2	Univariate predictor relationships	2
2.2.3	Correlation structure	5
2.3	Data split	6
3	Modeling Methodology	6
3.1	Preprocessing and target transformation	6
3.2	Ordinary least squares	6
3.3	Ridge regression	7
3.4	Lasso regression	7
3.5	Evaluation metrics	7
4	Validation Results and Model Selection	7
4.1	Coefficient identification issue	9
5	Held-Out Test Performance	9
6	Stable Coefficient Patterns	11
7	Optional Zillow Listing Application	11
7.1	Historical record and current feature mapping	12
7.2	Historical holdout check	13
7.3	Current prediction and comparison	13
8	Limitations and Possible Extensions	13
9	Conclusion	14
A	Complete List of 47 Features	15
B	Selected-Model Parameter Vector	16
C	Reproducibility Algorithm	17

1 Project Objective

The objective is to predict Iowa residential sale prices with the 47-feature specification from the course material. The required workflow has four parts:

1. clean the original spreadsheet and construct the prescribed regressors;
2. create fixed training, validation, and test samples;
3. estimate OLS, Ridge, and Lasso models and select a model using validation MSE;
4. refit the selected specification on the development data and evaluate it once on the held-out test set.

The central modeling decision is to predict log price rather than dollar price. House prices are positive and right-skewed, so the transformation $\log(1 + y)$ reduces the influence of the most expensive houses and makes squared error closer to a proportional-error criterion. It also keeps the transformation defined at zero.

2 Data Preparation

2.1 Source data and cleaning

The source workbook contains 2,908 observations and 81 substantive columns after removing empty spreadsheet columns. Cleaning and feature construction follow these rules:

- strip whitespace from column names and remove unnamed empty columns;
- construct `BsmtFinSF` as `BsmtFinSF1 + BsmtFinSF2`;
- encode basement quality as `Ex = 5`, `Gd = 4`, `TA = 3`, `Fa = 2`, `Po = 1`, `NA = 0`;
- one-hot encode all 25 observed neighborhood categories;
- retain the 21 numerical variables, one basement-quality ordinal variable, and 25 neighborhood indicators listed in [Appendix A](#); and
- verify that the selected predictors and target contain no missing values.

The resulting design matrix has dimension 2908×47 . The intercept is estimated separately and is not counted among the 47 predictors.

2.2 Exploratory data analysis

The EDA figures are the saved outputs from `notebooks/01_eda.ipynb` and use all 2,908 observations before the train-validation-test split. They are descriptive analyses that motivate the model specification. They do not establish causal effects or replace the out-of-sample evaluation.

2.2.1 Target distribution

Figure 1 compares the raw sale-price distribution with the transformed modeling target. Sale prices range from \$12,789 to \$625,000. The mean is \$180,272, compared with a median of \$160,000, and the raw-price skewness is 1.574. A small number of expensive properties creates a long right tail and has a large influence on squared dollar errors. After applying $\log(1 + \text{SalePrice})$, skewness falls to 0.038 and the distribution is approximately symmetric. The transformation compresses the upper tail and supports the log-price target used in the regression models.

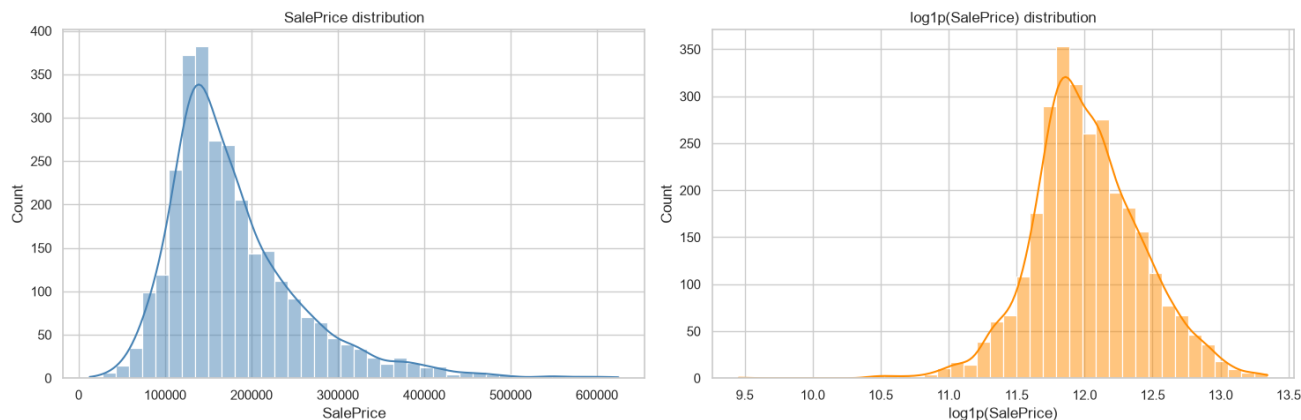


Figure 1: SalePrice and $\log(1 + \text{SalePrice})$ distributions for all 2,908 observations.

2.2.2 Univariate predictor relationships

Figure 2 shows each continuous regressor against SalePrice. Larger above-ground living area, newer construction or remodeling, and larger basement and garage areas generally align with higher sale prices. Above-ground living area has the strongest continuous-variable Pearson correlation with SalePrice (0.721), followed by total basement area (0.659), garage area (0.651), and first-floor area (0.640). Dispersion also increases for more expensive homes. These are univariate relationships and do not control for location, quality, or the other property characteristics.

Continuous regressors versus SalePrice

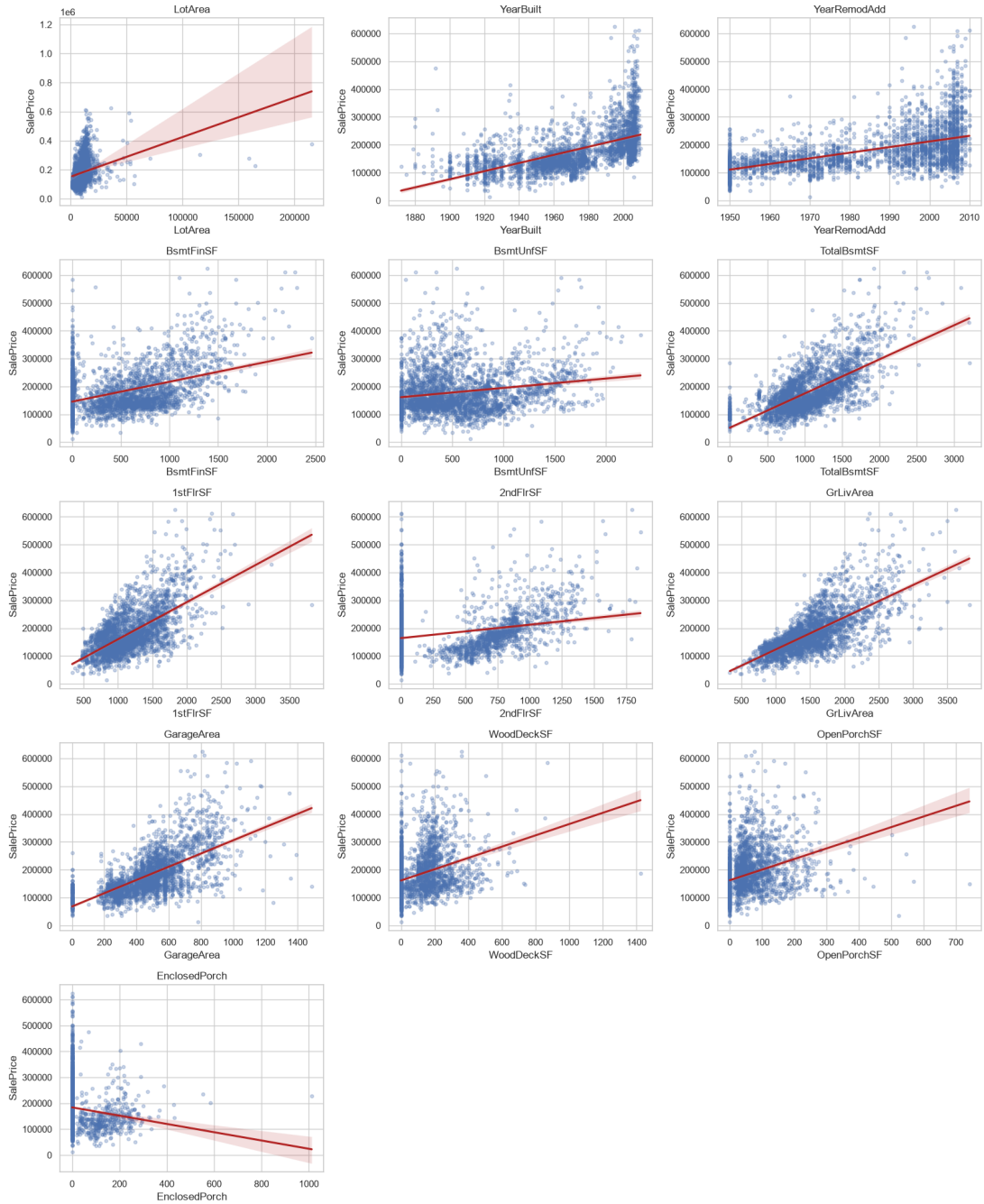


Figure 2: Continuous regressors versus SalePrice, with descriptive fitted lines.

Figure 3 summarizes the count and ordinal predictors. Overall quality, basement quality, garage capacity, number of fireplaces, and several room-count measures separate the observed sale-price distributions. The patterns are clearest for `OverallQual` and `BsmtQual_ordinal`. Sparse categories at the highest counts should be interpreted cautiously because their boxplots represent few observations.

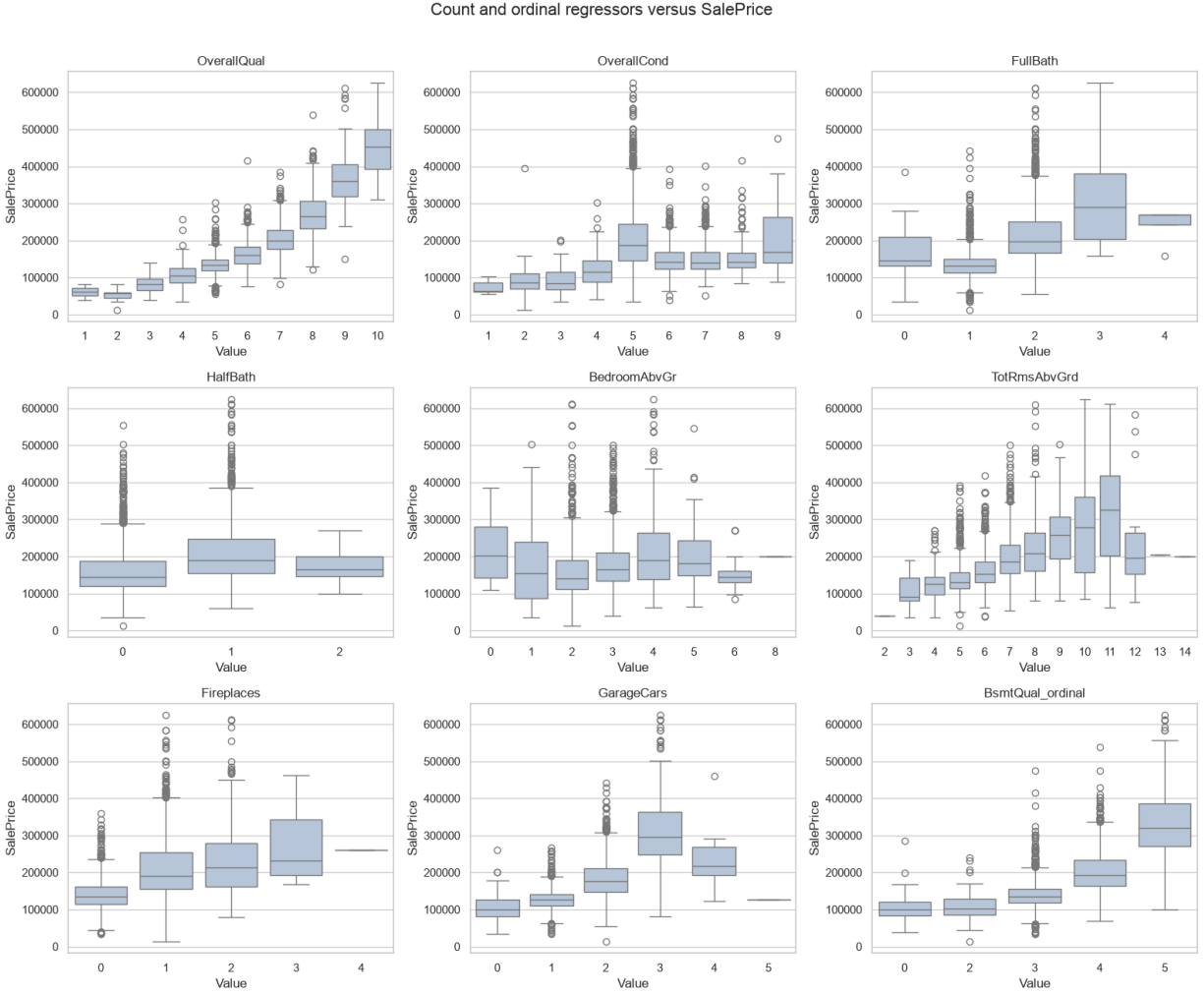


Figure 3: Count and ordinal regressors versus SalePrice.

Location also has a strong descriptive relationship with price. Figure 4 reports median SalePrice by neighborhood and labels each bar with its sample size. Stone Brook, Northridge Heights, and Northridge have the highest observed medians, whereas Meadow Village has the lowest. These differences support retaining neighborhood indicators, while the sample-size labels caution against overinterpreting small groups.

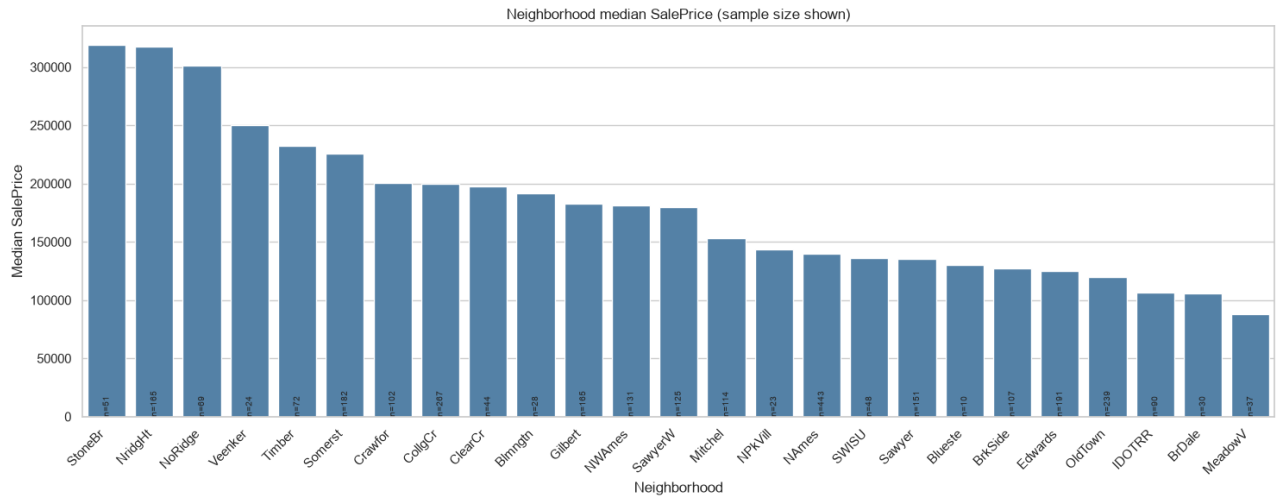


Figure 4: Median SalePrice by neighborhood, with sample sizes.

2.2.3 Correlation structure

Figure 5 displays Pearson correlations among the 13 continuous regressors used in the EDA notebook. Several physical-size variables move together. Total basement area and first-floor area have a correlation of 0.789, second-floor area and above-ground living area have a correlation of 0.664, and year built and year remodeled have a correlation of 0.612. This structure helps explain the instability of individual OLS coefficients and motivates the Ridge sensitivity analysis.

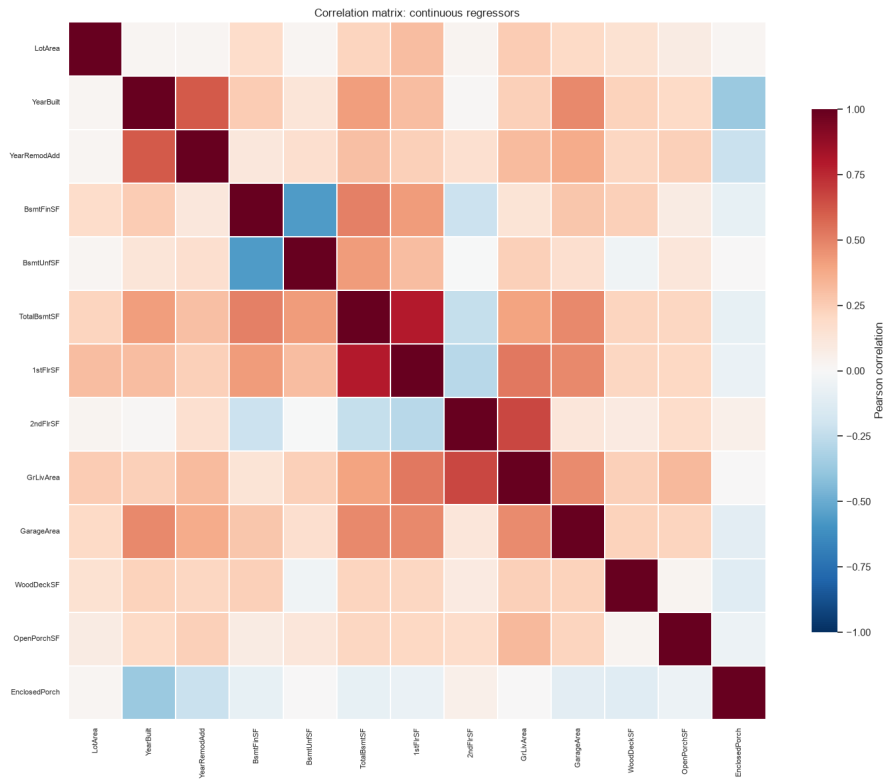


Figure 5: Pearson-correlation heatmap for the 13 continuous regressors.

2.3 Data split

A NumPy random generator with seed 20260928 produces one shuffled permutation of the observations. The first 1,800 observations form the training set, the next 600 form the validation set, and the remaining 508 form the test set. The test set is not used in model or penalty selection.

Table 1: Sample allocation and mean sale price.

Sample	Observations	Mean SalePrice (USD)
Training	1,800	180,018.88
Validation	600	178,133.52
Test	508	183,695.88

The similar mean prices in Table 1 indicate that the shuffled split did not produce an obvious price-level imbalance. Model selection nevertheless uses out-of-sample validation metrics rather than this descriptive comparison.

3 Modeling Methodology

3.1 Preprocessing and target transformation

Within each fit, the 21 numerical predictors and ordinal basement-quality variable are standardized using means and standard deviations learned only from that fit’s training sample. Neighborhood indicators remain 0/1. For a numerical feature x_j , the standardized value is

$$x_{ij}^* = \frac{x_{ij} - \bar{x}_j}{s_j}. \quad (1)$$

Let y_i be dollar sale price and let $\ell_i = \log(1 + y_i)$. The fitting pipeline also standardizes ℓ_i ,

$$z_i = \frac{\ell_i - \bar{\ell}}{s_\ell}, \quad (2)$$

and reverses both target transformations after prediction. Reported primary metrics are calculated on ℓ_i , not on the standardized target z_i . This distinction makes the reported MSE, RMSE, and MAE interpretable across fits.

3.2 Ordinary least squares

For design matrix X , response vector z , and intercept b_0 , OLS solves

$$\min_{b_0, \beta} \sum_{i=1}^n \left(z_i - b_0 - \mathbf{x}_i^\top \beta \right)^2. \quad (3)$$

When the columns are linearly independent, the normal-equation solution after centering is

$$\hat{\beta} = (X^\top X)^{-1} X^\top z. \quad (4)$$

The implemented numerical solver uses a least-squares decomposition and can also produce fitted values for a rank-deficient design. As discussed in Section 4.1, the fitted values can be well defined even when the individual coefficient vector is not unique.

3.3 Ridge regression

Ridge regression adds an L_2 penalty to shrink coefficients:

$$\min_{b_0, \beta} \|z - b_0 \mathbf{1} - X\beta\|_2^2 + \alpha \|\beta\|_2^2. \quad (5)$$

The intercept is not penalized. We estimate the three required values $\alpha \in \{0.10, 0.30, 0.60\}$. The penalty makes the coefficient solution unique and stable under the exact linear dependencies present in the feature set.

3.4 Lasso regression

Lasso uses an L_1 penalty and can set coefficients exactly to zero:

$$\min_{b_0, \beta} \frac{1}{2n} \|z - b_0 \mathbf{1} - X\beta\|_2^2 + \alpha \|\beta\|_1. \quad (6)$$

We estimate $\alpha \in \{0.02, 0.06, 0.10\}$ with a maximum of 50,000 coordinate-descent iterations. The loss scaling in Equation 6 follows the software implementation, so numerical α values should not be compared directly between Ridge and Lasso as if they represented identical penalty strength.

3.5 Evaluation metrics

For actual log prices ℓ_i , predicted log prices $\hat{\ell}_i$, and sample size m , the model-comparison statistics are

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\ell_i - \hat{\ell}_i)^2, \quad (7)$$

$$\text{RMSE} = \sqrt{\text{MSE}}, \quad (8)$$

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |\ell_i - \hat{\ell}_i|, \quad (9)$$

$$R^2 = 1 - \frac{\sum_i (\ell_i - \hat{\ell}_i)^2}{\sum_i (\ell_i - \bar{\ell})^2}. \quad (10)$$

Validation MSE is the sole model-selection criterion. Dollar RMSE and MAE are reported only after selection as intuitive supplementary measures.

4 Validation Results and Model Selection

Table 2: Training and validation metrics on the log-price scale.

Model	Train MSE	Train RMSE	Train R^2	Val. MSE	Val. RMSE	Val. R^2
OLS	0.013908	0.117930	0.912059	0.014917	0.122137	0.914905
Ridge, $\alpha = 0.60$	0.013904	0.117914	0.912083	0.014929	0.122186	0.914837
Ridge, $\alpha = 0.30$	0.013902	0.117908	0.912092	0.014933	0.122201	0.914816
Ridge, $\alpha = 0.10$	0.013902	0.117906	0.912094	0.014936	0.122213	0.914798
Lasso, $\alpha = 0.02$	0.016669	0.129110	0.894595	0.017505	0.132305	0.900147
Lasso, $\alpha = 0.06$	0.019338	0.139063	0.877718	0.021760	0.147513	0.875871
Lasso, $\alpha = 0.10$	0.022969	0.151554	0.854762	0.026563	0.162982	0.848473

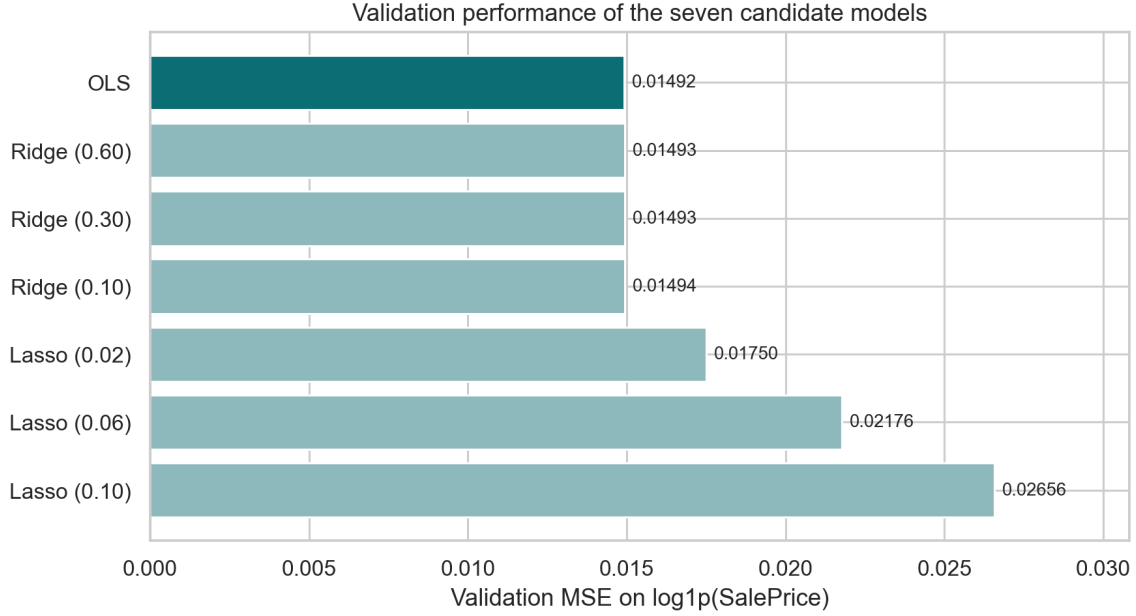


Figure 6: Validation MSE for all seven candidate models. Lower is better.

OLS has the smallest validation MSE and is therefore selected under the stated rule. Ridge with $\alpha = 0.60$ is only 0.000012 MSE units, or approximately 0.08%, above OLS. This is a practically negligible difference for this split. The Lasso models become progressively less accurate as α increases, which indicates that the stronger sparsity penalties discard useful predictive signal in this 47-feature specification.

Table 3: Practical comparison of OLS and Ridge regression.

Model	Advantages	Limitations
OLS	Achieves the lowest validation MSE (0.014917) and therefore satisfies the predefined selection rule. Its test performance is also strong, with $R^2 = 0.9087$.	The feature matrix is rank deficient, so individual OLS coefficients are not uniquely identified and should not be interpreted.
Ridge, $\alpha = 0.60$	Produces unique and substantially more stable coefficients under multicollinearity. Its validation performance is nearly identical to OLS, making it more suitable for examining coefficient patterns.	Its validation MSE (0.014929) is slightly higher than OLS. Ridge also introduces shrinkage bias, and coefficient magnitudes depend on feature scaling and the selected penalty.

The selected model has no regularization, so its method is OLS and its effective penalty is $\alpha = 0$. It is refitted on the combined training and validation sample of 2,400 observations before the single test evaluation.

4.1 Coefficient identification issue

The prescribed 47-feature matrix contains two exact dependencies:

$$\text{TotalBsmtSF} = \text{BsmtFinSF} + \text{BsmtUnfSF}, \quad (11)$$

$$\sum_{k=1}^{25} \text{Neighborhood}_{ik} = 1. \quad (12)$$

The second equality duplicates the intercept. Consequently, many different OLS coefficient vectors produce nearly the same fitted values. The numerical solution contains very large positive and negative coefficients that cancel, as shown in Appendix B. These values are a rank-deficiency artifact rather than evidence that a one-unit change in a feature changes log price by billions.

This issue does not invalidate the reported OLS prediction metrics. It does mean that the individual OLS slopes must not be interpreted. For a stable sensitivity view, Figure 9 uses Ridge with $\alpha = 0.60$, the closest validation competitor. A production model would normally remove one neighborhood indicator and one redundant basement-area variable, or use regularization.

5 Held-Out Test Performance

Table 4: Final performance of OLS refitted on 2,400 development observations.

Scale	Metric	Test value
$\log(1 + \text{SalePrice})$	MSE	0.014423
$\log(1 + \text{SalePrice})$	RMSE	0.120097
$\log(1 + \text{SalePrice})$	MAE	0.086159
$\log(1 + \text{SalePrice})$	R^2	0.908726
Dollars	RMSE	\$21,545.45
Dollars	MAE	\$14,996.77

The model explains approximately 90.9% of the variation in held-out log prices. An RMSE of 0.1201 log units corresponds roughly to a multiplicative error scale of $\exp(0.1201) - 1 \approx 12.8\%$, although RMSE is not itself a median percentage error. Dollar errors are larger for expensive houses, so the dollar RMSE exceeds the dollar MAE by about \$6,549.

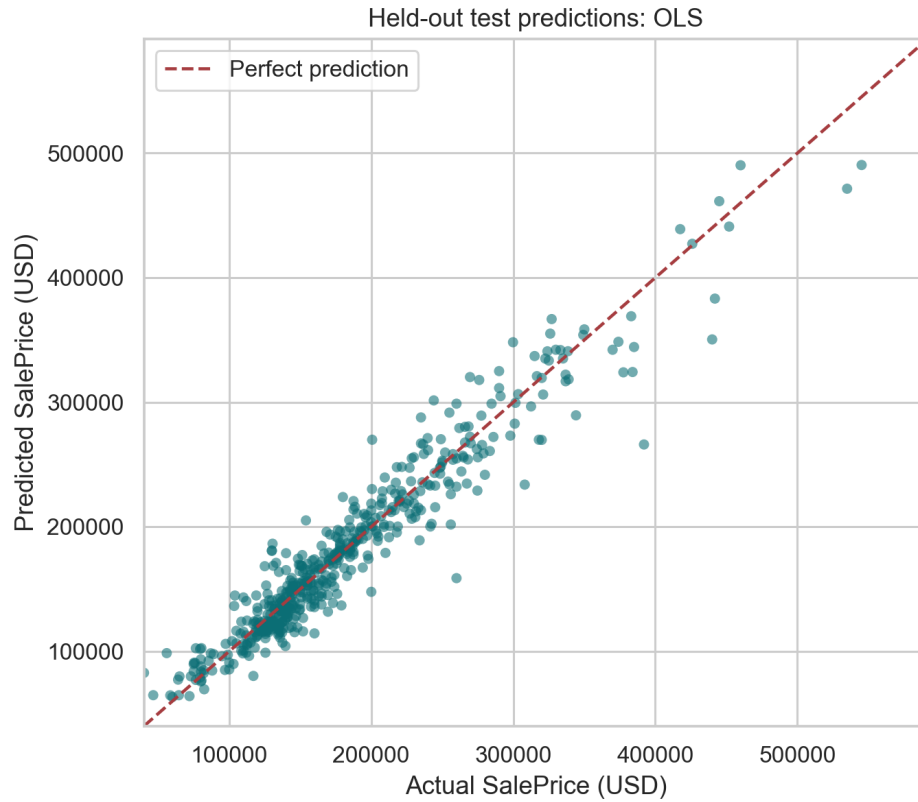


Figure 7: Actual and predicted sale prices for the 508 held-out test observations.

Figure 7 shows that most observations lie near the 45-degree reference line. The dispersion widens in dollar terms for expensive properties, which is consistent with modeling proportional rather than absolute dollar error.

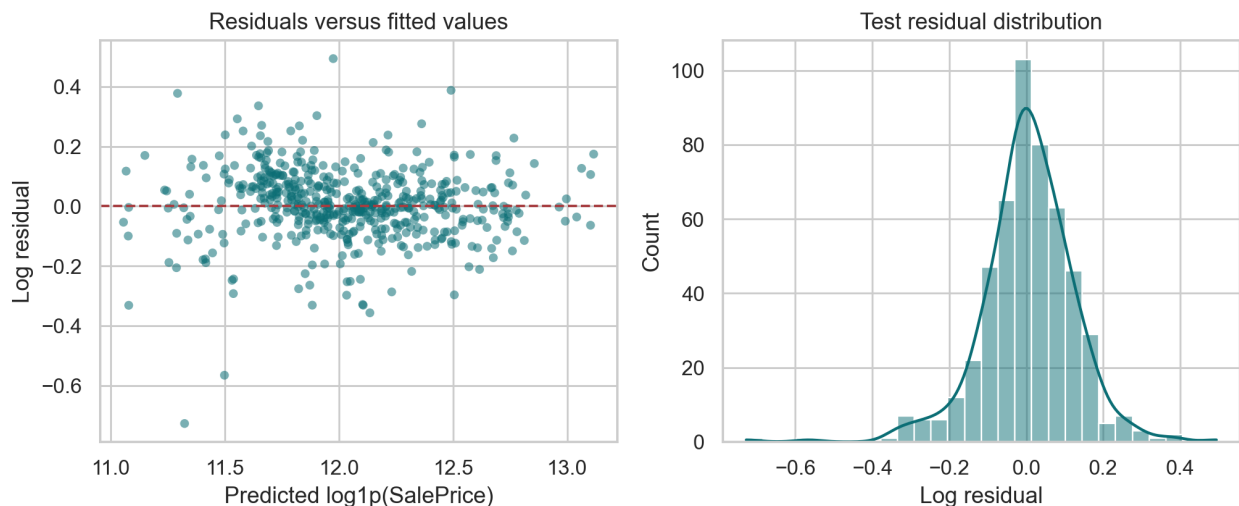


Figure 8: Test-set residual diagnostics on the log-price scale. Residual equals actual minus predicted log price.

The residual distribution is centered near zero, but several observations have large positive or negative errors. The residual-versus-fitted plot also suggests some nonconstant dispersion and local structure. These patterns are plausible for a linear model with a limited feature set and motivate the extensions

discussed in Section 8.

6 Stable Coefficient Patterns

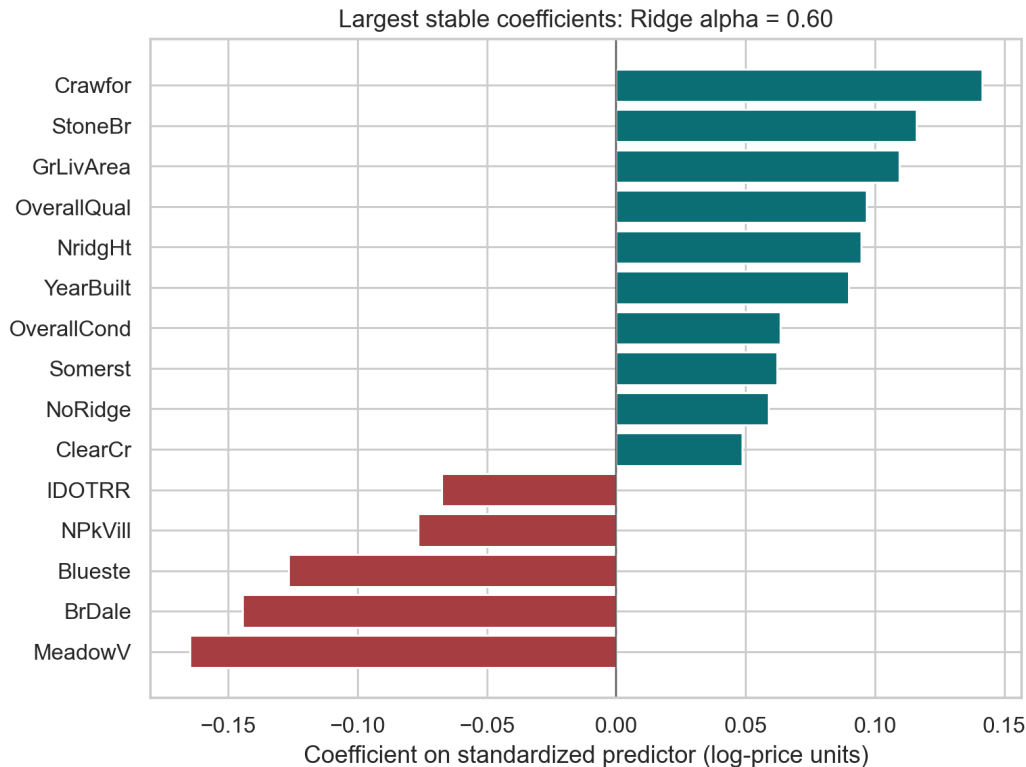


Figure 9: Largest standardized coefficients from Ridge with $\alpha = 0.60$, refitted on the 2,400-observation development sample.

The Ridge sensitivity model assigns large positive predictive associations to above-ground living area, overall quality, newer construction, and several neighborhood indicators, including Crawford, Stone Brook, and Northridge Heights. Meadow Village, Briardale, Bluestem, Northpark Villa, and Iowa DOT and Rail Road have negative associations conditional on the other included variables. The bars for standardized numerical predictors represent a one-standard-deviation change; neighborhood indicators remain 0/1. The magnitudes are therefore useful for pattern comparison but not identical unit changes across all feature types.

These are conditional predictive associations. They do not estimate causal effects of neighborhood, quality, renovation, or size because property characteristics and location are jointly determined and the design is observational.

7 Optional Zillow Listing Application

To compare the historical model with a current market value, we selected the active listing at 124 North Franklin Avenue, Ames, Iowa 50014. On October 4, 2026, Zillow reported a listing price of \$246,500 and a Zestimate of \$242,000.¹ The same parcel (0905476120) appears in public sale history at \$119,000 on

¹[Zillow listing page for 124 North Franklin Avenue.](#)

April 16, 2008 and \$186,820 on April 13, 2022. The current listing describes a four-bedroom, two-bath house with 1,262 square feet, a partial finished basement, and a one-car detached garage.²

This property is especially useful because its 2008 transaction is an exact match to observation `Id=391` in the supplied course data. Under random seed 20260928, this record belongs to the held-out test set, so neither the final OLS model nor the Ridge sensitivity model was fitted on this house. The exercise can therefore evaluate the model first at the historical sale date and then apply the same model to a transparent mapping of the current listing.

7.1 Historical record and current feature mapping

Table 5 shows how the exact course record was adapted. Values not contradicted by the current listing were retained from the historical record. The listing reports one full bath and one three-quarter bath; because the course features distinguish only full and half baths, both were mapped to two full baths. The listing’s fourth bedroom is in the basement, so `BedroomAbvGr` was set to three. The reported 1,262 square feet was treated as above-grade living area, with the historical first-floor area retained and the second-floor area adjusted to the difference.

Table 5: Exact 2008 record and mapping to the current listing.

Model input	2008 record	Current input	Mapping rule
<code>LotArea</code>	8,405	8,405	Retained from the exact course record
<code>OverallQual</code> / <code>OverallCond</code>	5 / 8	5 / 8	Retained
<code>YearBuilt</code> / <code>YearRemodAdd</code>	1900 / 1950	1900 / 1950	Retained
<code>BsmtFinSF</code> / <code>BsmtUnfSF</code>	632 / 229	632 / 229	Retained
<code>TotalBsmtSF</code>	861	861	Retained
<code>1stFlrSF</code> / <code>2ndFlrSF</code>	961 / 406	961 / 301	Upper floor adjusted to current total
<code>GrLivArea</code>	1,367	1,262	Current listing
<code>FullBath</code> / <code>HalfBath</code>	1 / 0	2 / 0	One full plus one three-quarter mapped to two full
<code>BedroomAbvGr</code> / <code>TotRmsAbvGrd</code>	4 / 7	3 / 7	Basement bedroom excluded from above-grade count
<code>Fireplaces</code> / <code>GarageCars</code>	0 / 1	0 / 1	Retained; one-car garage confirmed
<code>GarageArea</code>	384	384	Retained
<code>WoodDeck</code> / <code>OpenPorch</code> / <code>EnclosedPorch</code>	0 / 130 / 112	0 / 130 / 112	Retained
<code>BsmtQual_ordinal</code>	3	3	Retained
<code>Neighborhood</code>	Edwards	Edwards	Exact course-record indicator

Public listing sites do not reproduce every historical assessor field and may report slightly different area or lot measurements. Using the exact course-data record as the baseline avoids selectively imputing the unreported variables; only the currently reported living area, bathroom count, and above-grade bedroom count were changed.

²[Redfin property page and sale history for the same parcel.](#)

7.2 Historical holdout check

Table 6 compares the 2008 sale with predictions based on the unmodified 2008 feature row. OLS misses the actual sale by only \$4,195 (3.53%), while Ridge misses it by \$3,874 (3.26%). This close holdout result shows that both models describe this house reasonably well on the training period’s nominal price scale.

Table 6: Predictions for the held-out 2008 transaction.

Value	Amount	Difference from sale	Percent difference
Actual April 2008 sale	\$119,000	\$0	0.00%
OLS prediction	\$123,195	+\$4,195	+3.53%
Ridge, $\alpha = 0.60$	\$122,874	+\$3,874	+3.26%

7.3 Current prediction and comparison

Table 7 compares the mapped current-feature predictions with the 2022 sale, current listing price, and Zestimate. OLS and Ridge remain nearly identical, so the large gap is not caused by the choice between the two linear estimators.

Table 7: Current-feature predictions and public market values.

Value	Amount	Difference from list	Percent difference
April 2022 sale	\$186,820	-\$59,680	-24.21%
Current-feature OLS prediction	\$121,869	-\$124,631	-50.56%
Current-feature Ridge, $\alpha = 0.60$	\$121,451	-\$125,049	-50.73%
October 2026 listing price	\$246,500	\$0	0.00%
Zillow Zestimate	\$242,000	-\$4,500	-1.83%

The historical check and the current comparison tell different but consistent stories. The model is accurate for this held-out 2008 sale, yet its 2026 predictions stay close to the old nominal price level. The specification contains no sale-year variable, inflation adjustment, or housing-price index, so it cannot learn the market appreciation between 2008 and 2026. This same-property example therefore provides direct evidence of temporal distribution shift rather than a simple property mismatch. It also reinforces the report’s OLS–Ridge conclusion: OLS follows the predefined validation rule, while Ridge supplies more stable, uniquely identified coefficients under multicollinearity; neither adjustment solves an omitted time-price effect. These figures are an illustrative classroom application, not a professional appraisal.

8 Limitations and Possible Extensions

- **Single split.** Model ranking is based on one seeded validation split. The 0.08% validation-MSE gap between OLS and Ridge $\alpha = 0.60$ could change under another split. Repeated cross-validation would quantify this uncertainty.
- **Rank deficiency.** Exact feature dependencies make OLS coefficients non-unique. Reference-category encoding or removing a redundant basement variable would produce an identifiable OLS parameterization.

- **Linear functional form.** The model does not include interactions, splines, or nonlinear thresholds. Quality and size effects may vary across neighborhoods and price ranges.
- **Limited features.** The project deliberately uses 47 variables. Other available attributes, including additional exterior, condition, and location variables, may improve prediction.
- **Back-transformation.** Exponentiating a predicted conditional mean in log space does not generally equal the conditional mean price in dollars. A smearing correction could be considered when unbiased mean-dollar prediction is the goal.
- **External validity.** Results describe the supplied Iowa data and may not transfer to a different market or time period without retraining.

9 Conclusion

Using the required 47 features and fixed 1,800/600/508 split, OLS has the smallest validation MSE among the seven required candidates. After refitting on 2,400 development observations, it achieves test $R^2 = 0.9087$, log RMSE 0.1201, and dollar MAE of approximately \$15,000. Ridge with $\alpha = 0.60$ performs nearly identically on validation data and provides substantially more stable coefficients. The results support the usefulness of the log-price linear framework for prediction, while the rank-deficiency and residual diagnostics caution against interpreting the selected OLS coefficient vector as a set of unique economic effects.

A Complete List of 47 Features

Table 8: The 21 numerical features and one ordinal feature.

No.	Feature	Type	No.	Feature	Type
1	LotArea	Numerical	12	FullBath	Numerical
2	OverallQual	Numerical	13	HalfBath	Numerical
3	OverallCond	Numerical	14	BedroomAbvGr	Numerical
4	YearBuilt	Numerical	15	TotRmsAbvGrd	Numerical
5	YearRemodAdd	Numerical	16	Fireplaces	Numerical
6	BsmtFinSF	Numerical	17	GarageCars	Numerical
7	BsmtUnfSF	Numerical	18	GarageArea	Numerical
8	TotalBsmtSF	Numerical	19	WoodDeckSF	Numerical
9	1stFlrSF	Numerical	20	OpenPorchSF	Numerical
10	2ndFlrSF	Numerical	21	EnclosedPorch	Numerical
11	GrLivArea	Numerical	22	BsmtQual_ordinal	Ordinal

Table 9: The 25 neighborhood-indicator features.

No.	Feature	Type	No.	Feature	Type
23	NBHD_Blmngtn	Indicator	36	NBHD_NPkVill	Indicator
24	NBHD_Blueste	Indicator	37	NBHD_NWAmes	Indicator
25	NBHD_BrDale	Indicator	38	NBHD_NoRidge	Indicator
26	NBHD_BrkSide	Indicator	39	NBHD_NridgHt	Indicator
27	NBHD_ClearCr	Indicator	40	NBHD_OldTown	Indicator
28	NBHD_CollgCr	Indicator	41	NBHD_SWISU	Indicator
29	NBHD_Crawfor	Indicator	42	NBHD_Sawyer	Indicator
30	NBHD_Edwards	Indicator	43	NBHD_SawyerW	Indicator
31	NBHD_Gilbert	Indicator	44	NBHD_Somerst	Indicator
32	NBHD_IDOTRR	Indicator	45	NBHD_StoneBr	Indicator
33	NBHD_MeadowV	Indicator	46	NBHD_Timber	Indicator
34	NBHD_Mitchel	Indicator	47	NBHD_Veenker	Indicator
35	NBHD_NAmes	Indicator			

B Selected-Model Parameter Vector

Table 10 reports the solver’s parameter vector after refitting OLS on the 2,400 development observations. Parameters are expressed in original input units and log-price target units. The intercept is separate from the 47 slopes. Because the design is rank deficient, this is one numerical solution, not a unique coefficient vector. Predictions should be generated with the saved pipeline; the very large canceling coefficients must not be interpreted individually.

Table 10: One numerical parameter vector for the final OLS fit.

Parameter	Coefficient
Intercept	12093192747.369524
LotArea	2.3963165902358263e-06
OverallQual	0.070855827469600965
OverallCond	0.055650925487499212
YearBuilt	0.0028592520479720348
YearRemodAdd	0.00086390024081926787
BsmtFinSF	-767725470.55720007
BsmtUnfSF	-767725470.55728829
TotalBsmtSF	767725470.55737162
1stFlrSF	4.3564428622277209e-05
2ndFlrSF	2.3704389393927261e-05
GrLivArea	0.00022881150033979251
FullBath	0.0079451668564325569
HalfBath	0.0074616825071070758
BedroomAbvGr	-0.0077107208645243435
TotRmsAbvGrd	0.0036594772995293822
Fireplaces	0.031145754114527472
GarageCars	0.028188163713551193
GarageArea	0.00010946818198380031
WoodDeckSF	5.7800641971754614e-05
OpenPorchSF	9.7443749138376816e-05
EnclosedPorch	0.00015366988833127759
BsmtQual_ordinal	0.020465101325888978
NBHD_Blmngtn	-12093192744.249897
NBHD_Blueste	-12093192744.351295
NBHD_BrDale	-12093192744.378828
NBHD_BrkSide	-12093192744.199856
NBHD_ClearCr	-12093192744.173182
NBHD_CollgCr	-12093192744.207769
NBHD_Crawfor	-12093192744.087782
NBHD_Edwards	-12093192744.240747
NBHD_Gilbert	-12093192744.205662
NBHD_IDOTRR	-12093192744.301771
NBHD_MeadowV	-12093192744.399384
NBHD_Mitchel	-12093192744.239519
NBHD_NAmes	-12093192744.21439
NBHD_NPkVill	-12093192744.305983
NBHD_NWAmes	-12093192744.265966
NBHD_NoRidge	-12093192744.175354
NBHD_NridgHt	-12093192744.139254
NBHD_OldTown	-12093192744.277855
NBHD_SWISU	-12093192744.225033
NBHD_Sawyer	-12093192744.210047

Continued on next page

Parameter	Coefficient
NBHD_SawyerW	-12093192744.243877
NBHD_Somerst	-12093192744.172373
NBHD_StoneBr	-12093192744.124281
NBHD_Timber	-12093192744.198605
NBHD_Veenker	-12093192744.250326

C Reproducibility Algorithm

```

read and clean the spreadsheet
construct BsmtFinSF, ordinal BsmtQual, and 25 neighborhood indicators
assert 47 predictors and no missing selected values

permutation = random_generator(seed=20260928).permutation(2908)
train = permutation[:1800]
validation = permutation[1800:2400]
test = permutation[2400:]

for candidate in [OLS, Ridge(0.10, 0.30, 0.60), Lasso(0.02, 0.06, 0.10)]:
    fit preprocessing and target transformations on training data only
    fit candidate to standardized log1p(SalePrice)
    calculate training and validation metrics on the log-price scale

best_model = candidate with minimum validation MSE
refit best_model on training + validation data
evaluate once on the held-out test data

```

References

- [1] FIM590-003 course staff. *Project 1: Iowa House Price Prediction*. Course project handout, 2026.
- [2] Dean De Cock. Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. *Journal of Statistics Education*, 19(3), 2011.
- [3] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*, 2nd edition. Springer, 2009.
- [4] Fabian Pedregosa et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Iowa House Price Prediction - EDA

Workflow: construct the 47 specified regressors, audit and clean data, visualize `SalePrice`, and visualize numeric-regressor correlations.

In [1]:

```
from pathlib import Path
print(Path.cwd())
```

/Users/kerolin/Desktop/python_work/FIM590003Project/notebooks

1. Load data and construct the 47 specified regressors

The 47 regressors comprise 21 numeric features, ordinal `BsmtQual`, and 25 fixed `Neighborhood` indicators. `SalePrice` is the target.

In [2]:

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

pd.set_option("display.max_columns", 100)
sns.set_theme(style="whitegrid", context="notebook")

DATA_FILENAME = "IA_House_Price_Original_Data.xlsx"
DATA_CANDIDATES = [Path("../data") / DATA_FILENAME, Path("data") / DATA_FILENAME]
DATA_PATH = next((path for path in DATA_CANDIDATES if path.exists()), None)
if DATA_PATH is None:
    raise FileNotFoundError(f"Could not find {DATA_FILENAME} in the repository data directory.")
TARGET = "SalePrice"
numeric_features = [
    "LotArea", "OverallQual", "OverallCond", "YearBuilt", "YearRemodAdd", "BsmtFinSF",
    "BsmtUnfSF", "TotalBsmtSF", "1stFlrSF", "2ndFlrSF", "GrLivArea", "FullBath",
    "HalfBath", "BedroomAbvGr", "TotRmsAbvGrd", "Fireplaces", "GarageCars", "GarageArea",
    "WoodDeckSF", "OpenPorchSF", "EnclosedPorch",
]
continuous_features = [
    "LotArea", "YearBuilt", "YearRemodAdd", "BsmtFinSF", "BsmtUnfSF", "TotalBsmtSF",
    "1stFlrSF", "2ndFlrSF", "GrLivArea", "GarageArea", "WoodDeckSF", "OpenPorchSF", "EnclosedPorch",
]
neighbourhoods = [
    "Blmngtn", "Blueste", "BrDale", "BrkSide", "ClearCr", "CollgCr", "Crawfor", "Edwards",
    "Gilbert", "IDOTRR", "MeadowV", "Mitchel", "NAMES", "NPkVill", "NWAmes", "NoRidge",
    "NridgHt", "OldTown", "SWISU", "Sawyer", "SawyerW", "Somerst", "StoneBr", "Timber", "Veenker",
]
source_columns = [feature for feature in numeric_features if feature != "BsmtFinSF"] + ["BsmtFinSF1",
"BsmtFinSF2", "BsmtQual", "Neighborhood", TARGET]

raw = pd.read_excel(DATA_PATH, skiprows=3)
raw.columns = raw.columns.astype(str).str.strip()
blank_layout_columns = raw.columns[raw.columns.str.startswith("Unnamed")].tolist()
raw = raw.drop(columns=blank_layout_columns)
missing_source_columns = sorted(set(source_columns) - set(raw.columns))
if missing_source_columns:
    raise KeyError(f"Required source columns are missing: {missing_source_columns}")
df = raw.loc[:, source_columns].copy()
print(f"Observations: {len(df):,}")
print(f"Blank layout columns removed: {blank_layout_columns}")
print(f"Selected source columns, including target: {df.shape[1]}")
```

```
/Users/kerolin/anaconda3/lib/python3.11/site-packages/pandas/core/arrays/masked.py:61: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
```

```
from pandas.core import (
Observations: 2,908
Blank layout columns removed: ['Unnamed: 81', 'Unnamed: 82']
Selected source columns, including target: 25
```

2. Check data status and clean data

This step checks column availability, duplicates and missing values, then constructs `BsmtFinSF`, maps `BsmtQual` (`Ex/Gd/TA/Fa/Po/NA` to `5/4/3/2/1/0`), and creates the 25 fixed neighborhood indicators.

In [3]:

```
data_status = pd.Series({
    "observations": len(df), "duplicate_rows": int(df.duplicated().sum()),
    "missing_target": int(df[TARGET].isna().sum()), "source_columns": df.shape[1],
})
display(data_status.to_frame("value"))

missingness = (df.isna().sum().to_frame("missing_count")
               .assign(missing_pct=lambda t: 100 * t["missing_count"] / len(df))
               .query("missing_count > 0").sort_values("missing_count", ascending=False))
display(missingness)

clean = df.copy()
clean["BsmtFinSF"] = clean["BsmtFinSF1"] + clean["BsmtFinSF2"]
bsmtqual_map = {"Ex": 5, "Gd": 4, "TA": 3, "Fa": 2, "Po": 1, "NA": 0}
clean["BsmtQual_ordinal"] = clean["BsmtQual"].fillna("NA").map(bsmtqual_map)

observed_neighbourhoods = set(clean["Neighborhood"].dropna())
if observed_neighbourhoods != set(neighbourhoods):
    raise ValueError("Neighborhood categories do not match the 25-feature specification.")
dummies = pd.get_dummies(clean["Neighborhood"], prefix="NBHD", dtype=int).reindex(
    columns=[f"NBHD_{name}" for name in neighbourhoods], fill_value=0
)
X_47 = pd.concat([clean[numeric_features], clean[["BsmtQual_ordinal"]], dummies], axis=1)
y = clean[TARGET].copy()
if X_47.isna().any().any() or y.isna().any():
    raise ValueError("Missing values remain; define a train-only imputation or exclusion rule before modelling.")
feature_status = pd.Series({
    "numeric_regressors": len(numeric_features), "ordinal_regressors": 1,
    "neighbourhood_indicators": dummies.shape[1], "total_regressors": X_47.shape[1],
    "missing_values_in_47_regressors": int(X_47.isna().sum().sum()),
})
display(feature_status.to_frame("value"))
assert X_47.shape[1] == 47
```

	value
observations	2908
duplicate_rows	4
missing_target	0
source_columns	25

	missing_count	missing_pct
BsmtQual	79	2.716644

	value
numeric_regressors	21

	value
ordinal_regressors	1
neighbourhood_indicators	25
total_regressors	47
missing_values_in_47_regressors	0

3. Univariate analysis

This section examines each regressor's relationship with `SalePrice`. It is descriptive: it does not delete observations, transform the target, or select a final model.

In [4]:

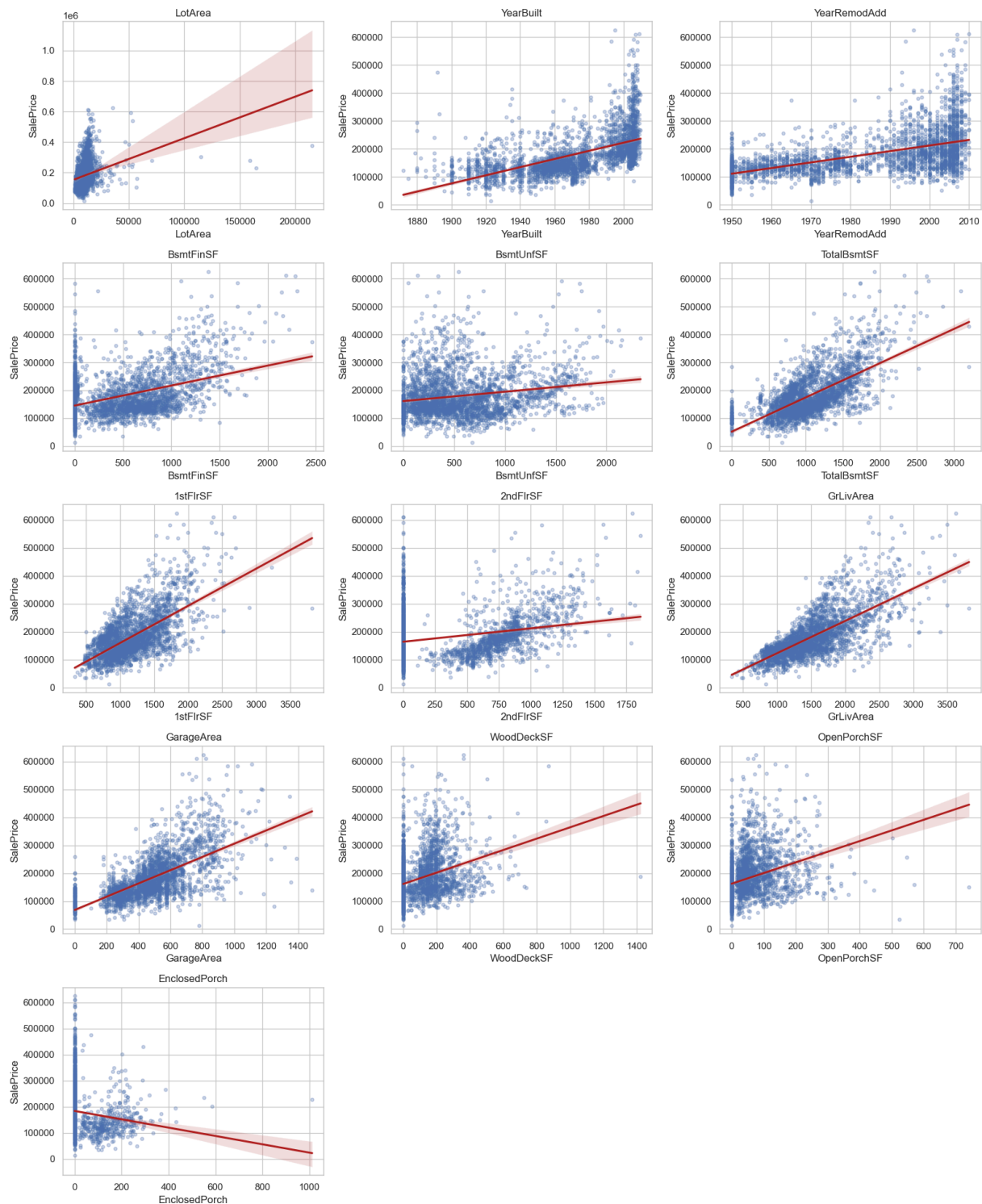
```
discrete_features = [feature for feature in numeric_features if feature not in continuous_features]
continuous_y_summary = pd.DataFrame({
    "pearson_correlation_with_saleprice": X_47[continuous_features].corrwith(y, method="pearson"),
    "spearman_correlation_with_saleprice": X_47[continuous_features].corrwith(y, method="spearman"),
}).assign(abs_pearson=lambda table: table["pearson_correlation_with_saleprice"].abs())
continuous_y_summary = continuous_y_summary.sort_values("abs_pearson", ascending=False)
display(continuous_y_summary.round(3))
```

	pearson_correlation_with_saleprice	spearman_correlation_with_saleprice	abs_pearson
GrLivArea	0.721	0.725	0.721
TotalBsmfSF	0.659	0.607	0.659
GarageArea	0.651	0.662	0.651
1stFlrSF	0.640	0.579	0.640
YearBuilt	0.566	0.681	0.566
YearRemodAdd	0.541	0.602	0.541
BsmfFinSF	0.421	0.328	0.421
WoodDeckSF	0.328	0.363	0.328
OpenPorchSF	0.325	0.481	0.325
LotArea	0.272	0.433	0.272
2ndFlrSF	0.262	0.249	0.262
BsmfUnfSF	0.189	0.166	0.189
EnclosedPorch	-0.131	-0.235	0.131

In [5]:

```
fig, axes = plt.subplots(5, 3, figsize=(16, 20))
for feature, axis in zip(continuous_features, axes.flat):
    sns.regplot(data=pd.DataFrame({feature: X_47[feature], TARGET: y}), x=feature, y=TARGET,
                scatter_kws={"alpha": 0.30, "s": 14}, line_kws={"color": "firebrick"}, ax=axis)
    axis.set_title(feature)
for axis in axes.flat[len(continuous_features):]:
    axis.set_visible(False)
fig.suptitle("Continuous regressors versus SalePrice", y=1.01, fontsize=16)
plt.tight_layout()
plt.show()
```

Continuous regressors versus SalePrice



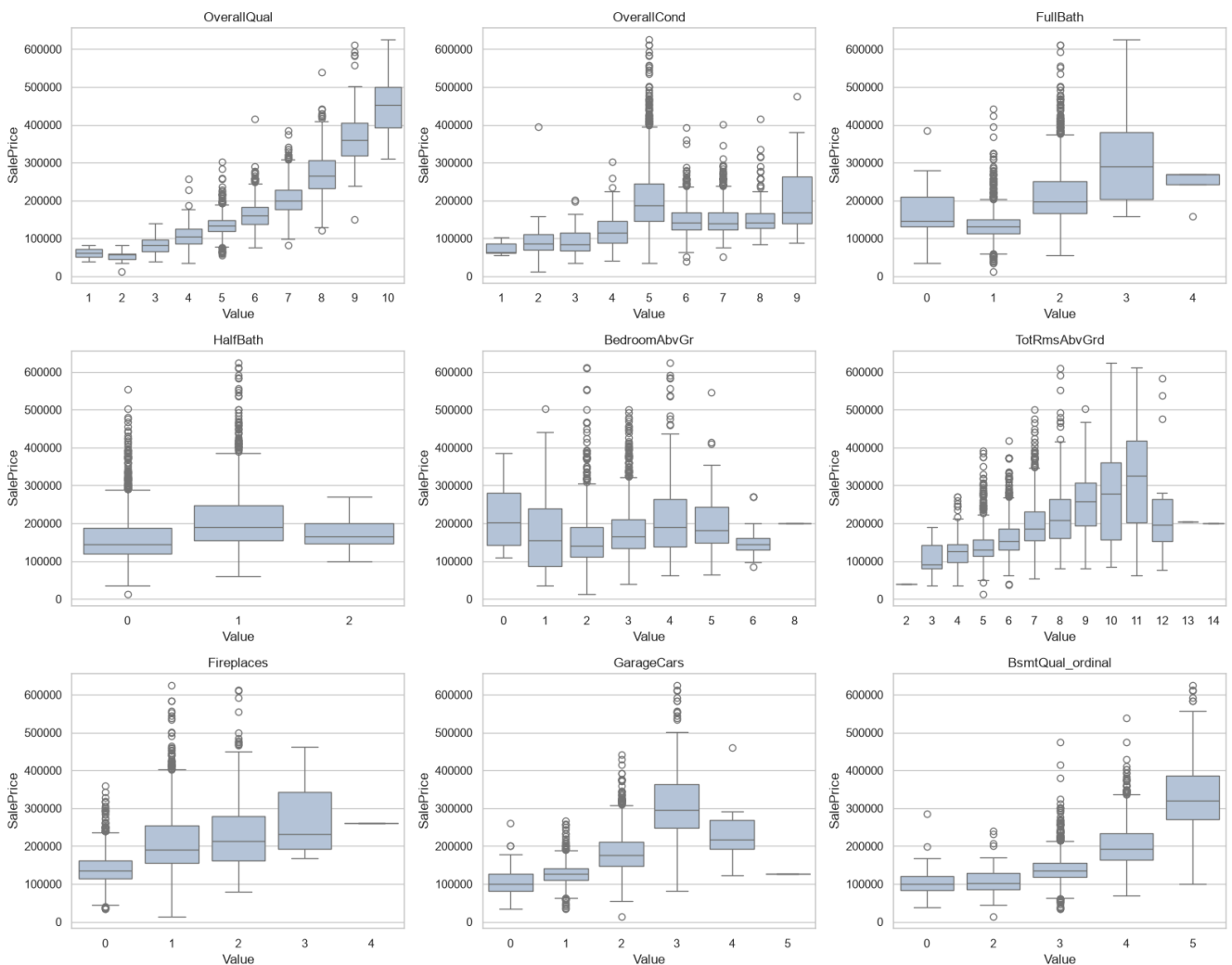
In [25]:

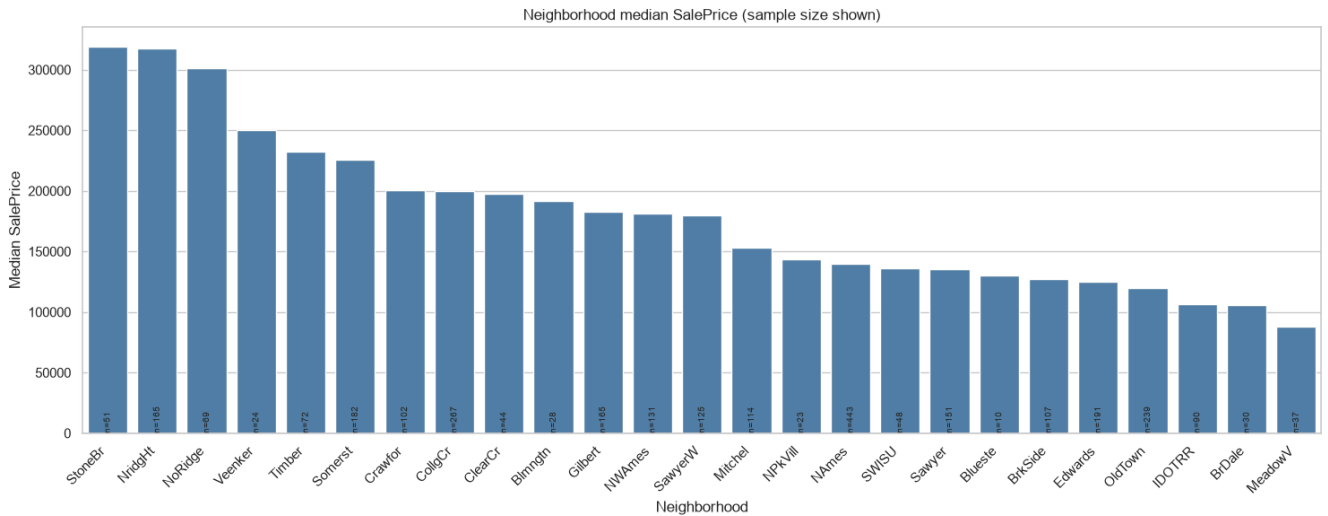
```
fig, axes = plt.subplots(3, 3, figsize=(16, 13))
for feature, axis in zip(discrete_features + ["BsmtQual_ordinal"], axes.flat):
    plot_data = pd.DataFrame({feature: X_47[feature], TARGET: y})
    sns.boxplot(data=plot_data, x=feature, y=TARGET, color="lightsteelblue", ax=axis)
    axis.set_title(feature)
    axis.set_xlabel("Value")
    axis.set_ylabel("SalePrice")
for axis in axes.flat[len(discrete_features) + 1:]:
    axis.set_visible(False)
fig.suptitle("Count and ordinal regressors versus SalePrice", y=1.01, fontsize=16)
```

```
plt.tight_layout()
plt.show()

neighbourhood_y = clean.groupby("Neighborhood")[TARGET].agg(observations="size", median_saleprice="median",
mean_saleprice="mean").sort_values("median_saleprice", ascending=False)
plt.figure(figsize=(15, 6))
axis = sns.barplot(x=neighbourhood_y.index, y=neighbourhood_y["median_saleprice"], color="steelblue")
for position, observations in enumerate(neighbourhood_y["observations"]):
    axis.text(position, 0, f"n={observations}", ha="center", va="bottom", rotation=90, fontsize=7)
plt.title("Neighborhood median SalePrice (sample size shown)")
plt.xlabel("Neighborhood")
plt.ylabel("Median SalePrice")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

Count and ordinal regressors versus SalePrice





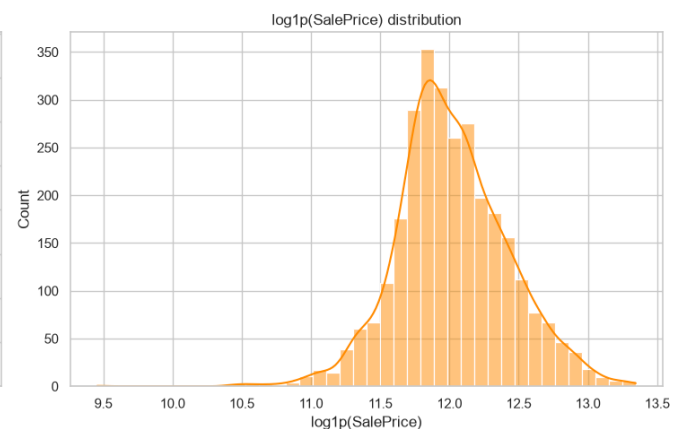
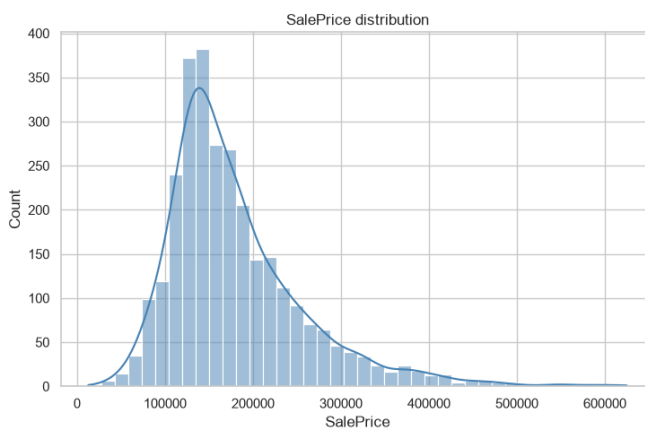
4. Visualize the target distribution

`SalePrice` is shown in levels and log scale. This visualization does not choose the final target transformation.

In [26]:

```
display(y.describe().to_frame(TARGET))
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
sns.histplot(y, bins=40, kde=True, color="steelblue", ax=axes[0])
axes[0].set(title="SalePrice distribution", xlabel="SalePrice")
sns.histplot(np.log1p(y), bins=40, kde=True, color="darkorange", ax=axes[1])
axes[1].set(title="log1p(SalePrice) distribution", xlabel="log1p(SalePrice)")
plt.tight_layout()
plt.show()
```

SalePrice	
count	2908.000000
mean	180272.213549
std	78139.090808
min	12789.000000
25%	129362.500000
50%	160000.000000
75%	213310.000000
max	625000.000000



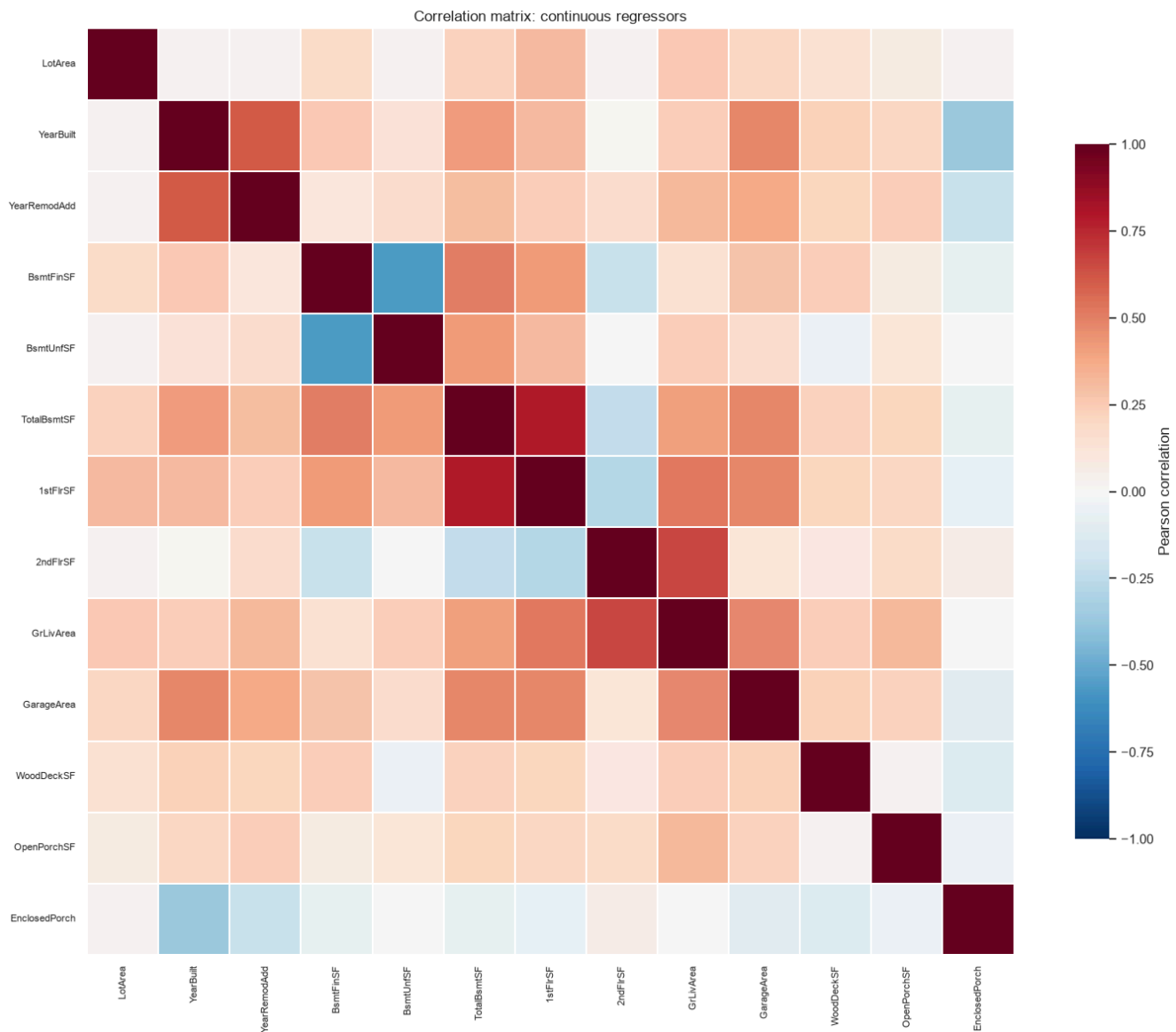
5. Visualize correlations among continuous regressors

The matrix uses only continuous regressors and excludes `SalePrice`, ordinal quality variables, count variables, and neighborhood dummies. Positive correlations are red, negative correlations are blue, and values close to zero are white.

In [27]:

```
continuous_correlation = X_47[continuous_features].corr(method="pearson")
plt.figure(figsize=(15, 12))
sns.heatmap(continuous_correlation, cmap="RdBu_r", center=0, vmin=-1, vmax=1,
            square=True, linewidths=0.35, linecolor="white",
            cbar_kws={"shrink": 0.75, "label": "Pearson correlation"},
            xticklabels=True, yticklabels=True)
plt.title("Correlation matrix: continuous regressors")
plt.xticks(rotation=90, fontsize=8)
plt.yticks(rotation=0, fontsize=8)
plt.tight_layout()
plt.show()

high_correlation_pairs = (continuous_correlation.where(np.triu(np.ones(continuous_correlation.shape),
k=1).astype(bool))
                        .stack().rename("correlation").to_frame()
                        .assign(abs_correlation=lambda t: t["correlation"].abs())
                        .sort_values("abs_correlation", ascending=False))
display(high_correlation_pairs.head(15))
```



		correlation	abs_correlation
TotalBsmtSF	1stFlrSF	0.789484	0.789484
2ndFlrSF	GrLivArea	0.664412	0.664412
YearBuilt	YearRemodAdd	0.611591	0.611591
BsmtFinSF	BsmtUnfSF	-0.565107	0.565107
1stFlrSF	GrLivArea	0.527698	0.527698
BsmtFinSF	TotalBsmtSF	0.503269	0.503269
1stFlrSF	GarageArea	0.482362	0.482362
YearBuilt	GarageArea	0.478887	0.478887
TotalBsmtSF	GarageArea	0.477684	0.477684
GrLivArea	GarageArea	0.475827	0.475827
BsmtUnfSF	TotalBsmtSF	0.428522	0.428522
BsmtFinSF	1stFlrSF	0.427463	0.427463
YearBuilt	TotalBsmtSF	0.415522	0.415522
TotalBsmtSF	GrLivArea	0.401733	0.401733
YearRemodAdd	GarageArea	0.374556	0.374556

02 — House-price modeling with a log-price target

This is the project's single modeling notebook. All candidate models are fitted to `log1p(SalePrice)`, selected using validation metrics on the log-price scale, and evaluated once on the held-out test set after selection.

Modeling protocol

- Fixed shuffled split: 1,800 training, 600 validation, and 508 test observations.
- The 47-feature contract is identical to the EDA notebook.
- Numeric predictors and the ordinal basement-quality variable are standardized within each fit. Neighborhood indicators remain 0/1.
- `log1p(SalePrice)` is standardized internally during fitting, then returned to the log-price scale before all reported metrics are calculated. Thus MSE, RMSE, and MAE are not arbitrary standardized-unit scores.
- Candidate models are selected exclusively by validation MSE. Test metrics are calculated only for the selected model, never used for selection.

In [1]:

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.linear_model import Lasso, LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, StandardScaler

pd.set_option('display.max_columns', 100)
sns.set_theme(style='whitegrid', context='notebook')

DATA_FILENAME = 'IA_House_Price_Original_Data.xlsx'
DATA_CANDIDATES = [Path('../data') / DATA_FILENAME, Path('data') / DATA_FILENAME]
DATA_PATH = next((path for path in DATA_CANDIDATES if path.exists()), None)
if DATA_PATH is None:
    raise FileNotFoundError(f'Could not find {DATA_FILENAME} in the repository data directory.')
TARGET = 'SalePrice'
RANDOM_SEED = 20260928

numeric_features = [
    'LotArea', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd', 'BsmtFinSF',
    'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'GrLivArea', 'FullBath',
    'HalfBath', 'BedroomAbvGr', 'TotRmsAbvGrd', 'Fireplaces', 'GarageCars', 'GarageArea',
    'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch',
]

neighbourhoods = [
    'Blmngtn', 'Blueste', 'BrDale', 'BrkSide', 'ClearCr', 'CollgCr', 'Crawfor', 'Edwards',
    'Gilbert', 'IDOTRR', 'MeadowV', 'Mitchel', 'Names', 'NPkVill', 'NWAmes', 'NoRidge',
    'NridgHt', 'OldTown', 'SWISU', 'Sawyer', 'SawyerW', 'Somerst', 'StoneBr', 'Timber', 'Veenker',
]

bsmtqual_map = {'Ex': 5, 'Gd': 4, 'TA': 3, 'Fa': 2, 'Po': 1, 'NA': 0}
```

```
/Users/kerolin/anaconda3/lib/python3.11/site-packages/pandas/core/arrays/masked.py:61: UserWarning: Pandas
requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
from pandas.core import (
```

In [2]:

```
raw = pd.read_excel(DATA_PATH, skiprows=3)
raw.columns = raw.columns.astype(str).str.strip()
raw = raw.loc[:, ~raw.columns.str.startswith('Unnamed')].copy()
```

```

clean = raw.copy()
clean['BsmtFinSF'] = clean['BsmtFinSF1'] + clean['BsmtFinSF2']
clean['BsmtQual_ordinal'] = clean['BsmtQual'].fillna('NA').map(bsmtqual_map)

if set(clean['Neighborhood'].dropna()) != set(neighbourhoods):
    raise ValueError('Neighborhood categories do not match the 25-feature specification.')
neighbourhood_columns = [f'NBHD_{name}' for name in neighbourhoods]
dummies = pd.get_dummies(clean['Neighborhood'], prefix='NBHD', dtype=int).reindex(
    columns=neighbourhood_columns, fill_value=0
)
X_47 = pd.concat([clean[numeric_features], clean[['BsmtQual_ordinal']], dummies], axis=1)
y_price = clean[TARGET].copy()

assert X_47.shape[1] == 47, f'Expected 47 features, found {X_47.shape[1]}.'
assert not X_47.isna().any().any(), 'Selected regressors contain missing values.'
assert not y_price.isna().any(), 'SalePrice contains missing values.'

display(pd.Series({
    'observations': len(X_47), 'features': X_47.shape[1],
    'missing_regressor_values': int(X_47.isna().sum().sum()),
    'missing_target_values': int(y_price.isna().sum()),
}).to_frame('value'))

```

	value
observations	2908
features	47
missing_regressor_values	0
missing_target_values	0

Complete list of 47 features

The intercept is separate and is not counted as a feature.

In [3]:

```

feature_list = pd.DataFrame({
    'No.': np.arange(1, 48), 'Feature': X_47.columns,
    'Type': ['Numerical'] * 21 + ['Ordinal'] + ['Neighborhood indicator'] * 25,
})
with pd.option_context('display.max_rows', None):
    display(feature_list.set_index('No.'))

```

No.	Feature	Type
1	LotArea	Numerical
2	OverallQual	Numerical
3	OverallCond	Numerical
4	YearBuilt	Numerical
5	YearRemodAdd	Numerical
6	BsmtFinSF	Numerical
7	BsmtUnfSF	Numerical
8	TotalBsmtSF	Numerical
9	1stFlrSF	Numerical
10	2ndFlrSF	Numerical
11	GrLivArea	Numerical
12	FullBath	Numerical
13	HalfBath	Numerical

No.	Feature	Type
14	BedroomAbvGr	Numerical
15	TotRmsAbvGrd	Numerical
16	Fireplaces	Numerical
17	GarageCars	Numerical
18	GarageArea	Numerical
19	WoodDeckSF	Numerical
20	OpenPorchSF	Numerical
21	EnclosedPorch	Numerical
22	BsmtQual_ordinal	Ordinal
23	NBHD_Blmngtn	Neighborhood indicator
24	NBHD_Blueste	Neighborhood indicator
25	NBHD_BrDale	Neighborhood indicator
26	NBHD_BrkSide	Neighborhood indicator
27	NBHD_ClearCr	Neighborhood indicator
28	NBHD_CollgCr	Neighborhood indicator
29	NBHD_Crawfor	Neighborhood indicator
30	NBHD_Edwards	Neighborhood indicator
31	NBHD_Gilbert	Neighborhood indicator
32	NBHD_IDOTRR	Neighborhood indicator
33	NBHD_MeadowV	Neighborhood indicator
34	NBHD_Mitchel	Neighborhood indicator
35	NBHD_NAmes	Neighborhood indicator
36	NBHD_NPkVill	Neighborhood indicator
37	NBHD_NWAmes	Neighborhood indicator
38	NBHD_NoRidge	Neighborhood indicator
39	NBHD_NridgHt	Neighborhood indicator
40	NBHD_OldTown	Neighborhood indicator
41	NBHD_SWISU	Neighborhood indicator
42	NBHD_Sawyer	Neighborhood indicator
43	NBHD_SawyerW	Neighborhood indicator
44	NBHD_Somerst	Neighborhood indicator
45	NBHD_StoneBr	Neighborhood indicator
46	NBHD_Timber	Neighborhood indicator
47	NBHD_Veenker	Neighborhood indicator

In [4]:

```

rng = np.random.default_rng(RANDOM_SEED)
permutation = rng.permutation(len(X_47))
train_idx, validation_idx, test_idx = permutation[:1800], permutation[1800:2400], permutation[2400:]

X_train, y_train = X_47.iloc[train_idx], y_price.iloc[train_idx]
X_validation, y_validation = X_47.iloc[validation_idx], y_price.iloc[validation_idx]
X_test, y_test = X_47.iloc[test_idx], y_price.iloc[test_idx]
assert (len(X_train), len(X_validation), len(X_test)) == (1800, 600, 508)

display(pd.DataFrame({
    'observations': [len(X_train), len(X_validation), len(X_test)],
    'mean_saleprice': [y_train.mean(), y_validation.mean(), y_test.mean()],
}, index=['train', 'validation', 'test']).round(2))

```

	observations	mean_saleprice
train	1800	180018.88
validation	600	178133.52
test	508	183695.88

Models and metrics

OLS, Ridge, and Lasso are fitted with the assignment's specified penalty strengths. The primary metrics below evaluate `log1p(SalePrice)` : MSE, RMSE, MAE, and R^2 . The raw-dollar target is never used as the fitting baseline.

In [5]:

```
scaled_features = numeric_features + ['BsmQual_ordinal']

def make_regressor_pipeline(estimator):
    preprocessing = ColumnTransformer(
        [('scaled', StandardScaler(), scaled_features),
         ('neighborhood', 'passthrough', neighbourhood_columns)],
        verbose_feature_names_out=False,
    )
    return Pipeline([('preprocess', preprocessing), ('model', estimator)])

def make_log_target_model(estimator):
    # Pipeline inversion is StandardScaler inverse followed by expm1.
    target_transformer = Pipeline([
        ('log', FunctionTransformer(np.log1p, inverse_func=np.expm1, validate=False)),
        ('scale', StandardScaler()),
    ])
    return TransformedTargetRegressor(
        regressor=make_regressor_pipeline(estimator),
        transformer=target_transformer,
    )

def log_scale_metrics(actual_price, predicted_price):
    actual_log = np.log1p(actual_price)
    predicted_log = np.log1p(np.maximum(predicted_price, 0))
    return {
        'mse': mean_squared_error(actual_log, predicted_log),
        'rmse': np.sqrt(mean_squared_error(actual_log, predicted_log)),
        'mae': mean_absolute_error(actual_log, predicted_log),
        'r2': r2_score(actual_log, predicted_log),
    }

candidates = {
    'OLS': LinearRegression(),
    **{f'Ridge (alpha={alpha:.2f})': Ridge(alpha=alpha) for alpha in [0.10, 0.30, 0.60]},
    **{f'Lasso (alpha={alpha:.2f})': Lasso(alpha=alpha, max_iter=50_000) for alpha in [0.02, 0.06, 0.10]},
}

fitted_candidates, records = {}, []
for name, estimator in candidates.items():
    model = make_log_target_model(estimator)
    model.fit(X_train, y_train)
    fitted_candidates[name] = model
    train_metrics = log_scale_metrics(y_train, model.predict(X_train))
    validation_metrics = log_scale_metrics(y_validation, model.predict(X_validation))
    records.append({
        'model': name,
        **{f'train_{metric}': value for metric, value in train_metrics.items()},
        **{f'validation_{metric}': value for metric, value in validation_metrics.items()},
    })

model_metrics = pd.DataFrame(records).sort_values('validation_mse').reset_index(drop=True)
best_validation_mse = model_metrics.loc[0, 'validation_mse']
model_metrics['validation_mse_delta_to_best'] = model_metrics['validation_mse'] - best_validation_mse
model_metrics['validation_mse_pct_above_best'] = 100 * model_metrics['validation_mse_delta_to_best'] /
best_validation_mse
display(model_metrics.round(8))
```

	model	train_mse	train_rmse	train_mae	train_r2	validation_mse	validation_rmse	validation_mae	validation_r2	validation_mse_delta_to_best	validation_mse_pct_above_best
0	OLS	0.013908	0.117930	0.080995	0.912059	0.014917	0.122137	0.084055	0.914905	0.000000	0.000000
1	Ridge (alpha=0.60)	0.013904	0.117914	0.081020	0.912083	0.014929	0.122186	0.083999	0.914837	0.000012	0.080107
2	Ridge (alpha=0.30)	0.013902	0.117908	0.081008	0.912092	0.014933	0.122201	0.084024	0.914816	0.000016	0.104815
3	Ridge (alpha=0.10)	0.013902	0.117906	0.081004	0.912094	0.014936	0.122213	0.084046	0.914798	0.000019	0.125443
4	Lasso (alpha=0.02)	0.016669	0.129110	0.091020	0.894595	0.017505	0.132305	0.092142	0.900147	0.002587	17.343160
5	Lasso (alpha=0.06)	0.019338	0.139063	0.097379	0.877718	0.021760	0.147513	0.103026	0.875871	0.006843	45.870798
6	Lasso (alpha=0.10)	0.022969	0.151554	0.107108	0.854762	0.026563	0.162982	0.114689	0.848473	0.011646	78.068131

Complete parameters of all seven candidates

The table contains the intercept and all 47 slopes for each 1,800-observation training fit. Predictor and target standardization are reversed: coefficients use **original input units** and the **log1p(SalePrice) target scale**. These units differ from the standardized-predictor coefficient plots below.

OLS coefficients represent one numerical solution, not uniquely identified: neighborhood indicators sum to one, and finished plus unfinished basement area equals total basement area.

In [6]:

```
def original_input_log_parameters(fitted_model, check_X):
    """Undo predictor and target scaling; retain the log1p price target."""
    reg = fitted_model.regressor_
    pre = reg.named_steps['preprocess']
    est = reg.named_steps['model']
    names = list(pre.get_feature_names_out())
    sy = fitted_model.transformer_.named_steps['scale']
    sx = pre.named_transformers_['scaled']
    mean_x = pd.Series(0.0, index=names)
    scale_x = pd.Series(1.0, index=names)
    mean_x.loc[scaled_features] = sx.mean_
    scale_x.loc[scaled_features] = sx.scale_
    slopes = pd.Series(est.coef_ * sy.scale_[0], index=names) / scale_x
    intercept = float(sy.mean_[0] + sy.scale_[0] * est.intercept_ - slopes.dot(mean_x))
    reconstructed = intercept + check_X[names].to_numpy(dtype=float) @ slopes.to_numpy()
    expected = sy.mean_[0] + sy.scale_[0] * reg.predict(check_X)
    # Collinear OLS fits can contain large cancelling coefficients. Account
    # for the magnitude of intermediate sums, not just the final prediction.
    raw_X = check_X[names].to_numpy(dtype=float)
    transformed_X = pre.transform(check_X)
    raw_magnitude = abs(intercept) + np.abs(raw_X) @ np.abs(slopes.to_numpy())
    fitted_magnitude = abs(sy.mean_[0]) + abs(sy.scale_[0]) * (
        abs(est.intercept_) + np.abs(transformed_X) @ np.abs(est.coef_)
    )
    conversion_magnitude = abs(sy.scale_[0] * est.intercept_) + float(
        np.abs(slopes.to_numpy()) @ np.abs(mean_x.to_numpy())
    )
    roundoff_bound = 64 * np.finfo(float).eps * (
        raw_magnitude + fitted_magnitude + conversion_magnitude
    )
    allowed_error = 1e-8 + 1e-8 * np.abs(expected) + roundoff_bound
    error = np.abs(reconstructed - expected)
    if not (np.isfinite(reconstructed).all() and np.isfinite(expected).all()):
        raise ValueError('Non-finite values found during parameter reconstruction.')
    if np.any(error > allowed_error):
        raise AssertionError()
```

```

        f'{type(est).__name__}: parameter reconstruction exceeds the numerical '
        f'error allowance (max error={error.max():.3g}).'
    )
    if error.max() > 1e-6:
        print(f'Note: {type(est).__name__} has cancelling coefficients; '
              f'parameter reconstruction differs by up to {error.max():.3g} '
              f'log-price units. Use the fitted pipeline for predictions and '
              f'do not interpret individual OLS coefficients as stable effects.')
    return pd.concat([pd.Series({'Intercept': intercept}), slopes])

candidate_parameter_table = pd.DataFrame({
    name: original_input_log_parameters(fitted_candidates[name], X_train)
    for name in candidates
})
candidate_parameter_table.index.name = 'Parameter'
assert candidate_parameter_table.shape == (48, 7)
print('Candidate models: 1,800 training observations.')
print('predicted log1p(SalePrice) = Intercept + sum(coefficient * original feature)')
print('Numerical inputs use original units; BsmtQual uses 0-5; neighborhood indicators use 0/1.')
with pd.option_context('display.max_rows', None, 'display.max_columns', None, 'display.float_format', lambda
v: f'{v:.10g}'):
    display(candidate_parameter_table)

```

Note: LinearRegression has cancelling coefficients; parameter reconstruction differs by up to 0.000114 log-price units. Use the fitted pipeline for predictions and do not interpret individual OLS coefficients as stable effects. Candidate models: 1,800 training observations.
predicted log1p(SalePrice) = Intercept + sum(coefficient * original feature)
Numerical inputs use original units; BsmtQual uses 0-5; neighborhood indicators use 0/1.

Parameter	OLS	Ridge (alpha=0.10)	Ridge (alpha=0.30)	Ridge (alpha=0.60)	Lasso (alpha=0.02)	Lasso (alpha=0.06)	Lasso (alpha=0.10)
Intercept	1187908551	3.413369764	3.402849611	3.387791696	3.425376715	5.010756053	6.380184107
LotArea	2.499664799 e-06	2.504491008e -06	2.513885041e -06	2.527367052e -06	2.429910922e -06	1.083995779e -06	0
OverallQual	0.072517299 37	0.0719481961	0.0720068499	0.07209240511	0.0868734937 2	0.0943426931 3	0.09864124711
OverallCond	0.054440957 87	0.0548526519 7	0.0547947748 8	0.0547106746 5	0.0376750270 5	0.0105793788 8	0
YearBuilt	0.002798266 905	0.0028207885 7	0.0028214603	0.0028223217 75	0.00241521255 6	0.00154219347 9	0.0010982265 4
YearRemodA dd	0.000795022 9212	0.0007713852 241	0.0007757630 463	0.00078212151 41	0.0011825939 28	0.0013466227 44	0.0011484650 73
BsmtFinSF	-214459505. 8	8.163171179e 05	8.161915808e 05	8.160088933e -05	8.54963232e -05	7.558002712e -05	6.024608072e -05
BsmtUnfSF	-214459505. 8	-6.847938812 e-06	-6.83044782e -06	-6.805328154 e-06	-0	-0	-0
TotalBsmtSF	214459505.8	8.895395603e -05	8.895821076e -05	8.896403798e -05	8.288758061e -05	8.010954019e -05	8.042773257e -05
1stFlrSF	6.02427597e -06	5.609285753e -06	8.713607027e -06	1.308095086e -05	2.133760904e -05	1.593536002e -05	1.148059237e -05
2ndFlrSF	5.730755178e -06	7.665520274e -06	1.047948581e -05	1.443269226e -05	0	0	0
GrLivArea	0.000254171 6176	0.0002562355 955	0.0002533875 669	0.0002493846 789	0.0002610779 5	0.0002420406 47	0.0002282164 818
FullBath	0.010261703 46	0.0096680899 88	0.0096929186 61	0.0097274509 68	0	0	0
HalfBath	0.001537344 001	0.0002060399 353	0.0002093490 089	0.00021175196 46	0	0	0
BedroomAbv Gr	-0.011253415 19	-0.0121162308 8	-0.0121274720 3	-0.0121432909 4	-0	0	0
TotRmsAbvG rd	0.005988984 868	0.0059979450 71	0.0060212495 63	0.0060544598 35	0	0	0
Fireplaces	0.029791526 78	0.0301730873 4	0.03017197892	0.0301737453 4	0.0352651987 7	0.02676957771	0.0162406255 1
GarageCars	0.029140809 33	0.029355013	0.0294194509 2	0.0295147379 7	0.0339708590 7	0.0346738546 7	0.0369813403 1

Parameter	OLS	Ridge (alpha=0.10)	Ridge (alpha=0.30)	Ridge (alpha=0.60)	Lasso (alpha=0.02)	Lasso (alpha=0.06)	Lasso (alpha=0.10)
GarageArea	9.505663708e-05	9.30456936e-05	9.301007515e-05	9.295898891e-05	8.110469606e-05	6.60988196e-05	4.75677203e-05
WoodDeckSF	6.715816619e-05	6.678936574e-05	6.671566983e-05	6.660977179e-05	8.134712872e-06	0	0
OpenPorchSF	0.0001301246952	0.0001302668314	0.000130310127	0.0001303692116	2.020263437e-05	0	0
EnclosedPorch	0.0001737713602	0.0001774553429	0.0001777856047	0.0001782527052	0	-0	-0
BsmtQual_dinal	0.01729223689	0.01762166712	0.01764785822	0.01768710583	0.01737644282	0.01284040109	0.008896869931
NBHD_Blmngtn	-1187908547	-0.0296819531	-0.02985365071	-0.03006754446	-0	-0	-0
NBHD_Blueste	-1187908547	-0.1373937577	-0.1330344964	-0.1269900042	-0	-0	-0
NBHD_BrDale	-1187908547	-0.1509761724	-0.1494326594	-0.1471701666	-0	-0	-0
NBHD_BrkSide	-1187908547	0.01960028661	0.01959385394	0.01958531212	0	-0	-0
NBHD_ClearCr	-1187908547	0.04306253344	0.04221636731	0.04100773497	0	0	0
NBHD_CollGr	-1187908547	0.0372510719	0.03677684398	0.03609967537	0	0	0
NBHD_Crawfor	-1187908547	0.1524468687	0.1517823356	0.1508088571	0	0	0
NBHD_Edwards	-1187908547	-0.01075426839	-0.01077384809	-0.01079408675	-0	-0	-0
NBHD_Gilbert	-1187908547	0.01975452027	0.01931323204	0.01868431611	-0	0	0
NBHD_IDOTRR	-1187908547	-0.06420981199	-0.06380343343	-0.06320689423	-0	-0	-0
NBHD_MeadowV	-1187908547	-0.1604651574	-0.158607967	-0.1558964878	-0	-0	-0
NBHD_Mitchel	-1187908547	-0.005767893241	-0.00602166411	-0.006377058615	-0	-0	-0
NBHD_NAMES	-1187908547	0.02281039863	0.02270959924	0.02257067463	0	0	0
NBHD_NPKVil	-1187908547	-0.06731860636	-0.06647565846	-0.06524373113	-0	-0	-0
NBHD_NWAmes	-1187908547	-0.0290721648	-0.02933234028	-0.02969386938	-0	-0	-0
NBHD_NoRidge	-1187908547	0.06061189456	0.05970824493	0.0584107238	0	0	0
NBHD_NridgeHt	-1187908547	0.0894697043	0.08851656665	0.08714371755	0	0	0
NBHD_OldTown	-1187908547	-0.05622126538	-0.05608250567	-0.05587664265	-0	-0	-0
NBHD_SWISU	-1187908547	-0.01031399809	-0.01002453278	-0.009611095459	-0	-0	-0
NBHD_Sawyer	-1187908547	0.02292908416	0.0227987695	0.02261610481	0	0	0
NBHD_SawyerW	-1187908547	-0.0063224119	-0.006675165304	-0.0071709536	-0	-0	-0
NBHD_Somerst	-1187908547	0.05782825293	0.05722222117	0.05635293632	0	0	0
NBHD_StoneBr	-1187908547	0.1277000501	0.1261063415	0.1238040595	0	0	0
NBHD_Timber	-1187908547	0.04709146517	0.04627263867	0.04509979403	0	0	0
NBHD_Veenker	-1187908547	0.02794132928	0.02710090711	0.02591462862	0	0	0

Final test evaluation

Only the candidate with the lowest validation MSE is refitted on training plus validation data. This prevents test-set metrics from influencing model selection.

In [7]:

```
best_model_name = model_metrics.loc[0, 'model']
best_estimator = candidates[best_model_name]
X_development = pd.concat([X_train, X_validation])
y_development = pd.concat([y_train, y_validation])

final_model = make_log_target_model(best_estimator)
final_model.fit(X_development, y_development)
test_prediction = final_model.predict(X_test)
test_metrics = log_scale_metrics(y_test, test_prediction)

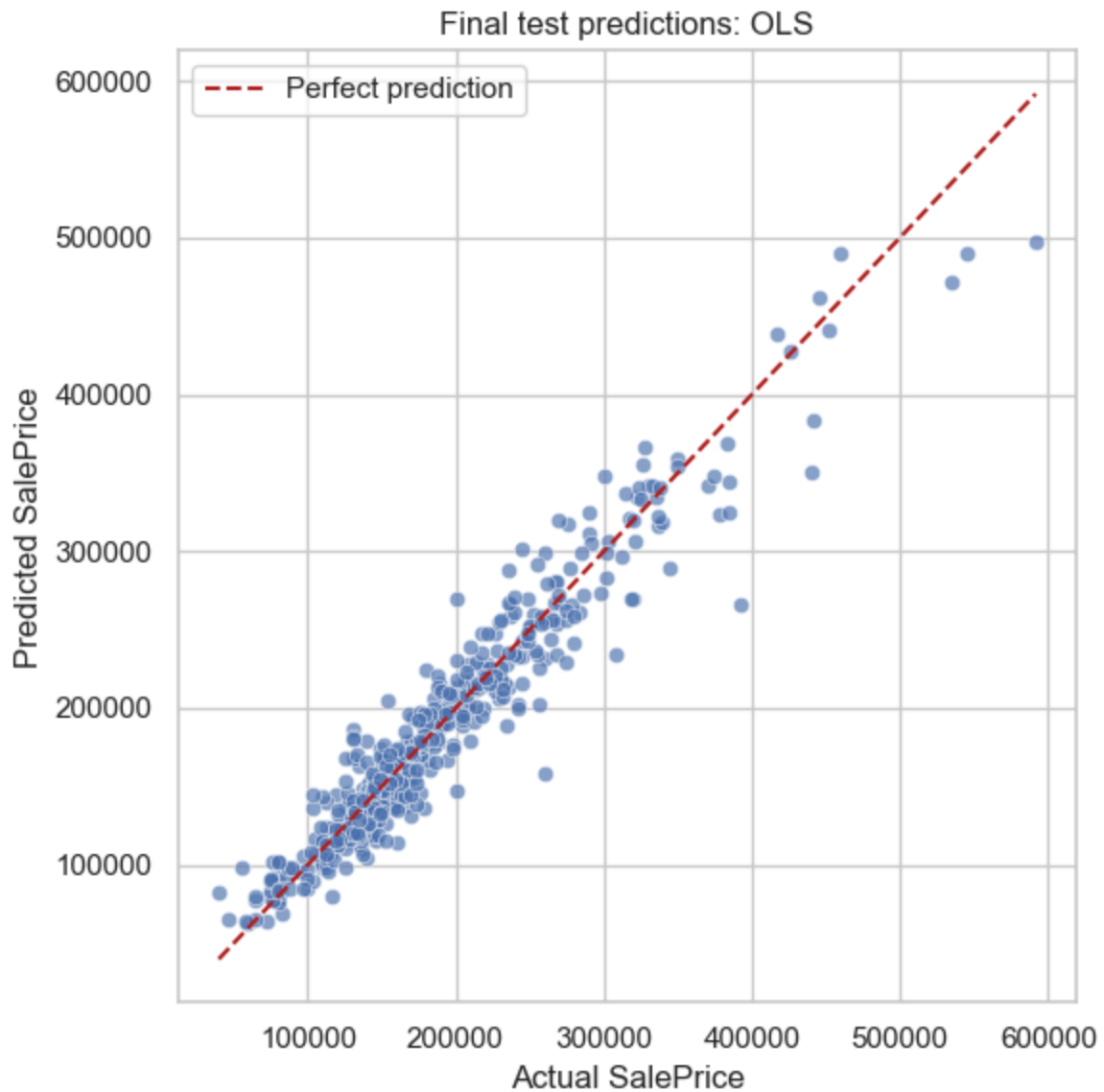
final_test_metrics = pd.DataFrame([
    'selected_model': best_model_name,
    'target_for_metrics': 'log1p(SalePrice)',
    'test_mse': test_metrics['mse'],
    'test_rmse': test_metrics['rmse'],
    'test_mae': test_metrics['mae'],
    'test_r2': test_metrics['r2'],
])
display(final_test_metrics.round(4))

dollar_metrics = pd.DataFrame([
    'test_rmse_usd': np.sqrt(mean_squared_error(y_test, test_prediction)),
    'test_mae_usd': mean_absolute_error(y_test, test_prediction),
])
display(dollar_metrics.round(2))

plot_data = pd.DataFrame({'actual_saleprice': y_test, 'predicted_saleprice': test_prediction})
plt.figure(figsize=(6, 6))
ax = sns.scatterplot(data=plot_data, x='actual_saleprice', y='predicted_saleprice', alpha=0.65)
limits = [plot_data.min().min(), plot_data.max().max()]
ax.plot(limits, limits, '--', color='firebrick', label='Perfect prediction')
ax.set(title=f'Final test predictions: {best_model_name}', xlabel='Actual SalePrice', ylabel='Predicted SalePrice')
ax.legend()
plt.tight_layout()
plt.show()
```

	selected_model	target_for_metrics	test_mse	test_rmse	test_mae	test_r2
0	OLS	log1p(SalePrice)	0.0144	0.1201	0.0862	0.9087

	test_rmse_usd	test_mae_usd
0	21545.45	14996.77



In [8]:

```
# Convert coefficients from standardized-target units back to log-price units.
target_log_std = final_model.transformer_.named_steps['scale'].scale_[0]
coefficient_table = pd.DataFrame({
    'feature': final_model.regressor_.named_steps['preprocess'].get_feature_names_out(),
    'log_price_coefficient': final_model.regressor_.named_steps['model'].coef_ * target_log_std,
})
coefficient_table['abs_coefficient'] = coefficient_table['log_price_coefficient'].abs()

# 1. Top 30 coefficients by absolute magnitude across all 47 features.
top_30_coefficients = coefficient_table.sort_values('abs_coefficient', ascending=False).head(30)

# 2. Categorical features: one ordinal basement-quality feature plus 25 neighborhood indicators.
categorical_features = ['BsmtQual_ordinal'] + neighbourhood_columns
categorical_coefficients = coefficient_table.set_index('feature').loc[categorical_features].reset_index()
categorical_coefficients['display_feature'] = categorical_coefficients['feature'].replace({'BsmtQual_ordinal':
'BsmtQual (ordinal)'})
categorical_coefficients['display_feature'] =
categorical_coefficients['display_feature'].str.removeprefix('NBHD_')
categorical_coefficients = categorical_coefficients.sort_values('log_price_coefficient', ascending=False)

# 3. Numerical features: all 21 numeric features in the assignment contract.
numerical_coefficients = coefficient_table.set_index('feature').loc[numerical_features].reset_index()
numerical_coefficients = numerical_coefficients.sort_values('log_price_coefficient', ascending=False)

print(f'Best model selected by validation MSE: {best_model_name}')
print('Coefficients are in log-price units. Numerical feature coefficients correspond to standardized
```

```

predictors.')
display(top_30_coefficients[['feature', 'log_price_coefficient', 'abs_coefficient']].round(8))
display(categorical_coefficients[['display_feature', 'log_price_coefficient']].round(8))
display(numerical_coefficients[['feature', 'log_price_coefficient']].round(8))

# Matplotlib bars avoid seaborn-version differences in the legend argument.
def plot_coefficient_bars(table, label_column, title, figsize):
    values = table['log_price_coefficient'].to_numpy()
    labels = table[label_column].astype(str).tolist()
    color_limit = max(float(np.abs(values).max()), 1e-12)
    cmap = plt.get_cmap('coolwarm')
    colors = cmap((values / color_limit + 1.0) / 2.0)
    fig, axis = plt.subplots(figsize=figsize)
    positions = np.arange(len(table))
    axis.barh(positions, values, color=colors)
    axis.set_yticks(positions)
    axis.set_yticklabels(labels)
    axis.invert_yaxis()
    axis.axvline(0, color='gray', linewidth=0.8)
    axis.set(title=title, xlabel='Coefficient (log-price units)', ylabel='')
    plt.tight_layout()
    plt.show()

plot_coefficient_bars(
    top_30_coefficients.sort_values('log_price_coefficient'), 'feature',
    'Top 30 coefficients by absolute magnitude', (10, 11))
plot_coefficient_bars(
    categorical_coefficients, 'display_feature',
    'Categorical and ordinal feature coefficients', (10, 10))
plot_coefficient_bars(
    numerical_coefficients, 'feature',
    'Numerical feature coefficients', (10, 8))

```

Best model selected by validation MSE: OLS

Coefficients are in log-price units. Numerical feature coefficients correspond to standardized predictors.

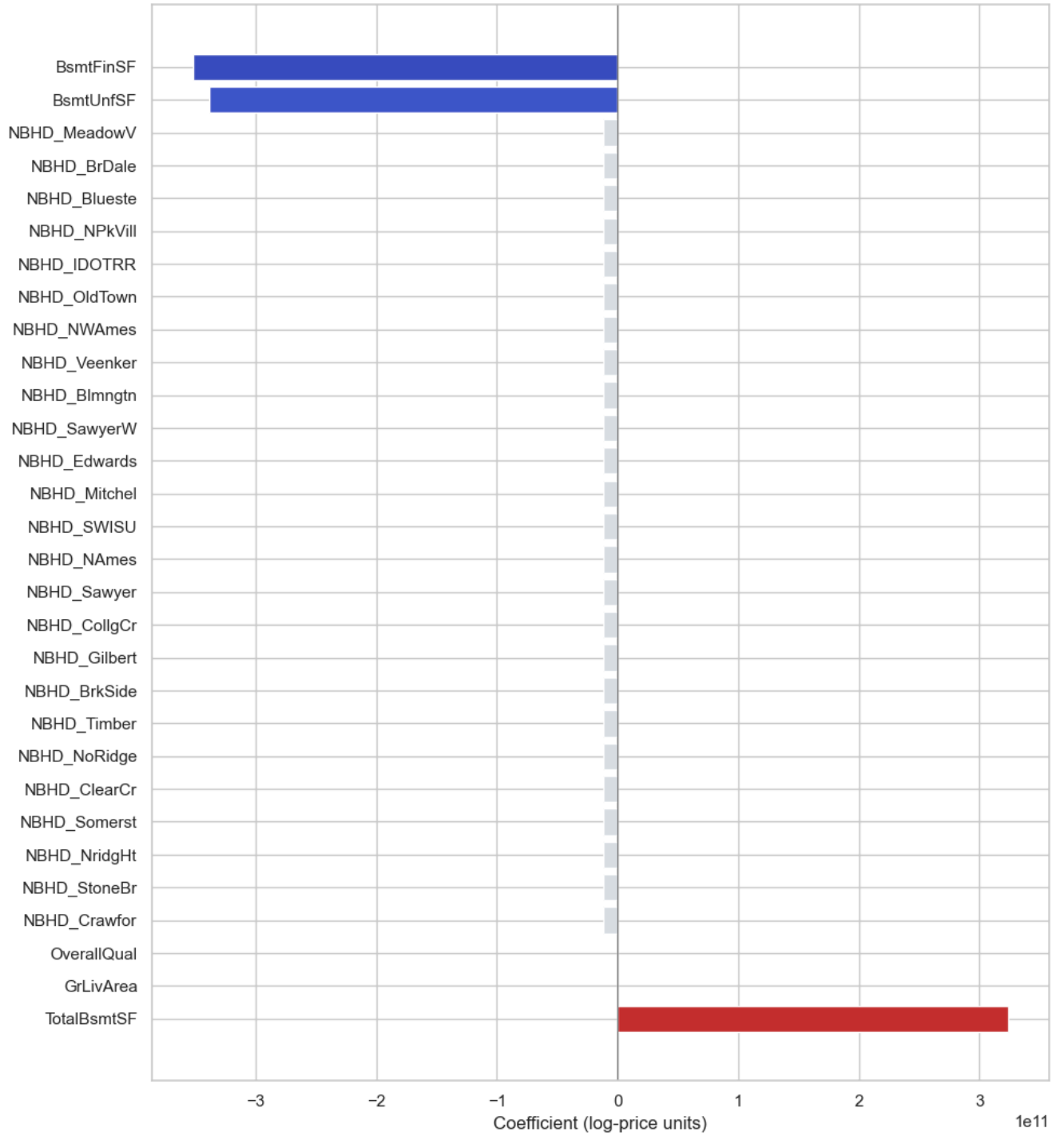
	feature	log_price_coefficient	abs_coefficient
5	BsmtFinSF	-3.521116e+11	3.521116e+11
6	BsmtUnfSF	-3.382053e+11	3.382053e+11
7	TotalBsmtSF	3.239798e+11	3.239798e+11
32	NBHD_MeadowV	-1.209319e+10	1.209319e+10
24	NBHD_BrDale	-1.209319e+10	1.209319e+10
23	NBHD_Blueste	-1.209319e+10	1.209319e+10
35	NBHD_NPkVill	-1.209319e+10	1.209319e+10
31	NBHD_IDOTRR	-1.209319e+10	1.209319e+10
39	NBHD_OldTown	-1.209319e+10	1.209319e+10
36	NBHD_NWAmes	-1.209319e+10	1.209319e+10
46	NBHD_Veenker	-1.209319e+10	1.209319e+10
22	NBHD_Blmngtn	-1.209319e+10	1.209319e+10
42	NBHD_SawyerW	-1.209319e+10	1.209319e+10
29	NBHD_Edwards	-1.209319e+10	1.209319e+10
33	NBHD_Mitchel	-1.209319e+10	1.209319e+10
40	NBHD_SWISU	-1.209319e+10	1.209319e+10
34	NBHD_NAmes	-1.209319e+10	1.209319e+10
41	NBHD_Sawyer	-1.209319e+10	1.209319e+10
27	NBHD_CollgCr	-1.209319e+10	1.209319e+10
30	NBHD_Gilbert	-1.209319e+10	1.209319e+10
25	NBHD_BrkSide	-1.209319e+10	1.209319e+10
45	NBHD_Timber	-1.209319e+10	1.209319e+10
37	NBHD_NoRidge	-1.209319e+10	1.209319e+10
26	NBHD_ClearCr	-1.209319e+10	1.209319e+10
43	NBHD_Somerst	-1.209319e+10	1.209319e+10
38	NBHD_NridgHt	-1.209319e+10	1.209319e+10
44	NBHD_StoneBr	-1.209319e+10	1.209319e+10
28	NBHD_Crawfor	-1.209319e+10	1.209319e+10

	feature	log_price_coefficient	abs_coefficient
10	GrLivArea	1.110710e-01	1.110710e-01
1	OverallQual	9.898155e-02	9.898155e-02

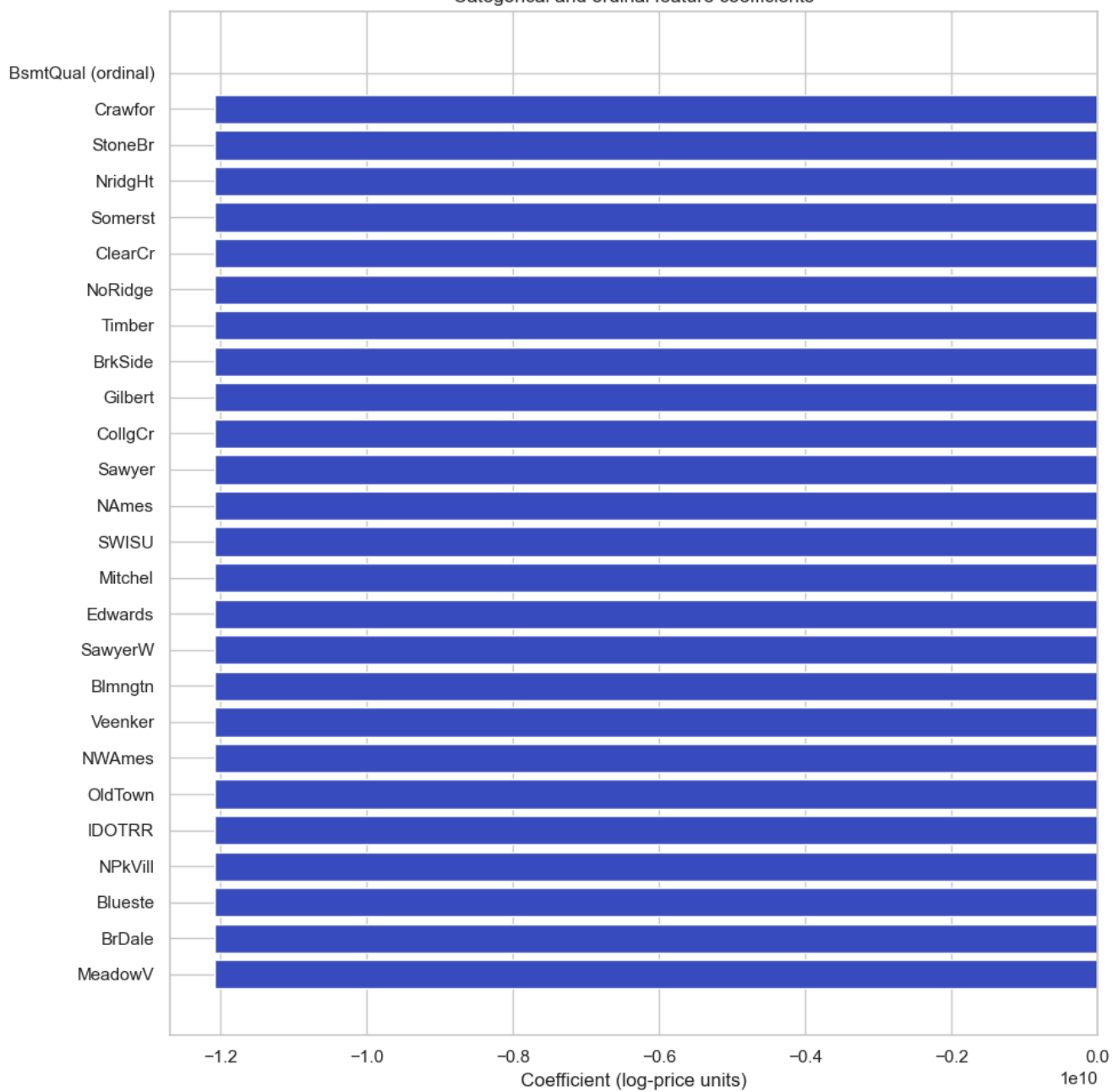
	display_feature	log_price_coefficient
0	BsmtQual (ordinal)	1.852880e-02
7	Crawfor	-1.209319e+10
23	StoneBr	-1.209319e+10
17	NridgHt	-1.209319e+10
22	Somerst	-1.209319e+10
5	ClearCr	-1.209319e+10
16	NoRidge	-1.209319e+10
24	Timber	-1.209319e+10
4	BrkSide	-1.209319e+10
9	Gilbert	-1.209319e+10
6	CollgCr	-1.209319e+10
20	Sawyer	-1.209319e+10
13	NAmes	-1.209319e+10
19	SWISU	-1.209319e+10
12	Mitchel	-1.209319e+10
8	Edwards	-1.209319e+10
21	SawyerW	-1.209319e+10
1	Blmngtn	-1.209319e+10
25	Veenker	-1.209319e+10
15	NWAmes	-1.209319e+10
18	OldTown	-1.209319e+10
10	IDOTRR	-1.209319e+10
14	NPKvill	-1.209319e+10
2	Blueste	-1.209319e+10
3	BrDale	-1.209319e+10
11	MeadowV	-1.209319e+10

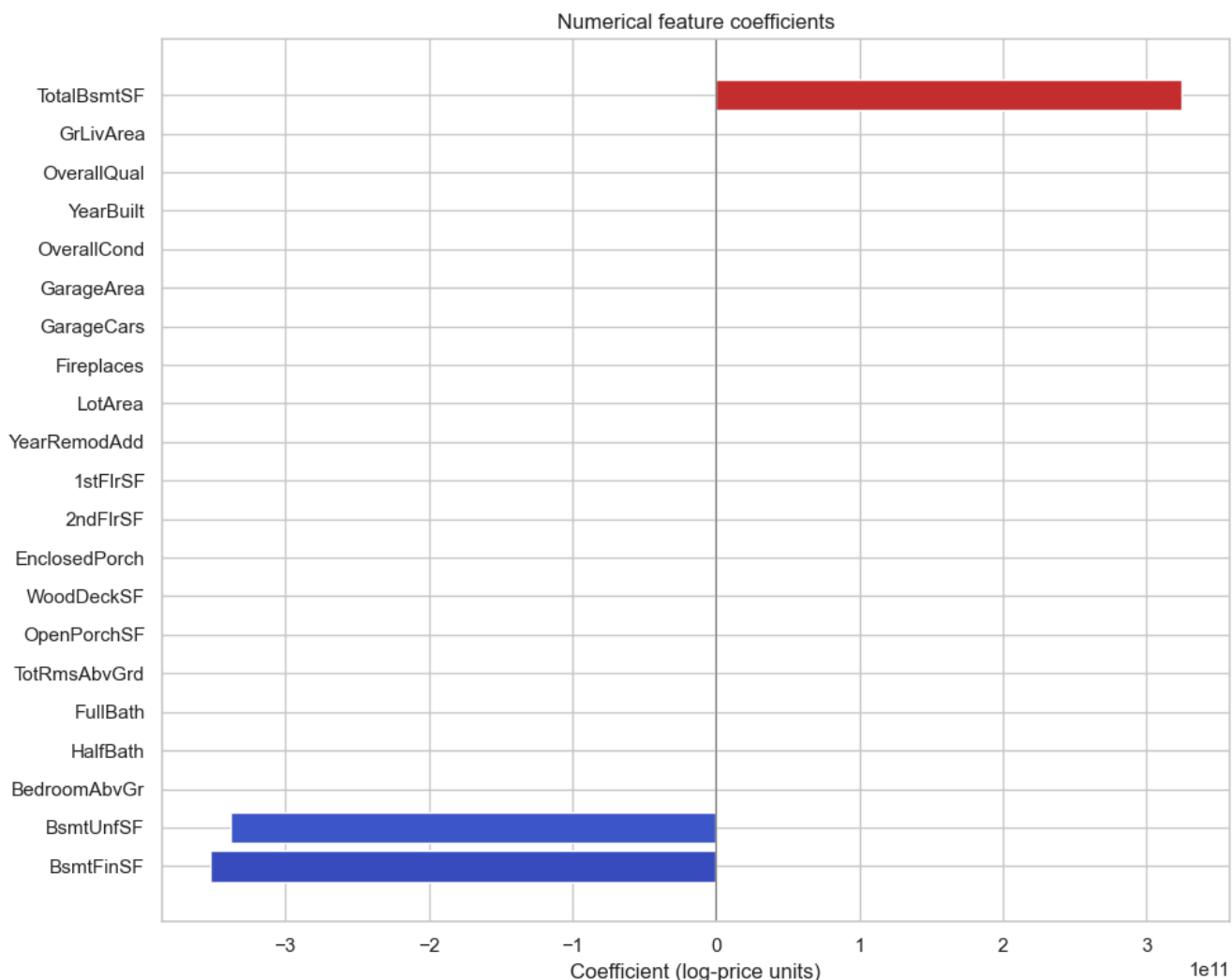
	feature	log_price_coefficient
7	TotalBsmtSF	3.239798e+11
10	GrLivArea	1.110710e-01
1	OverallQual	9.898155e-02
3	YearBuilt	8.692729e-02
2	OverallCond	6.192426e-02
17	GarageArea	2.337701e-02
16	GarageCars	2.134930e-02
15	Fireplaces	2.000200e-02
0	LotArea	1.901989e-02
4	YearRemodAdd	1.813341e-02
8	1stFlrSF	1.621551e-02
9	2ndFlrSF	1.010066e-02
20	EnclosedPorch	9.479710e-03
18	WoodDeckSF	7.128390e-03
19	OpenPorchSF	6.454810e-03
14	TotRmsAbvGrd	5.609670e-03
11	FullBath	4.377830e-03
12	HalfBath	3.737050e-03
13	BedroomAbvGr	-6.267540e-03
6	BsmtUnfSF	-3.382053e+11
5	BsmtFinSF	-3.521116e+11

Top 30 coefficients by absolute magnitude



Categorical and ordinal feature coefficients





Interpretation guardrails

The reported primary metrics are on the log-price scale, so they measure proportional pricing error more naturally than dollar-scale errors. Dollar RMSE and MAE are supplied only as an intuitive supplement for the final selected model. Coefficients are conditional prediction associations, not causal effects. Retaining all 25 neighborhood dummies honors the 47-feature contract; with an intercept, OLS dummy coefficients are not individually identified even though its predictions are well-defined.

Complete parameters of the final refitted model

This table uses the same units as the candidate table, but reflects the final fit on 2,400 observations.

In [9]:

```
final_parameter_table = original_input_log_parameters(final_model, X_development).to_frame(
    name=f'Final {best_model_name}')
final_parameter_table.index.name = 'Parameter'
print('Final model: refitted on 2,400 training + validation observations.')
print('The equation and units match the candidate parameter table above.')
with pd.option_context('display.max_rows', None, 'display.float_format', lambda v: f'{v:.10g}'):
    display(final_parameter_table)
```

Note: LinearRegression has cancelling coefficients; parameter reconstruction differs by up to 0.000384 log-price units. Use the fitted pipeline for predictions and do not interpret individual OLS coefficients as stable effects. Final model: refitted on 2,400 training + validation observations. The equation and units match the candidate parameter table above.

Final OLS	
Parameter	
Intercept	1.209319275e+10
LotArea	2.39631659e-06
OverallQual	0.07085582747
OverallCond	0.05565092549
YearBuilt	0.002859252048
YearRemodAdd	0.0008639002408
BsmtFinSF	-767725470.6
BsmtUnfSF	-767725470.6
TotalBsmtSF	767725470.6
1stFlrSF	4.356442862e-05
2ndFlrSF	2.370438939e-05
GrLivArea	0.0002288115003
FullBath	0.007945166856
HalfBath	0.007461682507
BedroomAbvGr	-0.007710720865
TotRmsAbvGrd	0.0036594773
Fireplaces	0.03114575411
GarageCars	0.02818816371
GarageArea	0.000109468182
WoodDeckSF	5.780064197e-05
OpenPorchSF	9.744374914e-05
EnclosedPorch	0.0001536698883
BsmtQual_ordinal	0.02046510133
NBHD_Blmngtn	-1.209319274e+10
NBHD_Blueste	-1.209319274e+10
NBHD_BrDale	-1.209319274e+10
NBHD_BrkSide	-1.209319274e+10
NBHD_ClearCr	-1.209319274e+10
NBHD_CollgCr	-1.209319274e+10
NBHD_Crawfor	-1.209319274e+10
NBHD_Edwards	-1.209319274e+10
NBHD_Gilbert	-1.209319274e+10
NBHD_IDOTRR	-1.209319274e+10
NBHD_MeadowV	-1.209319274e+10
NBHD_Mitchel	-1.209319274e+10
NBHD_NAmes	-1.209319274e+10
NBHD_NPkvill	-1.209319274e+10
NBHD_NWAmes	-1.209319274e+10
NBHD_NoRidge	-1.209319274e+10
NBHD_NridgHt	-1.209319274e+10
NBHD_OldTown	-1.209319274e+10
NBHD_SWISU	-1.209319274e+10
NBHD_Sawyer	-1.209319274e+10
NBHD_SawyerW	-1.209319274e+10
NBHD_Somerst	-1.209319274e+10
NBHD_StoneBr	-1.209319274e+10
NBHD_Timber	-1.209319274e+10
NBHD_Veenker	-1.209319274e+10

Optional Zillow case study

The project brief optionally asks for a current Iowa listing, a model prediction, and a comparison with the listing price and Zillow Zestimate. The selected property is **124 N Franklin Ave, Ames, IA 50014**, observed on October 4, 2026.

- Active Zillow listing: https://www.zillow.com/homedetails/124-N-Franklin-Ave-Ames-IA-50014/459888892_zpid/
- Zillow values used: list price **\$246,500** and Zestimate **\$242,000**.
- The same parcel (0905476120) sold for **\$119,000 in April 2008** and **\$186,820 in April 2022**.
- The current listing reports 4 bedrooms, 2 bathrooms, 1,262 square feet, a partial basement with finished living space, a one-car detached garage, a covered deck, and year built 1900.
- The exact 2008 property appears in the course spreadsheet as `Id = 391`, with neighborhood `Edwards`. Under the fixed random split, this observation belongs to the held-out test set and was not used to fit the final model.

The historical row supplies the detailed Ames variables that current websites do not report. For the current-listing prediction, the mapping updates the above-grade area, bathroom count, and above-grade bedroom count to the current listing while retaining the historical structural fields. Public sources report slightly different square-footage and lot-area values, so the assumptions are shown explicitly. This is an illustrative model application, not an appraisal.

In [10]:

```
# Current and historical values captured on October 4, 2026.
zillow_address = '124 N Franklin Ave, Ames, IA 50014'
zillow_list_price = 246_500.0
zillow_zestimate = 242_000.0
historical_sale_price = 119_000.0
sale_2022_price = 186_820.0
course_property_id = 391

# Locate the exact 2008 course-data record. Public records and the spreadsheet agree
# on the 1900 build year, 8,405-square-foot lot, April 2008 sale, and $119,000 price.
property_matches = clean.index[clean['Id'].eq(course_property_id)]
assert len(property_matches) == 1
historical_index = int(property_matches[0])
assert clean.loc[historical_index, 'YrSold'] == 2008
assert clean.loc[historical_index, 'MoSold'] == 4
assert clean.loc[historical_index, 'SalePrice'] == historical_sale_price
assert clean.loc[historical_index, 'Neighborhood'] == 'Edwards'
assert historical_index in test_idx
assert historical_index not in train_idx and historical_index not in validation_idx

historical_property_row = X_47.loc[[historical_index]].copy()
historical_property_row.index = ['2008 course-data record']

# Start from the exact historical property record. Update only fields that the current
# listing reports differently. The first-floor area remains 961 square feet; the
# current 1,262-square-foot total implies 301 square feet on the upper level.
zillow_row = historical_property_row.copy()
zillow_row.index = [zillow_address]
zillow_row.loc[zillow_address, '2ndFlrSF'] = 301
zillow_row.loc[zillow_address, 'GrLivArea'] = 1_262
zillow_row.loc[zillow_address, 'FullBath'] = 2 # one full plus one three-quarter bath
zillow_row.loc[zillow_address, 'BedroomAbvGr'] = 3 # fourth bedroom is in the basement

assert list(zillow_row.columns) == list(X_47.columns)
assert not zillow_row.isna().any().any()
assert zillow_row[neighbourhood_columns].sum(axis=1).eq(1).all()

changed_features = ['2ndFlrSF', 'GrLivArea', 'FullBath', 'BedroomAbvGr']
input_audit = pd.DataFrame({
    '2008_course_record': historical_property_row.iloc[0],
    'current_value_used': zillow_row.iloc[0],
})
input_audit['mapping'] = 'retained from exact 2008 property record'
input_audit.loc[changed_features, 'mapping'] = 'updated from current listing'
display(input_audit)
```

	2008_course_record	current_value_used	mapping
LotArea	8405	8405	retained from exact 2008 property record
OverallQual	5	5	retained from exact 2008 property record
OverallCond	8	8	retained from exact 2008 property record
YearBuilt	1900	1900	retained from exact 2008 property record
YearRemodAdd	1950	1950	retained from exact 2008 property record
BsmtFinSF	632	632	retained from exact 2008 property record
BsmtUnfSF	229	229	retained from exact 2008 property record
TotalBsmtSF	861	861	retained from exact 2008 property record
1stFlrSF	961	961	retained from exact 2008 property record
2ndFlrSF	406	301	updated from current listing
GrLivArea	1367	1262	updated from current listing
FullBath	1	2	updated from current listing
HalfBath	0	0	retained from exact 2008 property record
BedroomAbvGr	4	3	updated from current listing
TotRmsAbvGrd	7	7	retained from exact 2008 property record
Fireplaces	0	0	retained from exact 2008 property record
GarageCars	1	1	retained from exact 2008 property record
GarageArea	384	384	retained from exact 2008 property record
WoodDeckSF	0	0	retained from exact 2008 property record
OpenPorchSF	130	130	retained from exact 2008 property record
EnclosedPorch	112	112	retained from exact 2008 property record
BsmtQual_ordinal	3	3	retained from exact 2008 property record
NBHD_Blmngtn	0	0	retained from exact 2008 property record
NBHD_Blueste	0	0	retained from exact 2008 property record
NBHD_BrDale	0	0	retained from exact 2008 property record
NBHD_BrkSide	0	0	retained from exact 2008 property record
NBHD_ClearCr	0	0	retained from exact 2008 property record
NBHD_CollgCr	0	0	retained from exact 2008 property record
NBHD_Crawfor	0	0	retained from exact 2008 property record
NBHD_Edwards	1	1	retained from exact 2008 property record
NBHD_Gilbert	0	0	retained from exact 2008 property record
NBHD_IDOTRR	0	0	retained from exact 2008 property record
NBHD_MeadowV	0	0	retained from exact 2008 property record
NBHD_Mitchel	0	0	retained from exact 2008 property record
NBHD_NAmes	0	0	retained from exact 2008 property record
NBHD_NPkVill	0	0	retained from exact 2008 property record
NBHD_NWAmes	0	0	retained from exact 2008 property record
NBHD_NoRidge	0	0	retained from exact 2008 property record
NBHD_NridgHt	0	0	retained from exact 2008 property record
NBHD_OldTown	0	0	retained from exact 2008 property record
NBHD_SWISU	0	0	retained from exact 2008 property record
NBHD_Sawyer	0	0	retained from exact 2008 property record
NBHD_SawyerW	0	0	retained from exact 2008 property record
NBHD_Somerst	0	0	retained from exact 2008 property record
NBHD_StoneBr	0	0	retained from exact 2008 property record
NBHD_Timber	0	0	retained from exact 2008 property record
NBHD_Veenker	0	0	retained from exact 2008 property record

In [11]:

```
# The exact 2008 property record is in the test set, so the final model did
# not fit on this observation. First check its in-period historical prediction.
historical_ols_prediction = float(final_model.predict(historical_property_row)[0])

# Then apply the same fitted model to the current-listing feature mapping.
zillow_ols_prediction = float(final_model.predict(zillow_row)[0])

# Ridge alpha=0.60 is included as a stability comparison because its validation
# MSE is nearly identical and its coefficients are well behaved under collinearity.
zillow_ridge_model = make_log_target_model(Ridge(alpha=0.60))
```

```

zillow_ridge_model.fit(X_development, y_development)
historical_ridge_prediction = float(zillow_ridge_model.predict(historical_property_row)[0])
zillow_ridge_prediction = float(zillow_ridge_model.predict(zillow_row)[0])

historical_comparison = pd.DataFrame({
    'value_usd': [historical_sale_price, historical_ols_prediction, historical_ridge_prediction],
}, index=[
    '2008 actual sale price',
    'OLS prediction from 2008 features',
    'Ridge alpha=0.60 prediction from 2008 features',
])
historical_comparison['difference_from_2008_sale_usd'] = (
    historical_comparison['value_usd'] - historical_sale_price
)
historical_comparison['difference_from_2008_sale_pct'] = 100 * (
    historical_comparison['value_usd'] / historical_sale_price - 1
)
display(historical_comparison.round(2))

current_comparison = pd.DataFrame({
    'value_usd': [
        sale_2022_price,
        zillow_ols_prediction,
        zillow_ridge_prediction,
        zillow_list_price,
        zillow_zestimate,
    ],
}, index=[
    '2022 sale price',
    'Current-feature OLS prediction',
    'Current-feature Ridge alpha=0.60 prediction',
    '2026 Zillow listing price',
    '2026 Zillow Zestimate',
])
current_comparison['difference_from_list_usd'] = current_comparison['value_usd'] - zillow_list_price
current_comparison['difference_from_list_pct'] = 100 * (
    current_comparison['value_usd'] / zillow_list_price - 1
)
display(current_comparison.round(2))

print(f'2008 actual sale: ${historical_sale_price:,.2f}')
print(f'OLS prediction from 2008 features: ${historical_ols_prediction:,.2f}')
print(f'Ridge prediction from 2008 features: ${historical_ridge_prediction:,.2f}')
print(f'2022 sale: ${sale_2022_price:,.2f}')
print(f'Current-feature OLS prediction: ${zillow_ols_prediction:,.2f}')
print(f'Current-feature Ridge sensitivity prediction: ${zillow_ridge_prediction:,.2f}')
print(f'2026 listing price: ${zillow_list_price:,.2f}')
print(f'2026 Zestimate: ${zillow_zestimate:,.2f}')

```

	value_usd	difference_from_2008_sale_usd	difference_from_2008_sale_pct
2008 actual sale price	119000.00	0.00	0.00
OLS prediction from 2008 features	123195.25	4195.25	3.53
Ridge alpha=0.60 prediction from 2008 features	122873.90	3873.90	3.26

	value_usd	difference_from_list_usd	difference_from_list_pct
2022 sale price	186820.00	-59680.00	-24.21
Current-feature OLS prediction	121868.72	-124631.28	-50.56
Current-feature Ridge alpha=0.60 prediction	121450.58	-125049.42	-50.73
2026 Zillow listing price	246500.00	0.00	0.00
2026 Zillow Zestimate	242000.00	-4500.00	-1.83

2008 actual sale: \$119,000.00
 OLS prediction from 2008 features: \$123,195.25
 Ridge prediction from 2008 features: \$122,873.90
 2022 sale: \$186,820.00
 Current-feature OLS prediction: \$121,868.72
 Current-feature Ridge sensitivity prediction: \$121,450.58

2026 listing price: \$246,500.00
2026 Zestimate: \$242,000.00

Interpretation of the Zillow comparison

Using the exact 2008 course-data features, the selected OLS model predicts approximately **\$123,195**, only **\$4,195 (3.53%) above** the property's actual 2008 sale price of \$119,000. Because this observation is in the held-out test set, that comparison is not an in-sample fitted value.

After updating the area, bathroom, and above-grade bedroom fields to the current listing, OLS predicts approximately **\$121,869** and Ridge $\alpha=0.60$ predicts approximately **\$121,451**. Both are about **50.6% below** the \$246,500 listing price, while the \$242,000 Zestimate is only \$4,500 (1.83%) below the listing price. The same parcel sold for \$186,820 in 2022.

The strong 2008 comparison and weak 2026 comparison isolate the main limitation: the course model is calibrated to nominal 2006--2010 prices and contains no sale-year, inflation, or housing-market index. Some current public fields also differ from the historical assessor record. The current estimate should therefore be interpreted as a historical-price-scale prediction rather than a present-day appraisal.

Supplementary Appendix 1

01_eda(2).ipynb

Complete code and saved outputs from the uploaded notebook. Execution counts are preserved; cells have not been rerun.

Iowa House Price Prediction - EDA

Workflow: construct the 47 specified regressors, audit and clean data, visualize `SalePrice`, and visualize numeric-regressor correlations.

Cell 2 | In [1]:

```
from pathlib import Path
print(Path.cwd())
```

Output 1

/Users/kerolin/Desktop/python_work/FIM590003Project/notebooks

1. Load data and construct the 47 specified regressors

The 47 regressors comprise 21 numeric features, ordinal `BsmtQual`, and 25 fixed `Neighborhood` indicators. `SalePrice` is the target.

Cell 4 | In [2]:

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

pd.set_option("display.max_columns", 100)
sns.set_theme(style="whitegrid", context="notebook")

DATA_FILENAME = "IA_House_Price_Original_Data.xlsx"
DATA_CANDIDATES = [Path("../data") / DATA_FILENAME, Path("data") / DATA_FILENAME]
DATA_PATH = next((path for path in DATA_CANDIDATES if path.exists()), None)
if DATA_PATH is None:
    raise FileNotFoundError(f"Could not find {DATA_FILENAME} in the repository data directory.")
TARGET = "SalePrice"
numeric_features = [
    "LotArea", "OverallQual", "OverallCond", "YearBuilt", "YearRemodAdd", "BsmtFinSF",
    "BsmtUnfSF", "TotalBsmtSF", "1stFlrSF", "2ndFlrSF", "GrLivArea", "FullBath",
    "HalfBath", "BedroomAbvGr", "TotRmsAbvGrd", "Fireplaces", "GarageCars", "GarageArea",
    "WoodDeckSF", "OpenPorchSF", "EnclosedPorch",
]
continuous_features = [
    "LotArea", "YearBuilt", "YearRemodAdd", "BsmtFinSF", "BsmtUnfSF", "TotalBsmtSF",
    "1stFlrSF", "2ndFlrSF", "GrLivArea", "GarageArea", "WoodDeckSF", "OpenPorchSF", "EnclosedPorch",
]
neighbourhoods = [
    "Blmngtn", "Blueste", "BrDale", "BrkSide", "ClearCr", "CollgCr", "Crawfor", "Edwards",
    "Gilbert", "IDOTRR", "MeadowV", "Mitchel", "Names", "NPkVill", "NWAmes", "NoRidge",
    "NridgHt", "OldTown", "SWISU", "Sawyer", "SawyerW", "Somerst", "StoneBr", "Timber", "Veenker",
]
source_columns = [feature for feature in numeric_features if feature != "BsmtFinSF"] + ["BsmtFinSF1",
"BsmtFinSF2", "BsmtQual", "Neighborhood", TARGET]

raw = pd.read_excel(DATA_PATH, skiprows=3)
raw.columns = raw.columns.astype(str).str.strip()
blank_layout_columns = raw.columns[raw.columns.str.startswith("Unnamed")].tolist()
raw = raw.drop(columns=blank_layout_columns)
missing_source_columns = sorted(set(source_columns) - set(raw.columns))
if missing_source_columns:
    raise KeyError(f"Required source columns are missing: {missing_source_columns}")
df = raw.loc[:, source_columns].copy()
```

```
print(f"Observations: {len(df):,}")
print(f"Blank layout columns removed: {blank_layout_columns}")
print(f"Selected source columns, including target: {df.shape[1]}")
```

Output 1

/Users/kerolin/anaconda3/lib/python3.11/site-packages/pandas/core/arrays/masked.py:61: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
from pandas.core import (

Output 2

```
Observations: 2,908
Blank layout columns removed: ['Unnamed: 81', 'Unnamed: 82']
Selected source columns, including target: 25
```

2. Check data status and clean data

This step checks column availability, duplicates and missing values, then constructs `BsmtFinSF`, maps `BsmtQual` (`Ex/Gd/TA/Fa/Po/NA` to `5/4/3/2/1/0`), and creates the 25 fixed neighborhood indicators.

Cell 6 | In [3]:

```
data_status = pd.Series({
    "observations": len(df), "duplicate_rows": int(df.duplicated().sum()),
    "missing_target": int(df[TARGET].isna().sum()), "source_columns": df.shape[1],
})
display(data_status.to_frame("value"))

missingness = (df.isna().sum().to_frame("missing_count")
               .assign(missing_pct=lambda t: 100 * t["missing_count"] / len(df))
               .query("missing_count > 0").sort_values("missing_count", ascending=False))
display(missingness)

clean = df.copy()
clean["BsmtFinSF"] = clean["BsmtFinSF1"] + clean["BsmtFinSF2"]
bsmtqual_map = {"Ex": 5, "Gd": 4, "TA": 3, "Fa": 2, "Po": 1, "NA": 0}
clean["BsmtQual_ordinal"] = clean["BsmtQual"].fillna("NA").map(bsmtqual_map)

observed_neighbourhoods = set(clean["Neighborhood"].dropna())
if observed_neighbourhoods != set(neighbourhoods):
    raise ValueError("Neighborhood categories do not match the 25-feature specification.")
dummies = pd.get_dummies(clean["Neighborhood"], prefix="NBHD", dtype=int).reindex(
    columns=[f"NBHD_{name}" for name in neighbourhoods], fill_value=0
)
X_47 = pd.concat([clean[numeric_features], clean[["BsmtQual_ordinal"]], dummies], axis=1)
y = clean[TARGET].copy()
if X_47.isna().any().any() or y.isna().any():
    raise ValueError("Missing values remain; define a train-only imputation or exclusion rule before modelling.")
feature_status = pd.Series({
    "numeric_regressors": len(numeric_features), "ordinal_regressors": 1,
    "neighbourhood_indicators": dummies.shape[1], "total_regressors": X_47.shape[1],
    "missing_values_in_47_regressors": int(X_47.isna().sum().sum()),
})
display(feature_status.to_frame("value"))
assert X_47.shape[1] == 47
```

Output 1

	value
observations	2908
duplicate_rows	4
missing_target	0
source_columns	25

Output 2

	missing_count	missing_pct
BsmtQual	79	2.716644

Output 3

	value
numeric_regressors	21
ordinal_regressors	1
neighbourhood_indicators	25
total_regressors	47
missing_values_in_47_regressors	0

3. Univariate analysis

This section examines each regressor's relationship with `SalePrice`. It is descriptive: it does not delete observations, transform the target, or select a final model.

Cell 8 | In [4]:

```
discrete_features = [feature for feature in numeric_features if feature not in continuous_features]
continuous_y_summary = pd.DataFrame({
    "pearson_correlation_with_saleprice": X_47[continuous_features].corrwith(y, method="pearson"),
    "spearman_correlation_with_saleprice": X_47[continuous_features].corrwith(y, method="spearman"),
}).assign(abs_pearson=lambda table: table["pearson_correlation_with_saleprice"].abs())
continuous_y_summary = continuous_y_summary.sort_values("abs_pearson", ascending=False)
display(continuous_y_summary.round(3))
```

Output 1

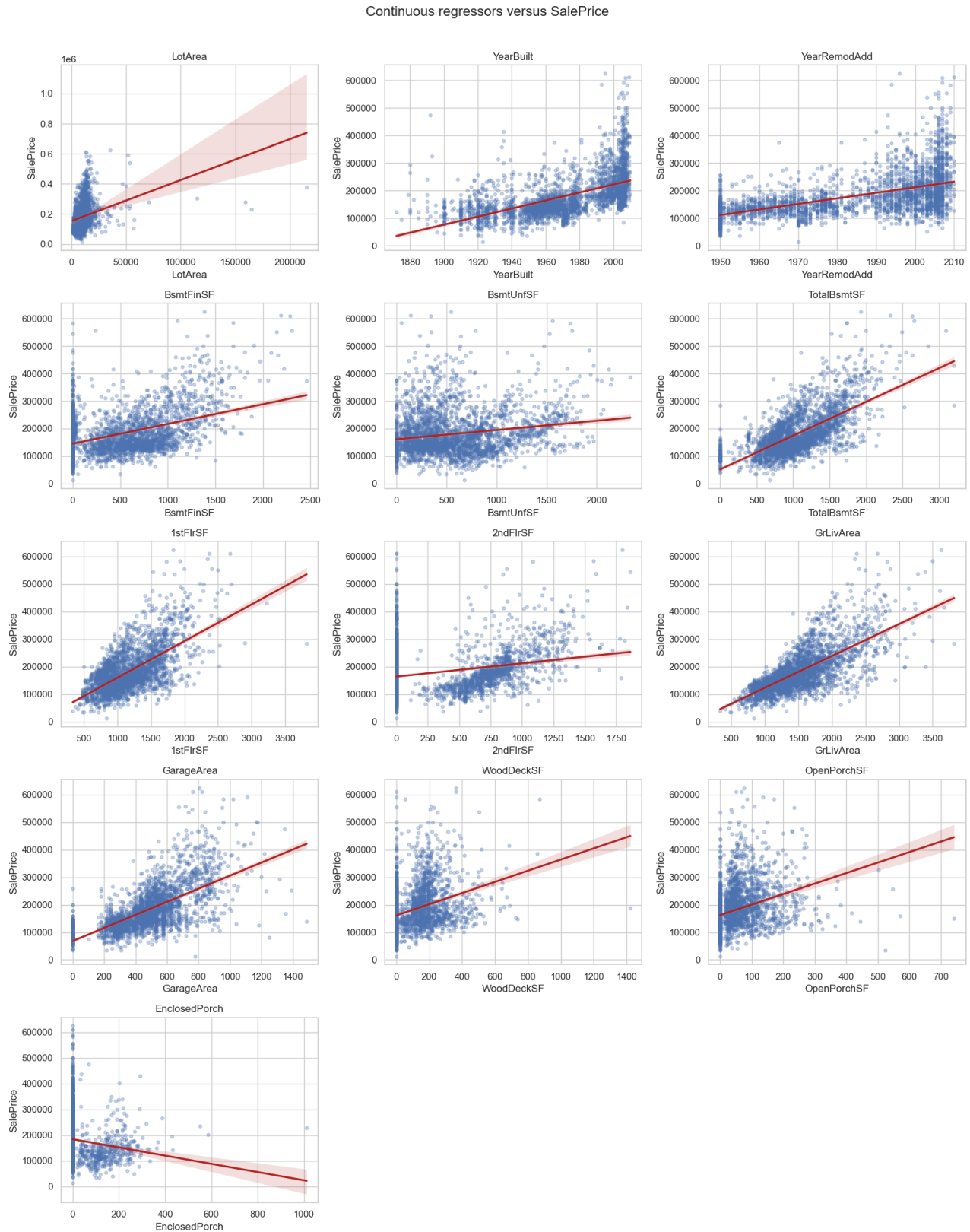
	pearson_correlation_with_saleprice	spearman_correlation_with_saleprice	abs_pearson
GrLivArea	0.721	0.725	0.721
TotalBsmtSF	0.659	0.607	0.659
GarageArea	0.651	0.662	0.651
1stFlrSF	0.640	0.579	0.640
YearBuilt	0.566	0.681	0.566
YearRemodAdd	0.541	0.602	0.541
BsmtFinSF	0.421	0.328	0.421
WoodDeckSF	0.328	0.363	0.328
OpenPorchSF	0.325	0.481	0.325
LotArea	0.272	0.433	0.272
2ndFlrSF	0.262	0.249	0.262
BsmtUnfSF	0.189	0.166	0.189
EnclosedPorch	-0.131	-0.235	0.131

Cell 9 | In [5]:

```
fig, axes = plt.subplots(5, 3, figsize=(16, 20))
for feature, axis in zip(continuous_features, axes.flat):
    sns.regplot(data=pd.DataFrame({feature: X_47[feature], TARGET: y}), x=feature, y=TARGET,
        scatter_kws={"alpha": 0.30, "s": 14}, line_kws={"color": "firebrick"}, ax=axis)
    axis.set_title(feature)
for axis in axes.flat[len(continuous_features):]:
    axis.set_visible(False)
fig.suptitle("Continuous regressors versus SalePrice", y=1.01, fontsize=16)
plt.tight_layout()
```

```
plt.show()
```

Output 1



Cell 10 | In [25]:

```
fig, axes = plt.subplots(3, 3, figsize=(16, 13))
```

```

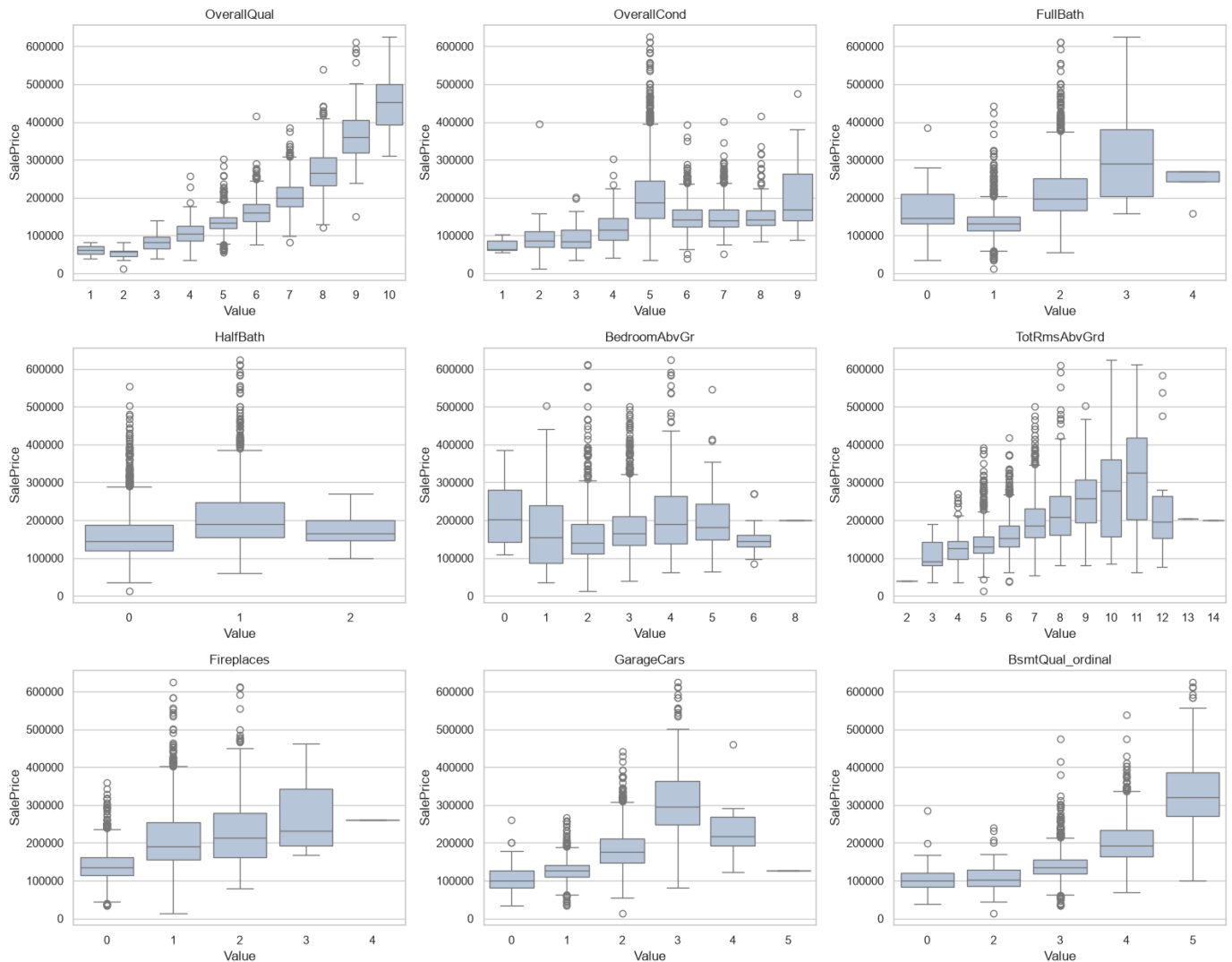
for feature, axis in zip(discrete_features + ["BsmtQual_ordinal"], axes.flat):
    plot_data = pd.DataFrame({feature: X_47[feature], TARGET: y})
    sns.boxplot(data=plot_data, x=feature, y=TARGET, color="lightsteelblue", ax=axis)
    axis.set_title(feature)
    axis.set_xlabel("Value")
    axis.set_ylabel("SalePrice")
for axis in axes.flat[len(discrete_features) + 1:]:
    axis.set_visible(False)
fig.suptitle("Count and ordinal regressors versus SalePrice", y=1.01, fontsize=16)
plt.tight_layout()
plt.show()

neighbourhood_y = clean.groupby("Neighborhood")[TARGET].agg(observations="size", median_saleprice="median",
mean_saleprice="mean").sort_values("median_saleprice", ascending=False)
plt.figure(figsize=(15, 6))
axis = sns.barplot(x=neighbourhood_y.index, y=neighbourhood_y["median_saleprice"], color="steelblue")
for position, observations in enumerate(neighbourhood_y["observations"]):
    axis.text(position, 0, f"n={observations}", ha="center", va="bottom", rotation=90, fontsize=7)
plt.title("Neighborhood median SalePrice (sample size shown)")
plt.xlabel("Neighborhood")
plt.ylabel("Median SalePrice")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

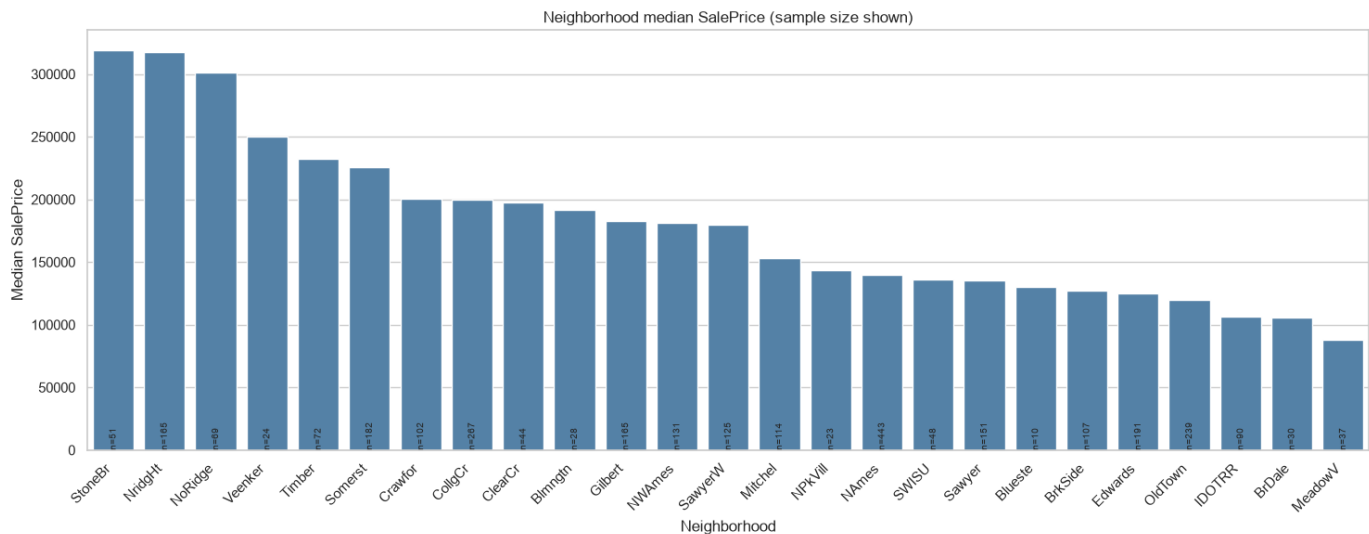
```

Output 1

Count and ordinal regressors versus SalePrice



Output 2



4. Visualize the target distribution

`SalePrice` is shown in levels and log scale. This visualization does not choose the final target transformation.

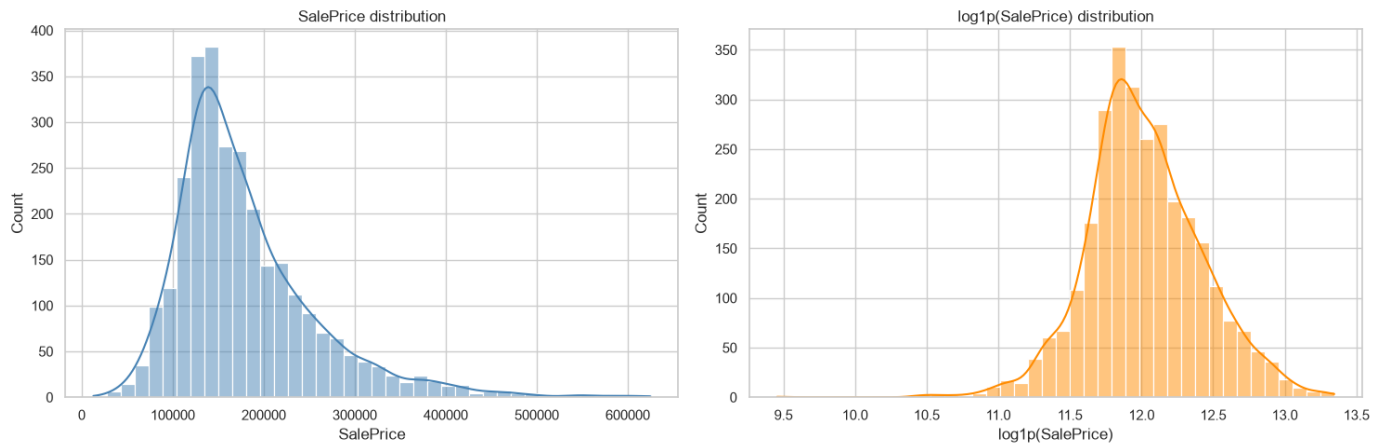
Cell 12 | In [26]:

```
display(y.describe().to_frame(TARGET))
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
sns.histplot(y, bins=40, kde=True, color="steelblue", ax=axes[0])
axes[0].set(title="SalePrice distribution", xlabel="SalePrice")
sns.histplot(np.log1p(y), bins=40, kde=True, color="darkorange", ax=axes[1])
axes[1].set(title="log1p(SalePrice) distribution", xlabel="log1p(SalePrice)")
plt.tight_layout()
plt.show()
```

Output 1

	SalePrice
count	2908.000000
mean	180272.213549
std	78139.090808
min	12789.000000
25%	129362.500000
50%	160000.000000
75%	213310.000000
max	625000.000000

Output 2



5. Visualize correlations among continuous regressors

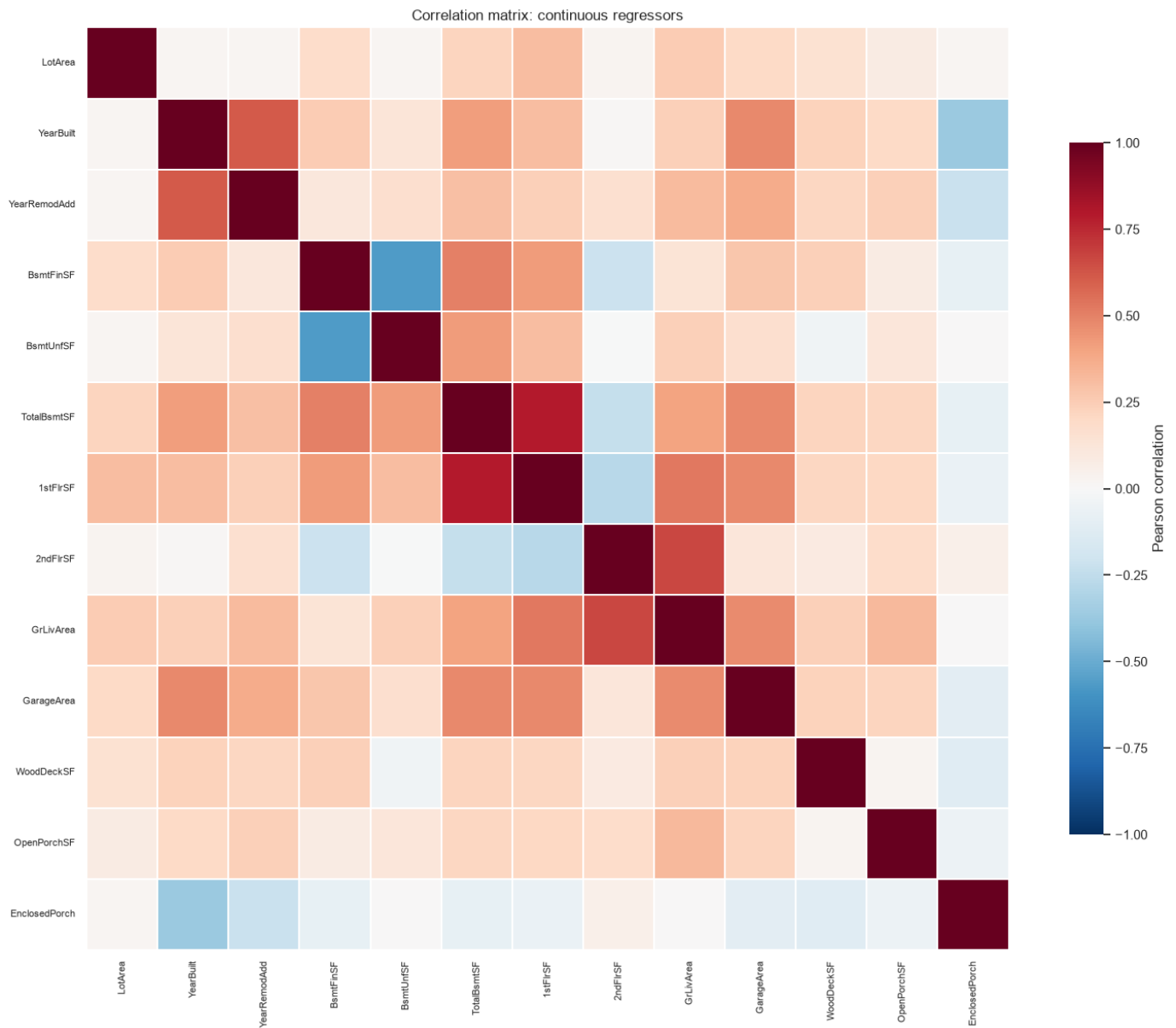
The matrix uses only continuous regressors and excludes `SalePrice`, ordinal quality variables, count variables, and neighborhood dummies. Positive correlations are red, negative correlations are blue, and values close to zero are white.

Cell 14 | In [27]:

```
continuous_correlation = X_47[continuous_features].corr(method="pearson")
plt.figure(figsize=(15, 12))
sns.heatmap(continuous_correlation, cmap="RdBu_r", center=0, vmin=-1, vmax=1,
            square=True, linewidths=0.35, linecolor="white",
            cbar_kws={"shrink": 0.75, "label": "Pearson correlation"},
            xticklabels=True, yticklabels=True)
plt.title("Correlation matrix: continuous regressors")
plt.xticks(rotation=90, fontsize=8)
plt.yticks(rotation=0, fontsize=8)
plt.tight_layout()
plt.show()

high_correlation_pairs = (continuous_correlation.where(np.triu(np.ones(continuous_correlation.shape),
k=1).astype(bool))
                          .stack().rename("correlation").to_frame()
                          .assign(abs_correlation=lambda t: t["correlation"].abs())
                          .sort_values("abs_correlation", ascending=False))
display(high_correlation_pairs.head(15))
```

Output 1



Output 2

		correlation	abs_correlation
TotalBsmtSF	1stFlrSF	0.789484	0.789484
2ndFlrSF	GrLivArea	0.664412	0.664412
YearBuilt	YearRemodAdd	0.611591	0.611591
BsmtFinSF	BsmtUnfSF	-0.565107	0.565107
1stFlrSF	GrLivArea	0.527698	0.527698
BsmtFinSF	TotalBsmtSF	0.503269	0.503269
1stFlrSF	GarageArea	0.482362	0.482362
YearBuilt	GarageArea	0.478887	0.478887
TotalBsmtSF	GarageArea	0.477684	0.477684
GrLivArea	GarageArea	0.475827	0.475827
BsmtUnfSF	TotalBsmtSF	0.428522	0.428522

		correlation	abs_correlation
BsmtFinSF	1stFlrSF	0.427463	0.427463
YearBuilt	TotalBsmtSF	0.415522	0.415522
TotalBsmtSF	GrLivArea	0.401733	0.401733
YearRemodAdd	GarageArea	0.374556	0.374556

Supplementary Appendix 2

02_modeling_new(1).ipynb

Complete code and saved outputs from the uploaded notebook. Execution counts are preserved; cells have not been rerun.

02 — House-price modeling with a log-price target

This is the project's single modeling notebook. All candidate models are fitted to ``log1p(SalePrice)``, selected using validation metrics on the log-price scale, and evaluated once on the held-out test set after selection.

Modeling protocol

- Fixed shuffled split: 1,800 training, 600 validation, and 508 test observations.
- The 47-feature contract is identical to the EDA notebook.
- Numeric predictors and the ordinal basement-quality variable are standardized within each fit. Neighborhood indicators remain 0/1.
- ``log1p(SalePrice)`` is standardized internally during fitting, then returned to the log-price scale before all reported metrics are calculated. Thus MSE, RMSE, and MAE are not arbitrary standardized-unit scores.
- Candidate models are selected exclusively by validation MSE. Test metrics are calculated only for the selected model, never used for selection.

Cell 3 | In [1]:

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.linear_model import Lasso, LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, StandardScaler

pd.set_option('display.max_columns', 100)
sns.set_theme(style='whitegrid', context='notebook')

DATA_FILENAME = 'IA_House_Price_Original_Data.xlsx'
DATA_CANDIDATES = [Path('../data') / DATA_FILENAME, Path('data') / DATA_FILENAME]
DATA_PATH = next((path for path in DATA_CANDIDATES if path.exists()), None)
if DATA_PATH is None:
    raise FileNotFoundError(f'Could not find {DATA_FILENAME} in the repository data directory.')
TARGET = 'SalePrice'
RANDOM_SEED = 20260928

numeric_features = [
    'LotArea', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd', 'BsmtFinSF',
    'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF', '2ndFlrSF', 'GrLivArea', 'FullBath',
    'HalfBath', 'BedroomAbvGr', 'TotRmsAbvGrd', 'Fireplaces', 'GarageCars', 'GarageArea',
    'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch',
]
neighbourhoods = [
    'Blmngtn', 'Blueste', 'BrDale', 'BrkSide', 'ClearCr', 'CollgCr', 'Crawfor', 'Edwards',
    'Gilbert', 'IDOTRR', 'MeadowV', 'Mitchel', 'Names', 'NPKVill', 'NWAmes', 'NoRidge',
    'NridgHt', 'OldTown', 'SWISU', 'Sawyer', 'SawyerW', 'Somerst', 'StoneBr', 'Timber', 'Veenker',
]
bsmtqual_map = {'Ex': 5, 'Gd': 4, 'TA': 3, 'Fa': 2, 'Po': 1, 'NA': 0}
```

Output 1

```
/Users/kerolin/anaconda3/lib/python3.11/site-packages/pandas/core/arrays/masked.py:61: UserWarning: Pandas
requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
  from pandas.core import (
```

Cell 4 | In [2]:

```
raw = pd.read_excel(DATA_PATH, skiprows=3)
raw.columns = raw.columns.astype(str).str.strip()
raw = raw.loc[:, ~raw.columns.str.startswith('Unnamed')].copy()

clean = raw.copy()
clean['BsmtFinSF'] = clean['BsmtFinSF1'] + clean['BsmtFinSF2']
clean['BsmtQual_ordinal'] = clean['BsmtQual'].fillna('NA').map(bsmtqual_map)

if set(clean['Neighborhood'].dropna()) != set(neighbourhoods):
    raise ValueError('Neighborhood categories do not match the 25-feature specification.')
neighbourhood_columns = [f'NBHD_{name}' for name in neighbourhoods]
dummies = pd.get_dummies(clean['Neighborhood'], prefix='NBHD', dtype=int).reindex(
    columns=neighbourhood_columns, fill_value=0
)
X_47 = pd.concat([clean[numeric_features], clean[['BsmtQual_ordinal']], dummies], axis=1)
y_price = clean[TARGET].copy()

assert X_47.shape[1] == 47, f'Expected 47 features, found {X_47.shape[1]}.'
assert not X_47.isna().any().any(), 'Selected regressors contain missing values.'
assert not y_price.isna().any(), 'SalePrice contains missing values.'

display(pd.Series({
    'observations': len(X_47), 'features': X_47.shape[1],
    'missing_regressor_values': int(X_47.isna().sum().sum()),
    'missing_target_values': int(y_price.isna().sum()),
}).to_frame('value'))
```

Output 1

	value
observations	2908
features	47
missing_regressor_values	0
missing_target_values	0

Complete list of 47 features

The intercept is separate and is not counted as a feature.

Cell 6 | In [3]:

```
feature_list = pd.DataFrame({
    'No.': np.arange(1, 48), 'Feature': X_47.columns,
    'Type': ['Numerical'] * 21 + ['Ordinal'] + ['Neighborhood indicator'] * 25,
})
with pd.option_context('display.max_rows', None):
    display(feature_list.set_index('No.'))
```

Output 1

	Feature	Type
No.		
1	LotArea	Numerical
2	OverallQual	Numerical
3	OverallCond	Numerical
4	YearBuilt	Numerical
5	YearRemodAdd	Numerical
6	BsmtFinSF	Numerical
7	BsmtUnfSF	Numerical
8	TotalBsmtSF	Numerical
9	1stFlrSF	Numerical

	Feature	Type
10	2ndFlrSF	Numerical
11	GrLivArea	Numerical
12	FullBath	Numerical
13	HalfBath	Numerical
14	BedroomAbvGr	Numerical
15	TotRmsAbvGrd	Numerical
16	Fireplaces	Numerical
17	GarageCars	Numerical
18	GarageArea	Numerical
19	WoodDeckSF	Numerical
20	OpenPorchSF	Numerical
21	EnclosedPorch	Numerical
22	BsmtQual_ordinal	Ordinal
23	NBHD_Blmngtn	Neighborhood indicator
24	NBHD_Blueste	Neighborhood indicator
25	NBHD_BrDale	Neighborhood indicator
26	NBHD_BrkSide	Neighborhood indicator
27	NBHD_ClearCr	Neighborhood indicator
28	NBHD_CollgCr	Neighborhood indicator
29	NBHD_Crawfor	Neighborhood indicator
30	NBHD_Edwards	Neighborhood indicator
31	NBHD_Gilbert	Neighborhood indicator
32	NBHD_IDOTRR	Neighborhood indicator
33	NBHD_MeadowV	Neighborhood indicator
34	NBHD_Mitchel	Neighborhood indicator
35	NBHD_NAmes	Neighborhood indicator
36	NBHD_NPkVill	Neighborhood indicator
37	NBHD_NWAmes	Neighborhood indicator
38	NBHD_NoRidge	Neighborhood indicator
39	NBHD_NridgHt	Neighborhood indicator
40	NBHD_OldTown	Neighborhood indicator
41	NBHD_SWISU	Neighborhood indicator
42	NBHD_Sawyer	Neighborhood indicator
43	NBHD_SawyerW	Neighborhood indicator
44	NBHD_Somerst	Neighborhood indicator
45	NBHD_StoneBr	Neighborhood indicator
46	NBHD_Timber	Neighborhood indicator
47	NBHD_Veenker	Neighborhood indicator

Cell 7 | In [4]:

```

rng = np.random.default_rng(RANDOM_SEED)
permutation = rng.permutation(len(X_47))
train_idx, validation_idx, test_idx = permutation[:1800], permutation[1800:2400], permutation[2400:]

```

```
X_train, y_train = X_47.iloc[train_idx], y_price.iloc[train_idx]
X_validation, y_validation = X_47.iloc[validation_idx], y_price.iloc[validation_idx]
X_test, y_test = X_47.iloc[test_idx], y_price.iloc[test_idx]
assert (len(X_train), len(X_validation), len(X_test)) == (1800, 600, 508)

display(pd.DataFrame({
    'observations': [len(X_train), len(X_validation), len(X_test)],
    'mean_saleprice': [y_train.mean(), y_validation.mean(), y_test.mean()],
}, index=['train', 'validation', 'test']).round(2))
```

Output 1

	observations	mean_saleprice
train	1800	180018.88
validation	600	178133.52
test	508	183695.88

Models and metrics

OLS, Ridge, and Lasso are fitted with the assignment's specified penalty strengths. The primary metrics below evaluate $\log_{10}(\text{SalePrice})$: MSE, RMSE, MAE, and R^2 . The raw-dollar target is never used as the fitting baseline.

Cell 9 | In [5]:

```
scaled_features = numeric_features + ['BsmtQual_ordinal']

def make_regressor_pipeline(estimator):
    preprocessing = ColumnTransformer(
        [
            ('scaled', StandardScaler(), scaled_features),
            ('neighborhood', 'passthrough', neighbourhood_columns)],
        verbose_feature_names_out=False,
    )
    return Pipeline([('preprocess', preprocessing), ('model', estimator)])

def make_log_target_model(estimator):
    # Pipeline inversion is StandardScaler inverse followed by expm1.
    target_transformer = Pipeline([
        ('log', FunctionTransformer(np.log1p, inverse_func=np.expm1, validate=False)),
        ('scale', StandardScaler()),
    ])
    return TransformedTargetRegressor(
        regressor=make_regressor_pipeline(estimator),
        transformer=target_transformer,
    )

def log_scale_metrics(actual_price, predicted_price):
    actual_log = np.log1p(actual_price)
    predicted_log = np.log1p(np.maximum(predicted_price, 0))
    return {
        'mse': mean_squared_error(actual_log, predicted_log),
        'rmse': np.sqrt(mean_squared_error(actual_log, predicted_log)),
        'mae': mean_absolute_error(actual_log, predicted_log),
        'r2': r2_score(actual_log, predicted_log),
    }

candidates = {
    'OLS': LinearRegression(),
    **{f'Ridge (alpha={alpha:.2f})': Ridge(alpha=alpha) for alpha in [0.10, 0.30, 0.60]},
    **{f'Lasso (alpha={alpha:.2f})': Lasso(alpha=alpha, max_iter=50_000) for alpha in [0.02, 0.06, 0.10]},
}

fitted_candidates, records = {}, []
for name, estimator in candidates.items():
    model = make_log_target_model(estimator)

    model.fit(X_train, y_train)
    fitted_candidates[name] = model
    train_metrics = log_scale_metrics(y_train, model.predict(X_train))
    validation_metrics = log_scale_metrics(y_validation, model.predict(X_validation))
    records.append({
        'model': name,
```

```

    **{f'train_{metric}': value for metric, value in train_metrics.items()},
    **{f'validation_{metric}': value for metric, value in validation_metrics.items()},
})

```

```

model_metrics = pd.DataFrame(records).sort_values('validation_mse').reset_index(drop=True)
best_validation_mse = model_metrics.loc[0, 'validation_mse']
model_metrics['validation_mse_delta_to_best'] = model_metrics['validation_mse'] - best_validation_mse
model_metrics['validation_mse_pct_above_best'] = 100 * model_metrics['validation_mse_delta_to_best'] /
best_validation_mse
display(model_metrics.round(8))

```

Output 1

	model	train_mse	train_rmse	train_mae	train_r2	validation_mse	validation_rmse	validation_mae	validation_r2	validation_mse_delta_to_best	validation_mse_pct_above_best
0	OLS	0.013908	0.117930	0.080995	0.912059	0.014917	0.122137	0.084055	0.914905	0.000000	0.000000
1	Ridge (alpha=0.60)	0.013904	0.117914	0.081020	0.912083	0.014929	0.122186	0.083999	0.914837	0.000012	0.080107
2	Ridge (alpha=0.30)	0.013902	0.117908	0.081008	0.912092	0.014933	0.122201	0.084024	0.914816	0.000016	0.104815
3	Ridge (alpha=0.10)	0.013902	0.117906	0.081004	0.912094	0.014936	0.122213	0.084046	0.914798	0.000019	0.125443
4	Lasso (alpha=0.02)	0.016669	0.129110	0.091020	0.894595	0.017505	0.132305	0.092142	0.900147	0.002587	17.343160
5	Lasso (alpha=0.06)	0.019338	0.139063	0.097379	0.877718	0.021760	0.147513	0.103026	0.875871	0.006843	45.870798
6	Lasso (alpha=0.10)	0.022969	0.151554	0.107108	0.854762	0.026563	0.162982	0.114689	0.848473	0.011646	78.068131

Complete parameters of all seven candidates

The table contains the intercept and all 47 slopes for each 1,800-observation training fit. Predictor and target standardization are reversed: coefficients use original input units and the log1p(SalePrice) target scale. These units differ from the standardized-predictor coefficient plots below.

OLS coefficients represent one numerical solution, not uniquely identified: neighborhood indicators sum to one, and finished plus unfinished basement area equals total basement area.

Cell 11 | In [6]:

```

def original_input_log_parameters(fitted_model, check_X):
    """Undo predictor and target scaling; retain the log1p price target."""
    reg = fitted_model.regressor_
    pre = reg.named_steps['preprocess']
    est = reg.named_steps['model']
    names = list(pre.get_feature_names_out())
    sy = fitted_model.transformer_.named_steps['scale']
    sx = pre.named_transformers_['scaled']
    mean_x = pd.Series(0.0, index=names)
    scale_x = pd.Series(1.0, index=names)
    mean_x.loc[scaled_features] = sx.mean_
    scale_x.loc[scaled_features] = sx.scale_
    slopes = pd.Series(est.coef_ * sy.scale_[0], index=names) / scale_x
    intercept = float(sy.mean_[0] + sy.scale_[0] * est.intercept_ - slopes.dot(mean_x))
    reconstructed = intercept + check_X[names].to_numpy(dtype=float) @ slopes.to_numpy()
    expected = sy.mean_[0] + sy.scale_[0] * reg.predict(check_X)
    # Collinear OLS fits can contain large cancelling coefficients. Account
    # for the magnitude of intermediate sums, not just the final prediction.
    raw_X = check_X[names].to_numpy(dtype=float)
    transformed_X = pre.transform(check_X)
    raw_magnitude = abs(intercept) + np.abs(raw_X) @ np.abs(slopes.to_numpy())
    fitted_magnitude = abs(sy.mean_[0]) + abs(sy.scale_[0]) * (
        abs(est.intercept_) + np.abs(transformed_X) @ np.abs(est.coef_)
    )
    conversion_magnitude = abs(sy.scale_[0] * est.intercept_) + float(
        np.abs(slopes.to_numpy()) @ np.abs(mean_x.to_numpy())
    )
    roundoff_bound = 64 * np.finfo(float).eps * (

```

```

        raw_magnitude + fitted_magnitude + conversion_magnitude
    )
    allowed_error = 1e-8 + 1e-8 * np.abs(expected) + roundoff_bound
    error = np.abs(reconstructed - expected)
    if not (np.isfinite(reconstructed).all() and np.isfinite(expected).all()):
        raise ValueError('Non-finite values found during parameter reconstruction.')
    if np.any(error > allowed_error):
        raise AssertionError(
            f'{type(est).__name__}: parameter reconstruction exceeds the numerical '
            f'error allowance (max error={error.max():.3g}).'
        )
    if error.max() > 1e-6:
        print(f'Note: {type(est).__name__} has cancelling coefficients; '
              f'parameter reconstruction differs by up to {error.max():.3g} '
              f'log-price units. Use the fitted pipeline for predictions and '
              f'do not interpret individual OLS coefficients as stable effects.')
    return pd.concat([pd.Series({'Intercept': intercept}), slopes])

candidate_parameter_table = pd.DataFrame({
    name: original_input_log_parameters(fitted_candidates[name], X_train)
    for name in candidates
})
candidate_parameter_table.index.name = 'Parameter'
assert candidate_parameter_table.shape == (48, 7)
print('Candidate models: 1,800 training observations.')
print('predicted loglp(SalePrice) = Intercept + sum(coefficient * original feature)')
print('Numerical inputs use original units; BsmtQual uses 0-5; neighborhood indicators use 0/1.')
with pd.option_context('display.max_rows', None, 'display.max_columns', None, 'display.float_format', lambda
v: f'{v:.10g}'):
    display(candidate_parameter_table)

```

Output 1

Note: LinearRegression has cancelling coefficients; parameter reconstruction differs by up to 0.000114 log-price units. Use the fitted pipeline for predictions and do not interpret individual OLS coefficients as stable effects.

Candidate models: 1,800 training observations.

predicted loglp(SalePrice) = Intercept + sum(coefficient * original feature)

Numerical inputs use original units; BsmtQual uses 0-5; neighborhood indicators use 0/1.

Output 2

	OLS	Ridge (alpha=0.10)	Ridge (alpha=0.30)	Ridge (alpha=0.60)	Lasso (alpha=0.02)	Lasso (alpha=0.06)	Lasso (alpha=0.10)
Parameter							
Intercept	1187908551	3.413369764	3.402849611	3.387791696	3.425376715	5.010756053	6.380184107
LotArea	2.499664799e-06	2.504491008e-06	2.513885041e-06	2.527367052e-06	2.429910922e-06	1.083995779e-06	0
OverallQual	0.07251729937	0.0719481961	0.0720068499	0.07209240511	0.08687349372	0.09434269313	0.09864124711
OverallCond	0.05444095787	0.05485265197	0.05479477488	0.05471067465	0.03767502705	0.01057937888	0
YearBuilt	0.002798266905	0.00282078857	0.0028214603	0.002822321775	0.002415212556	0.001542193479	0.00109822654
YearRemodAdd	0.0007950229212	0.0007713852241	0.0007757630463	0.0007821215141	0.001182593928	0.001346622744	0.001148465073
BsmtFinSF	-214459505.8	8.163171179e-05	8.161915808e-05	8.160088933e-05	8.54963232e-05	7.558002712e-05	6.024608072e-05
BsmtUnfSF	-214459505.8	-6.847938812e-06	-6.83044782e-06	-6.805328154e-06	-0	-0	-0
TotalBsmtSF	214459505.8	8.895395603e-05	8.895821076e-05	8.896403798e-05	8.288758061e-05	8.010954019e-05	8.042773257e-05
1stFlrSF	6.02427597e-06	5.609285753e-06	8.713607027e-06	1.308095086e-05	2.133760904e-05	1.593536002e-05	1.148059237e-05
2ndFlrSF	5.730755178e-06	7.665520274e-06	1.047948581e-05	1.443269226e-05	0	0	0
GrLivArea	0.0002541716176	0.0002562355955	0.0002533875669	0.0002493846789	0.00026107795	0.000242040647	0.0002282164818

	OLS	Ridge (alpha=0.10)	Ridge (alpha=0.30)	Ridge (alpha=0.60)	Lasso (alpha=0.02)	Lasso (alpha=0.06)	Lasso (alpha=0.10)
FullBath	0.01026170346	0.009668089988	0.009692918661	0.009727450968	0	0	0
HalfBath	0.001537344001	0.0002060399353	0.0002093490089	0.0002117519646	0	0	0
BedroomAbvGr	-0.01125341519	-0.01211623088	-0.01212747203	-0.01214329094	-0	0	0
TotRmsAbvGrd	0.005988984868	0.005997945071	0.006021249563	0.006054459835	0	0	0
Fireplaces	0.02979152678	0.03017308734	0.03017197892	0.03017374534	0.03526519877	0.02676957771	0.01624062551
GarageCars	0.02914080933	0.029355013	0.02941945092	0.02951473797	0.03397085907	0.03467385467	0.03698134031
GarageArea	9.505663708e-05	9.30456936e-05	9.301007515e-05	9.295898891e-05	8.110469606e-05	6.60988196e-05	4.75677203e-05
WoodDeckSF	6.715816619e-05	6.678936574e-05	6.671566983e-05	6.660977179e-05	8.134712872e-06	0	0
OpenPorchSF	0.0001301246952	0.0001302668314	0.000130310127	0.0001303692116	2.020263437e-05	0	0
EnclosedPorch	0.0001737713602	0.0001774553429	0.0001777856047	0.0001782527052	0	-0	-0
BsmtQual_ordinal	0.01729223689	0.01762166712	0.01764785822	0.01768710583	0.01737644282	0.01284040109	0.008896869931
NBHD_Blmngtn	-1187908547	-0.0296819531	-0.02985365071	-0.03006754446	-0	-0	-0
NBHD_Blueste	-1187908547	-0.1373937577	-0.1330344964	-0.1269900042	-0	-0	-0
NBHD_BrDale	-1187908547	-0.1509761724	-0.1494326594	-0.1471701666	-0	-0	-0
NBHD_BrkSide	-1187908547	0.01960028661	0.01959385394	0.01958531212	0	-0	-0
NBHD_ClearCr	-1187908547	0.04306253344	0.04221636731	0.04100773497	0	0	0
NBHD_CollgCr	-1187908547	0.0372510719	0.03677684398	0.03609967537	0	0	0
NBHD_Crawfor	-1187908547	0.1524468687	0.1517823356	0.1508088571	0	0	0
NBHD_Edwards	-1187908547	-0.01075426839	-0.01077384809	-0.01079408675	-0	-0	-0
NBHD_Gilbert	-1187908547	0.01975452027	0.01931323204	0.01868431611	-0	0	0
NBHD_IDOTRR	-1187908547	-0.06420981199	-0.06380343343	-0.06320689423	-0	-0	-0
NBHD_MeadowV	-1187908547	-0.1604651574	-0.158607967	-0.1558964878	-0	-0	-0
NBHD_Mitchel	-1187908547	-0.005767893241	-0.00602166411	-0.006377058615	-0	-0	-0
NBHD_NAmes	-1187908547	0.02281039863	0.02270959924	0.02257067463	0	0	0
NBHD_NPkVill	-1187908547	-0.06731860636	-0.06647565846	-0.06524373113	-0	-0	-0
NBHD_NWAmes	-1187908547	-0.0290721648	-0.02933234028	-0.02969386938	-0	-0	-0
NBHD_NoRidge	-1187908547	0.06061189456	0.05970824493	0.0584107238	0	0	0
NBHD_NridgHt	-1187908547	0.0894697043	0.08851656665	0.08714371755	0	0	0
NBHD_OldTown	-1187908547	-0.05622126538	-0.05608250567	-0.05587664265	-0	-0	-0
NBHD_SWISU	-1187908547	-0.01031399809	-0.01002453278	-0.009611095459	-0	-0	-0
NBHD_Sawyer	-1187908547	0.02292908416	0.0227987695	0.02261610481	0	0	0
NBHD_SawyerW	-1187908547	-0.00632241119	-0.006675165304	-0.0071709536	-0	-0	-0
NBHD_Somerst	-1187908547	0.05782825293	0.05722222117	0.05635293632	0	0	0
NBHD_StoneBr	-1187908547	0.1277000501	0.1261063415	0.1238040595	0	0	0
NBHD_Timber	-1187908547	0.04709146517	0.04627263867	0.04509979403	0	0	0

	OLS	Ridge (alpha=0.10)	Ridge (alpha=0.30)	Ridge (alpha=0.60)	Lasso (alpha=0.02)	Lasso (alpha=0.06)	Lasso (alpha=0.10)
NBHD_Veenker	-1187908547	0.02794132928	0.02710090711	0.02591462862	0	0	0

Final test evaluation

Only the candidate with the lowest validation MSE is refitted on training plus validation data. This prevents test-set metrics from influencing model selection.

Cell 13 | In [7]:

```
best_model_name = model_metrics.loc[0, 'model']
best_estimator = candidates[best_model_name]
X_development = pd.concat([X_train, X_validation])
y_development = pd.concat([y_train, y_validation])

final_model = make_log_target_model(best_estimator)
final_model.fit(X_development, y_development)
test_prediction = final_model.predict(X_test)
test_metrics = log_scale_metrics(y_test, test_prediction)

final_test_metrics = pd.DataFrame([
    'selected_model': best_model_name,
    'target_for_metrics': 'log1p(SalePrice)',
    'test_mse': test_metrics['mse'],
    'test_rmse': test_metrics['rmse'],
    'test_mae': test_metrics['mae'],
    'test_r2': test_metrics['r2'],
])
display(final_test_metrics.round(4))

dollar_metrics = pd.DataFrame([
    'test_rmse_usd': np.sqrt(mean_squared_error(y_test, test_prediction)),
    'test_mae_usd': mean_absolute_error(y_test, test_prediction),
])
display(dollar_metrics.round(2))

plot_data = pd.DataFrame({'actual_saleprice': y_test, 'predicted_saleprice': test_prediction})
plt.figure(figsize=(6, 6))
ax = sns.scatterplot(data=plot_data, x='actual_saleprice', y='predicted_saleprice', alpha=0.65)
limits = [plot_data.min().min(), plot_data.max().max()]
ax.plot(limits, limits, '--', color='firebrick', label='Perfect prediction')
ax.set(title=f'Final test predictions: {best_model_name}', xlabel='Actual SalePrice', ylabel='Predicted SalePrice')
ax.legend()
plt.tight_layout()
plt.show()
```

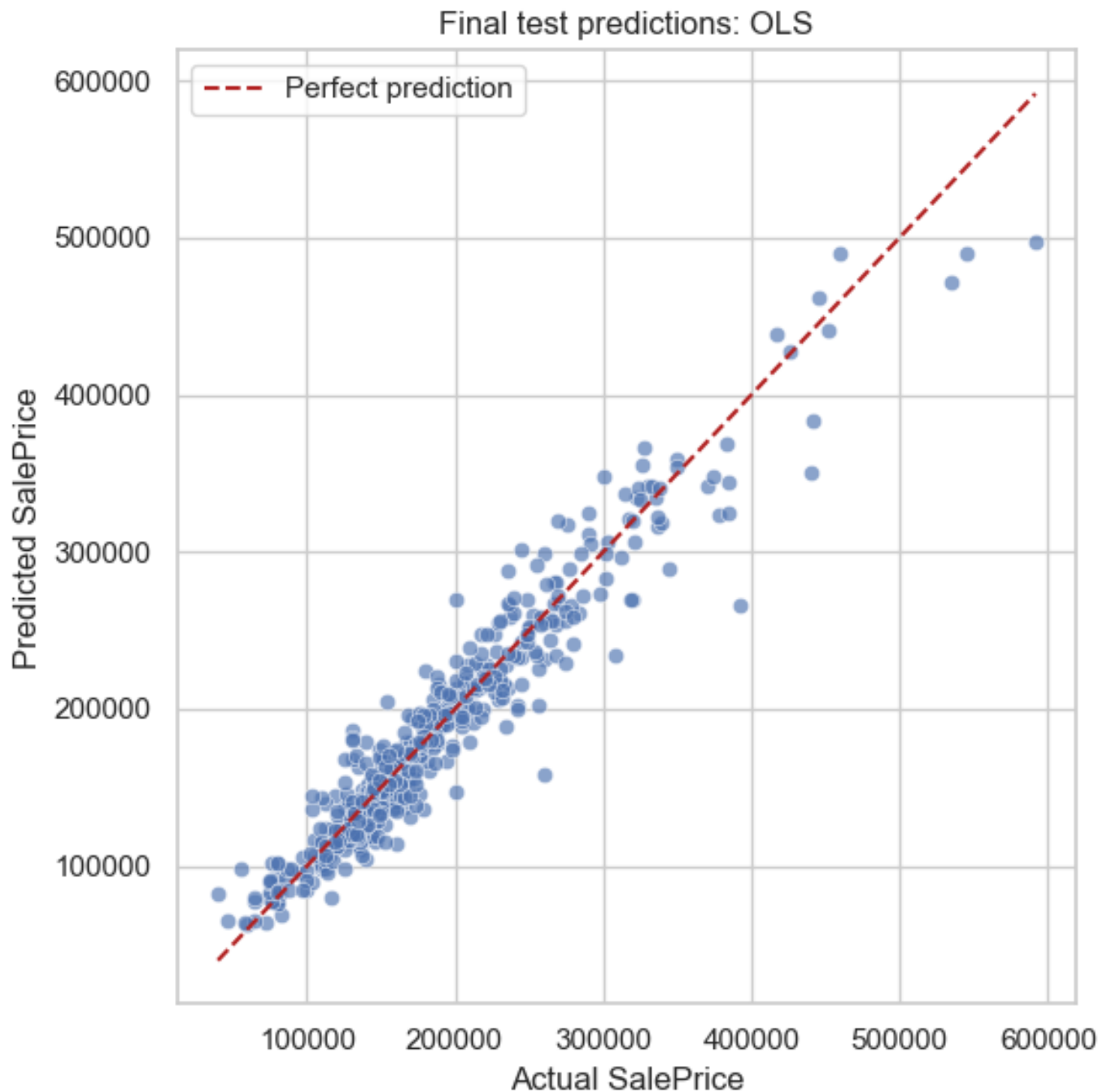
Output 1

	selected_model	target_for_metrics	test_mse	test_rmse	test_mae	test_r2
0	OLS	log1p(SalePrice)	0.0144	0.1201	0.0862	0.9087

Output 2

	test_rmse_usd	test_mae_usd
0	21545.45	14996.77

Output 3



Cell 14 | In [8]:

```
# Convert coefficients from standardized-target units back to log-price units.
target_log_std = final_model.transformer_.named_steps['scale'].scale_[0]
coefficient_table = pd.DataFrame({
    'feature': final_model.regressor_.named_steps['preprocess'].get_feature_names_out(),
    'log_price_coefficient': final_model.regressor_.named_steps['model'].coef_ * target_log_std,
})
coefficient_table['abs_coefficient'] = coefficient_table['log_price_coefficient'].abs()

# 1. Top 30 coefficients by absolute magnitude across all 47 features.
top_30_coefficients = coefficient_table.sort_values('abs_coefficient', ascending=False).head(30)

# 2. Categorical features: one ordinal basement-quality feature plus 25 neighborhood indicators.
categorical_features = ['BsmtQual_ordinal'] + neighbourhood_columns
categorical_coefficients = coefficient_table.set_index('feature').loc[categorical_features].reset_index()
categorical_coefficients['display_feature'] = categorical_coefficients['feature'].replace({'BsmtQual_ordinal':
```

```

'BsmtQual (ordinal)'))
categorical_coefficients['display_feature'] =
categorical_coefficients['display_feature'].str.removeprefix('NBHD_')
categorical_coefficients = categorical_coefficients.sort_values('log_price_coefficient', ascending=False)

# 3. Numerical features: all 21 numeric features in the assignment contract.
numerical_coefficients = coefficient_table.set_index('feature').loc[numeric_features].reset_index()
numerical_coefficients = numerical_coefficients.sort_values('log_price_coefficient', ascending=False)

print(f'Best model selected by validation MSE: {best_model_name}')
print('Coefficients are in log-price units. Numerical feature coefficients correspond to standardized
predictors.')
display(top_30_coefficients[['feature', 'log_price_coefficient', 'abs_coefficient']].round(8))
display(categorical_coefficients[['display_feature', 'log_price_coefficient']].round(8))
display(numerical_coefficients[['feature', 'log_price_coefficient']].round(8))

# Matplotlib bars avoid seaborn-version differences in the legend argument.
def plot_coefficient_bars(table, label_column, title, figsize):
    values = table['log_price_coefficient'].to_numpy()
    labels = table[label_column].astype(str).tolist()
    color_limit = max(float(np.abs(values).max()), 1e-12)
    cmap = plt.get_cmap('coolwarm')
    colors = cmap((values / color_limit + 1.0) / 2.0)
    fig, axis = plt.subplots(figsize=figsize)
    positions = np.arange(len(table))

    axis.barh(positions, values, color=colors)
    axis.set_yticks(positions)
    axis.set_yticklabels(labels)
    axis.invert_yaxis()
    axis.axvline(0, color='gray', linewidth=0.8)
    axis.set(title=title, xlabel='Coefficient (log-price units)', ylabel='')
    plt.tight_layout()
    plt.show()

plot_coefficient_bars(
    top_30_coefficients.sort_values('log_price_coefficient'), 'feature',
    'Top 30 coefficients by absolute magnitude', (10, 11))
plot_coefficient_bars(
    categorical_coefficients, 'display_feature',
    'Categorical and ordinal feature coefficients', (10, 10))
plot_coefficient_bars(
    numerical_coefficients, 'feature',
    'Numerical feature coefficients', (10, 8))

```

Output 1

Best model selected by validation MSE: OLS
Coefficients are in log-price units. Numerical feature coefficients correspond to standardized predictors.

Output 2

	feature	log_price_coefficient	abs_coefficient
5	BsmtFinSF	-3.521116e+11	3.521116e+11
6	BsmtUnfSF	-3.382053e+11	3.382053e+11
7	TotalBsmtSF	3.239798e+11	3.239798e+11
32	NBHD_MeadowV	-1.209319e+10	1.209319e+10
24	NBHD_BrDale	-1.209319e+10	1.209319e+10
23	NBHD_Blueste	-1.209319e+10	1.209319e+10
35	NBHD_NPkvill	-1.209319e+10	1.209319e+10
31	NBHD_IDOTRR	-1.209319e+10	1.209319e+10
39	NBHD_OldTown	-1.209319e+10	1.209319e+10
36	NBHD_NWAmes	-1.209319e+10	1.209319e+10
46	NBHD_Veenker	-1.209319e+10	1.209319e+10
22	NBHD_Blmngtn	-1.209319e+10	1.209319e+10
42	NBHD_SawyerW	-1.209319e+10	1.209319e+10

	feature	log_price_coefficient	abs_coefficient
29	NBHD_Edwards	-1.209319e+10	1.209319e+10
33	NBHD_Mitchel	-1.209319e+10	1.209319e+10
40	NBHD_SWISU	-1.209319e+10	1.209319e+10
34	NBHD_NAmes	-1.209319e+10	1.209319e+10
41	NBHD_Sawyer	-1.209319e+10	1.209319e+10
27	NBHD_CollgCr	-1.209319e+10	1.209319e+10
30	NBHD_Gilbert	-1.209319e+10	1.209319e+10
25	NBHD_BrkSide	-1.209319e+10	1.209319e+10
45	NBHD_Timber	-1.209319e+10	1.209319e+10
37	NBHD_NoRidge	-1.209319e+10	1.209319e+10
26	NBHD_ClearCr	-1.209319e+10	1.209319e+10
43	NBHD_Somerst	-1.209319e+10	1.209319e+10
38	NBHD_NridgHt	-1.209319e+10	1.209319e+10
44	NBHD_StoneBr	-1.209319e+10	1.209319e+10
28	NBHD_Crawfor	-1.209319e+10	1.209319e+10
10	GrLivArea	1.110710e-01	1.110710e-01
1	OverallQual	9.898155e-02	9.898155e-02

Output 3

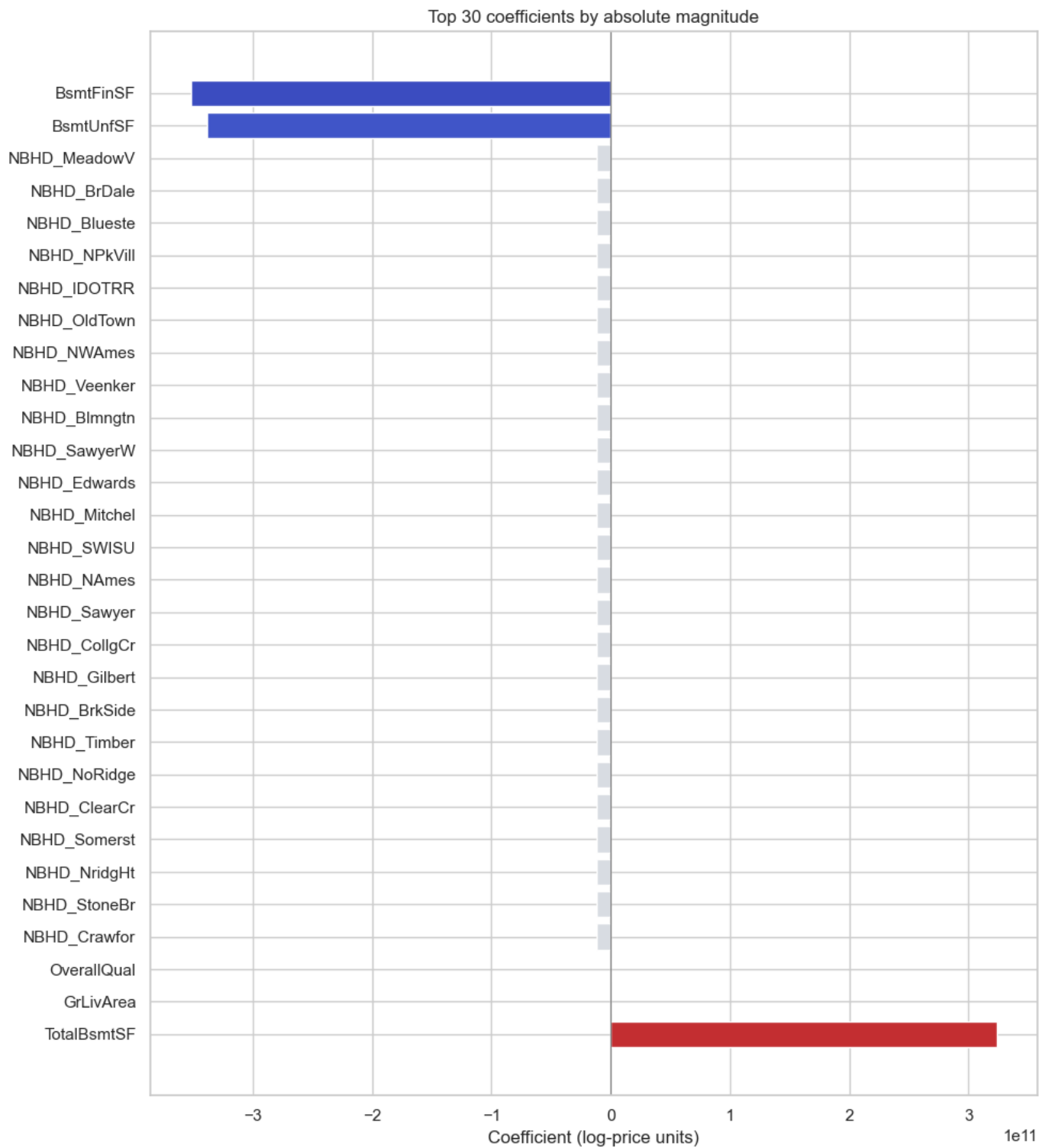
	display_feature	log_price_coefficient
0	BsmtQual (ordinal)	1.852880e-02
7	Crawfor	-1.209319e+10
23	StoneBr	-1.209319e+10
17	NridgHt	-1.209319e+10
22	Somerst	-1.209319e+10
5	ClearCr	-1.209319e+10
16	NoRidge	-1.209319e+10
24	Timber	-1.209319e+10
4	BrkSide	-1.209319e+10
9	Gilbert	-1.209319e+10
6	CollgCr	-1.209319e+10
20	Sawyer	-1.209319e+10
13	NAmes	-1.209319e+10
19	SWISU	-1.209319e+10
12	Mitchel	-1.209319e+10
8	Edwards	-1.209319e+10
21	SawyerW	-1.209319e+10
1	Blmngtn	-1.209319e+10
25	Veenker	-1.209319e+10
15	NWAmes	-1.209319e+10
18	OldTown	-1.209319e+10
10	IDOTRR	-1.209319e+10

	display_feature	log_price_coefficient
14	NPkVill	-1.209319e+10
2	Blueste	-1.209319e+10
3	BrDale	-1.209319e+10
11	MeadowV	-1.209319e+10

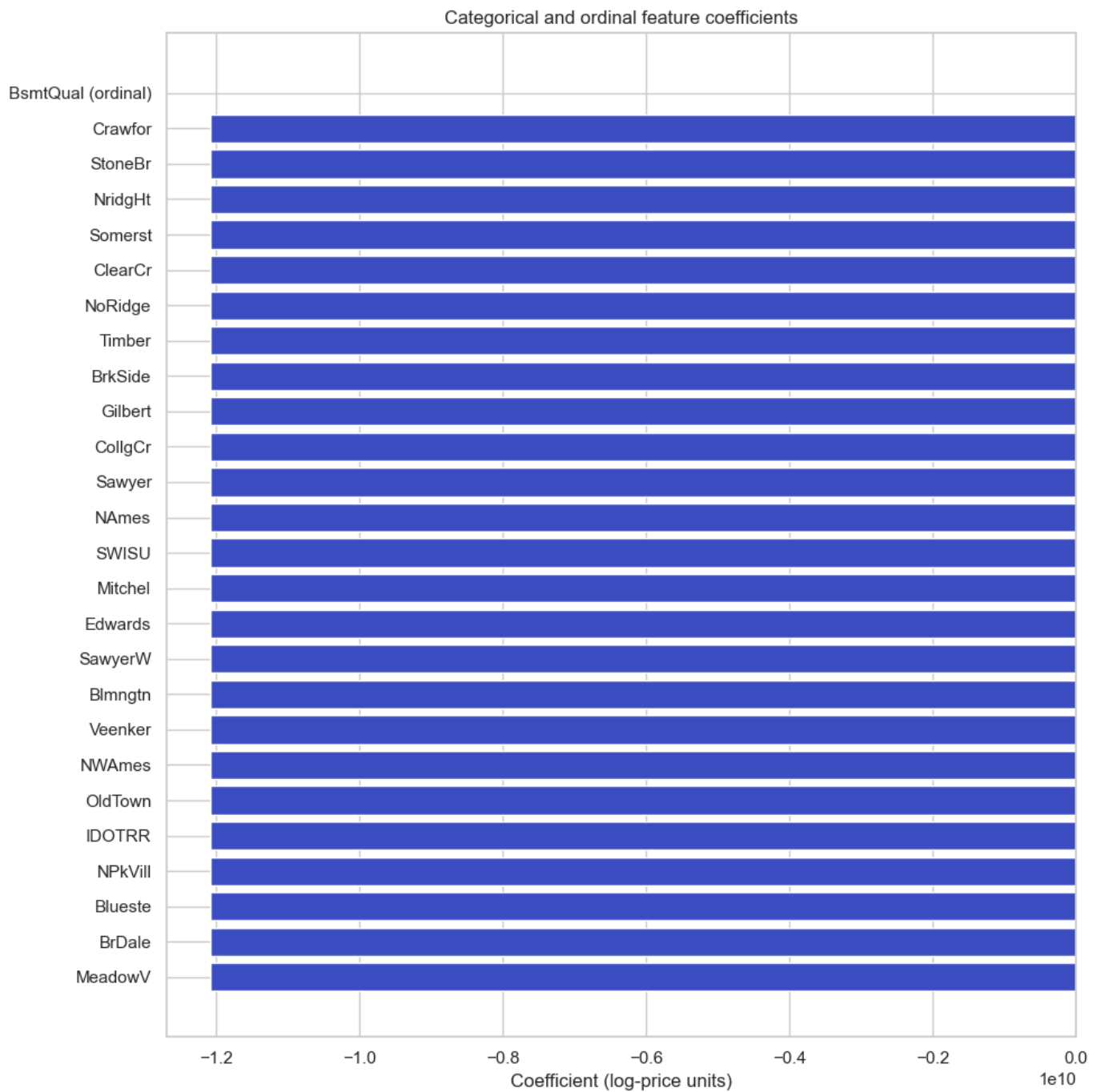
Output 4

	feature	log_price_coefficient
7	TotalBsmtSF	3.239798e+11
10	GrLivArea	1.110710e-01
1	OverallQual	9.898155e-02
3	YearBuilt	8.692729e-02
2	OverallCond	6.192426e-02
17	GarageArea	2.337701e-02
16	GarageCars	2.134930e-02
15	Fireplaces	2.000200e-02
0	LotArea	1.901989e-02
4	YearRemodAdd	1.813341e-02
8	1stFlrSF	1.621551e-02
9	2ndFlrSF	1.010066e-02
20	EnclosedPorch	9.479710e-03
18	WoodDeckSF	7.128390e-03
19	OpenPorchSF	6.454810e-03
14	TotRmsAbvGrd	5.609670e-03
11	FullBath	4.377830e-03
12	HalfBath	3.737050e-03
13	BedroomAbvGr	-6.267540e-03
6	BsmtUnfSF	-3.382053e+11
5	BsmtFinSF	-3.521116e+11

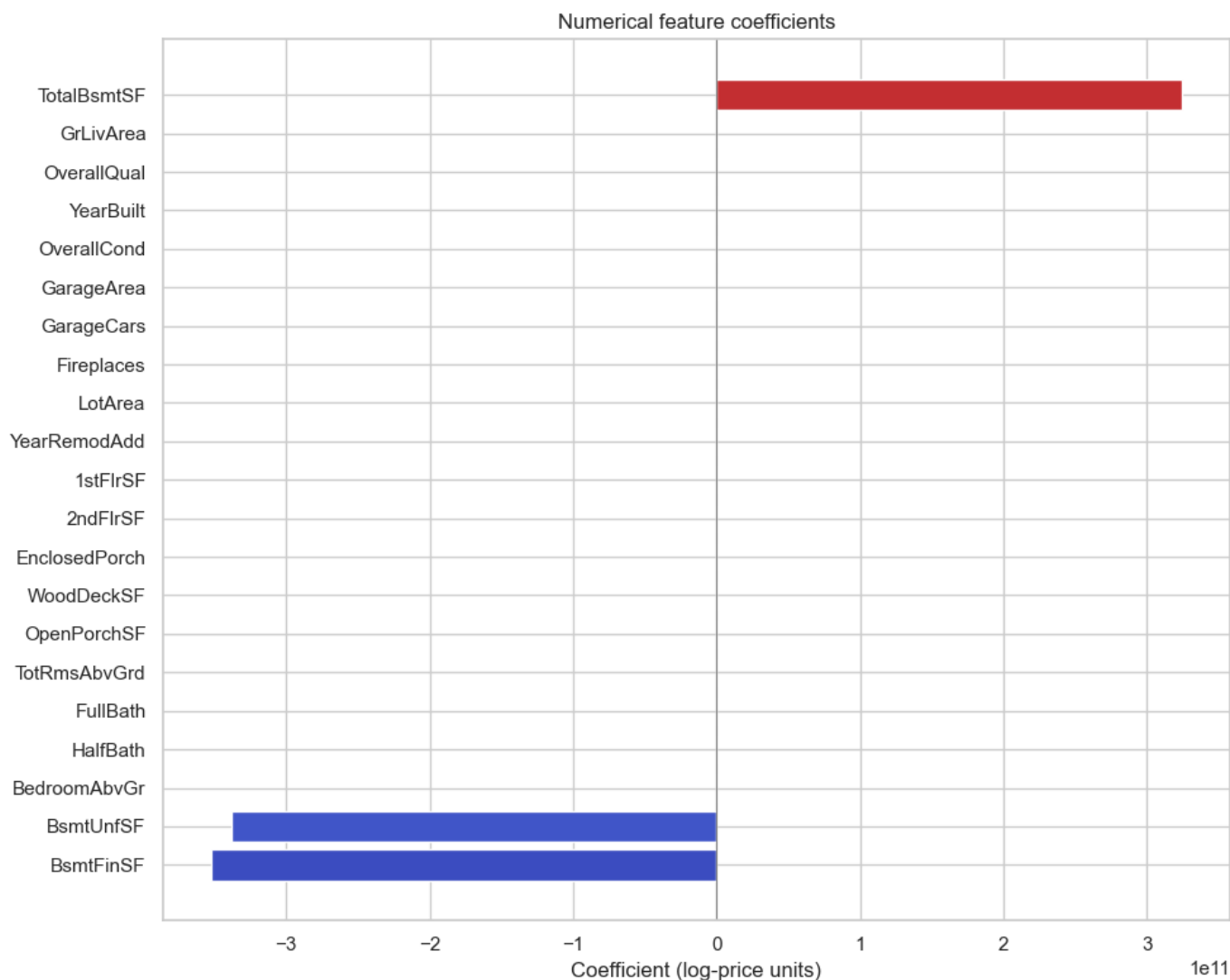
Output 5



Output 6



Output 7



Interpretation guardrails

The reported primary metrics are on the log-price scale, so they measure proportional pricing error more naturally than dollar-scale errors. Dollar RMSE and MAE are supplied only as an intuitive supplement for the final selected model. Coefficients are conditional prediction associations, not causal effects. Retaining all 25 neighborhood dummies honors the 47-feature contract; with an intercept, OLS dummy coefficients are not individually identified even though its predictions are well-defined.

Complete parameters of the final refitted model

This table uses the same units as the candidate table, but reflects the final fit on 2,400 observations.

Cell 17 | In [9]:

```
final_parameter_table = original_input_log_parameters(final_model, X_development).to_frame(
    name=f'Final {best_model_name}')
final_parameter_table.index.name = 'Parameter'
print('Final model: refitted on 2,400 training + validation observations.')
print('The equation and units match the candidate parameter table above.')
with pd.option_context('display.max_rows', None, 'display.float_format', lambda v: f'{v:.10g}'):
    display(final_parameter_table)
```

Output 1

Note: LinearRegression has cancelling coefficients; parameter reconstruction differs by up to 0.000384 log-price units. Use the fitted pipeline for predictions and do not interpret individual OLS coefficients as stable effects.

Final model: refitted on 2,400 training + validation observations.
The equation and units match the candidate parameter table above.

Output 2

	Final OLS
Parameter	
Intercept	1.209319275e+10
LotArea	2.39631659e-06
OverallQual	0.07085582747
OverallCond	0.05565092549
YearBuilt	0.002859252048
YearRemodAdd	0.0008639002408
BsmtFinSF	-767725470.6
BsmtUnfSF	-767725470.6
TotalBsmtSF	767725470.6
1stFlrSF	4.356442862e-05
2ndFlrSF	2.370438939e-05
GrLivArea	0.0002288115003
FullBath	0.007945166856
HalfBath	0.007461682507
BedroomAbvGr	-0.007710720865
TotRmsAbvGrd	0.0036594773
Fireplaces	0.03114575411
GarageCars	0.02818816371
GarageArea	0.000109468182
WoodDeckSF	5.780064197e-05
OpenPorchSF	9.744374914e-05
EnclosedPorch	0.0001536698883
BsmtQual_ordinal	0.02046510133
NBHD_Blmngtn	-1.209319274e+10
NBHD_Blueste	-1.209319274e+10
NBHD_BrDale	-1.209319274e+10
NBHD_BrkSide	-1.209319274e+10
NBHD_ClearCr	-1.209319274e+10
NBHD_CollgCr	-1.209319274e+10
NBHD_Crawfor	-1.209319274e+10
NBHD_Edwards	-1.209319274e+10
NBHD_Gilbert	-1.209319274e+10
NBHD_IDOTRR	-1.209319274e+10
NBHD_MeadowV	-1.209319274e+10
NBHD_Mitchel	-1.209319274e+10
NBHD_NAmes	-1.209319274e+10
NBHD_NPkVill	-1.209319274e+10
NBHD_NWAmes	-1.209319274e+10

	Final OLS
NBHD_NoRidge	-1.209319274e+10
NBHD_NridgHt	-1.209319274e+10
NBHD_OldTown	-1.209319274e+10
NBHD_SWISU	-1.209319274e+10
NBHD_Sawyer	-1.209319274e+10
NBHD_SawyerW	-1.209319274e+10
NBHD_Somerst	-1.209319274e+10
NBHD_StoneBr	-1.209319274e+10
NBHD_Timber	-1.209319274e+10
NBHD_Veenker	-1.209319274e+10

Optional Zillow case study

The project brief optionally asks for a current Iowa listing, a model prediction, and a comparison with the listing price and Zillow Zestimate. The selected property is 124 N Franklin Ave, Ames, IA 50014, observed on October 4, 2026.

- Active Zillow listing:

<https://www.zillow.com/homedetails/124-N-Franklin-Ave-Ames-IA-50014/459888892_zpid/>

- Zillow values used: list price \$246,500 and Zestimate \$242,000.

- The same parcel ('0905476120') sold for \$119,000 in April 2008 and \$186,820 in April 2022.

- The current listing reports 4 bedrooms, 2 bathrooms, 1,262 square feet, a partial basement with finished living space, a one-car detached garage, a covered deck, and year built 1900.

- The exact 2008 property appears in the course spreadsheet as `Id = 391`, with neighborhood `Edwards`. Under the fixed random split, this observation belongs to the held-out test set and was not used to fit the final model.

The historical row supplies the detailed Ames variables that current websites do not report. For the current-listing prediction, the mapping updates the above-grade area, bathroom count, and above-grade bedroom count to the current listing while retaining the historical structural fields. Public sources report slightly different square-footage and lot-area values, so the assumptions are shown explicitly. This is an illustrative model application, not an appraisal.

Cell 19 | In [10]:

```
# Current and historical values captured on October 4, 2026.
zillow_address = '124 N Franklin Ave, Ames, IA 50014'
zillow_list_price = 246_500.0
zillow_zestimate = 242_000.0
historical_sale_price = 119_000.0
sale_2022_price = 186_820.0
course_property_id = 391

# Locate the exact 2008 course-data record. Public records and the spreadsheet agree
# on the 1900 build year, 8,405-square-foot lot, April 2008 sale, and $119,000 price.
property_matches = clean.index[clean['Id'].eq(course_property_id)]
assert len(property_matches) == 1
historical_index = int(property_matches[0])
assert clean.loc[historical_index, 'YrSold'] == 2008
assert clean.loc[historical_index, 'MoSold'] == 4
assert clean.loc[historical_index, 'SalePrice'] == historical_sale_price
assert clean.loc[historical_index, 'Neighborhood'] == 'Edwards'
assert historical_index in test_idx
assert historical_index not in train_idx and historical_index not in validation_idx

historical_property_row = X_47.loc[[historical_index]].copy()
```

```

historical_property_row.index = ['2008 course-data record']

# Start from the exact historical property record. Update only fields that the current
# listing reports differently. The first-floor area remains 961 square feet; the
# current 1,262-square-foot total implies 301 square feet on the upper level.
zillow_row = historical_property_row.copy()
zillow_row.index = [zillow_address]
zillow_row.loc[zillow_address, '2ndFlrSF'] = 301
zillow_row.loc[zillow_address, 'GrLivArea'] = 1_262
zillow_row.loc[zillow_address, 'FullBath'] = 2 # one full plus one three-quarter bath
zillow_row.loc[zillow_address, 'BedroomAbvGr'] = 3 # fourth bedroom is in the basement

assert list(zillow_row.columns) == list(X_47.columns)
assert not zillow_row.isna().any().any()
assert zillow_row[neighbourhood_columns].sum(axis=1).eq(1).all()

changed_features = ['2ndFlrSF', 'GrLivArea', 'FullBath', 'BedroomAbvGr']
input_audit = pd.DataFrame({
    '2008_course_record': historical_property_row.iloc[0],
    'current_value_used': zillow_row.iloc[0],
})
input_audit['mapping'] = 'retained from exact 2008 property record'
input_audit.loc[changed_features, 'mapping'] = 'updated from current listing'
display(input_audit)

```

Output 1

	2008_course_record	current_value_used	mapping
LotArea	8405	8405	retained from exact 2008 property record
OverallQual	5	5	retained from exact 2008 property record
OverallCond	8	8	retained from exact 2008 property record
YearBuilt	1900	1900	retained from exact 2008 property record
YearRemodAdd	1950	1950	retained from exact 2008 property record
BsmtFinSF	632	632	retained from exact 2008 property record
BsmtUnfSF	229	229	retained from exact 2008 property record
TotalBsmtSF	861	861	retained from exact 2008 property record
1stFlrSF	961	961	retained from exact 2008 property record
2ndFlrSF	406	301	updated from current listing
GrLivArea	1367	1262	updated from current listing
FullBath	1	2	updated from current listing
HalfBath	0	0	retained from exact 2008 property record
BedroomAbvGr	4	3	updated from current listing
TotRmsAbvGrd	7	7	retained from exact 2008 property record
Fireplaces	0	0	retained from exact 2008 property record
GarageCars	1	1	retained from exact 2008 property record
GarageArea	384	384	retained from exact 2008 property record
WoodDeckSF	0	0	retained from exact 2008 property record
OpenPorchSF	130	130	retained from exact 2008 property record
EnclosedPorch	112	112	retained from exact 2008 property record
BsmtQual_ordinal	3	3	retained from exact 2008 property record
NBHD_Blmngtn	0	0	retained from exact 2008 property record
NBHD_Blueste	0	0	retained from exact 2008 property record
NBHD_BrDale	0	0	retained from exact 2008 property record
NBHD_BrkSide	0	0	retained from exact 2008 property record
NBHD_ClearCr	0	0	retained from exact 2008 property record

	2008_course_record	current_value_used	mapping
NBHD_CollgCr	0	0	retained from exact 2008 property record
NBHD_Crawfor	0	0	retained from exact 2008 property record
NBHD_Edwards	1	1	retained from exact 2008 property record
NBHD_Gilbert	0	0	retained from exact 2008 property record
NBHD_IDOTRR	0	0	retained from exact 2008 property record
NBHD_MeadowV	0	0	retained from exact 2008 property record
NBHD_Mitchel	0	0	retained from exact 2008 property record
NBHD_NAmes	0	0	retained from exact 2008 property record
NBHD_NPkVill	0	0	retained from exact 2008 property record
NBHD_NWAmes	0	0	retained from exact 2008 property record
NBHD_NoRidge	0	0	retained from exact 2008 property record
NBHD_NridgHt	0	0	retained from exact 2008 property record
NBHD_OldTown	0	0	retained from exact 2008 property record
NBHD_SWISU	0	0	retained from exact 2008 property record
NBHD_Sawyer	0	0	retained from exact 2008 property record
NBHD_SawyerW	0	0	retained from exact 2008 property record
NBHD_Somerst	0	0	retained from exact 2008 property record
NBHD_StoneBr	0	0	retained from exact 2008 property record
NBHD_Timber	0	0	retained from exact 2008 property record
NBHD_Veenker	0	0	retained from exact 2008 property record

Cell 20 | In [11]:

```
# The exact 2008 property record is in the test set, so the final model did
# not fit on this observation. First check its in-period historical prediction.
historical_ols_prediction = float(final_model.predict(historical_property_row)[0])

# Then apply the same fitted model to the current-listing feature mapping.
zillow_ols_prediction = float(final_model.predict(zillow_row)[0])

# Ridge alpha=0.60 is included as a stability comparison because its validation
# MSE is nearly identical and its coefficients are well behaved under collinearity.
zillow_ridge_model = make_log_target_model(Ridge(alpha=0.60))
zillow_ridge_model.fit(X_development, y_development)
historical_ridge_prediction = float(zillow_ridge_model.predict(historical_property_row)[0])
zillow_ridge_prediction = float(zillow_ridge_model.predict(zillow_row)[0])

historical_comparison = pd.DataFrame({
    'value_usd': [historical_sale_price, historical_ols_prediction, historical_ridge_prediction],
}, index=[
    '2008 actual sale price',
    'OLS prediction from 2008 features',
    'Ridge alpha=0.60 prediction from 2008 features',
])
historical_comparison['difference_from_2008_sale_usd'] = (
    historical_comparison['value_usd'] - historical_sale_price
)
historical_comparison['difference_from_2008_sale_pct'] = 100 * (
    historical_comparison['value_usd'] / historical_sale_price - 1
)
display(historical_comparison.round(2))

current_comparison = pd.DataFrame({
    'value_usd': [
        sale_2022_price,
        zillow_ols_prediction,
        zillow_ridge_prediction,
        zillow_list_price,
    ]
})
```

```

        zillow_zestimate,
    ], index=[
        '2022 sale price',
        'Current-feature OLS prediction',
        'Current-feature Ridge alpha=0.60 prediction',
        '2026 Zillow listing price',
        '2026 Zillow Zestimate',
    ])
current_comparison['difference_from_list_usd'] = current_comparison['value_usd'] - zillow_list_price
current_comparison['difference_from_list_pct'] = 100 * (
    current_comparison['value_usd'] / zillow_list_price - 1
)
display(current_comparison.round(2))

print(f'2008 actual sale: ${historical_sale_price:,.2f}')
print(f'OLS prediction from 2008 features: ${historical_ols_prediction:,.2f}')
print(f'Ridge prediction from 2008 features: ${historical_ridge_prediction:,.2f}')
print(f'2022 sale: ${sale_2022_price:,.2f}')
print(f'Current-feature OLS prediction: ${zillow_ols_prediction:,.2f}')
print(f'Current-feature Ridge sensitivity prediction: ${zillow_ridge_prediction:,.2f}')
print(f'2026 listing price: ${zillow_list_price:,.2f}')
print(f'2026 Zestimate: ${zillow_zestimate:,.2f}')

```

Output 1

	value_usd	difference_from_2008_sale_usd	difference_from_2008_sale_pct
2008 actual sale price	119000.00	0.00	0.00
OLS prediction from 2008 features	123195.25	4195.25	3.53
Ridge alpha=0.60 prediction from 2008 features	122873.90	3873.90	3.26

Output 2

	value_usd	difference_from_list_usd	difference_from_list_pct
2022 sale price	186820.00	-59680.00	-24.21
Current-feature OLS prediction	121868.72	-124631.28	-50.56
Current-feature Ridge alpha=0.60 prediction	121450.58	-125049.42	-50.73
2026 Zillow listing price	246500.00	0.00	0.00
2026 Zillow Zestimate	242000.00	-4500.00	-1.83

Output 3

```

2008 actual sale: $119,000.00
OLS prediction from 2008 features: $123,195.25
Ridge prediction from 2008 features: $122,873.90
2022 sale: $186,820.00
Current-feature OLS prediction: $121,868.72
Current-feature Ridge sensitivity prediction: $121,450.58
2026 listing price: $246,500.00
2026 Zestimate: $242,000.00

```

Interpretation of the Zillow comparison

Using the exact 2008 course-data features, the selected OLS model predicts approximately \$123,195, only \$4,195 (3.53%) above the property's actual 2008 sale price of \$119,000. Because this observation is in the held-out test set, that comparison is not an in-sample fitted value.

After updating the area, bathroom, and above-grade bedroom fields to the current listing, OLS predicts approximately \$121,869 and Ridge $\alpha=0.60$ predicts approximately \$121,451. Both are about 50.6% below the \$246,500 listing price, while the \$242,000 Zestimate is only \$4,500 (1.83%) below the listing price. The same parcel sold for \$186,820 in 2022.

The strong 2008 comparison and weak 2026 comparison isolate the main limitation: the course model is calibrated to nominal 2006--2010 prices and contains no sale-year, inflation, or housing-market index. Some current public fields also differ from the historical assessor record. The current estimate should therefore be interpreted as a historical-price-scale prediction rather than a present-day appraisal.